1492-IF-25/26

OPTIMASI MODEL K-NEAREST NEIGHBOR (KNN) DALAM MEMPREDIKSI TINGKAT KEPARAHAN KECELAKAAN LALU LINTAS

SKRIPSI

Oleh:

MUHAMMAD ALPI ASHARI
2209106017

FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN

SAMARINDA
2026

OPTIMASI MODEL K-NEAREST NEIGHBOR (KNN) DALAM MEMPREDIKSI TINGKAT KEPARAHAN KECELAKAAN LALU LINTAS

SKRIPSI

Diajukan sebagai salah satu syarat untuk menyelesaikan pendidikan
pada Program Studi Strata 1 Informatika,
Fakultas Teknik, Universitas Mulawarman

Oleh:

MUHAMMAD ALPI ASHARI
2209106017



FAKULTAS TEKNIK
UNIVERSITAS MULAWARMAN

SAMARINDA
2026

PERNYATAAN KEASLIAN SKRIPSI

Saya menyatakan dengan sesungguhnya bahwa skripsi dengan judul:

OPTIMASI MODEL K-NEAREST NEIGHBOR (KNN) DALAM MEMPREDIKSI TINGKAT KEPARAHAN KECELAKAAN LALU LINTAS

yang dibuat sebagai salah satu syarat untuk menyelesaikan pendidikan pada Program Studi S1 Informatika, Fakultas Teknik, Universitas Mulawarman, sejauh yang saya ketahui bukan merupakan tiruan atau duplikasi dari skripsi yang sudah dipublikasikan dan atau pernah dipakai untuk mendapatkan gelar kesarjanaan di lingkungan Universitas Mulawarman maupun di Perguruan Tinggi atau instansi manapun, kecuali bagian yang sumber informasinya dicantumkan sebagaimana mestinya.

Samarinda, Tgl bln thn

Materai

Muhammad Alpi Ashari
NIM. 2209106017

OPTIMASI MODEL K-NEAREST NEIGHBOR (KNN) DALAM MEMPREDIKSI TINGKAT KEPARAHAN KECELAKAAN LALU LINTAS

Oleh:

Muhammad Alpi Ashari
2209106017

Telah diujikan pada 20 Agustus 2026 dan dinyatakan telah memenuhi syarat

Samarinda, 3 Sepetember 2026

Disahkan oleh:

Pembimbing I,	Pembimbing II,
Ramadiani, S.Pd., M.Si., M.Kom., Ph.D	**Anton Prafanto, S.Kom., M.T**
NIP. 197211012001122001	NIP. 199310222019031016

Mengetahui,
Dekan Fakultas Teknik
Universitas Mulawarman

Prof. Dr. Ir. H. Tamrin, ST., MT., IPU., APEC. Eng
NIP. 19700227 200012 1 001

”Maka sesungguhnya bersama kesulitan ada kemudahan”
(*QS. Al Insyirah:5-6)*

”Jangan menyerah sebab keajaiban sering datang ketika kita hampir putus asa”
(QS. Al Kahfi:39)

”Aku tidak pernah satu kali pun menyesali diamku, tetapi aku berkali-kali menyesali bicaraku”
(Umar bin Khattab)

”Apa yang melewatkanku tidak akan menjadi takdirku, dan apa yang ditakdirkan untukku, tidak akan pernah melewatkanku”
(Umar bin Khattab)

”Keberhasilan bukanlah milik orang yang pintar, keberhasilan adalah milik mereka yang senantiasa berusaha”
(BJ Habibie)

”Sesuatu yang tidak diperjuangkan, tidak mungkin ada kemenangan”
(Penulis)

”Aku membahayakan nyawa ibu untuk hadir kedunia, dan aku membuat ayah bekerja keras hingga lelah, jadi aku pastikan, tidak mungkin aku tidak ada artinya”
(Penulis)

RIWAYAT HIDUP



Muhammad Alpi Ashari lahir di Kecamatan Sangkulirang, Kabupaten Kutai Timur, Provinsi Kalimantan Timur pada tanggal 20 April 2004. Merupakan anak keenam dari enam bersaudara, Putra dari Bapak Tajuddin dan Ibu Harisa. Penulis menyelesaikan pendidikannya di SD Negeri 007 Sangkulirang pada tahun 2016, SMP Negeri 1 Sangkulirang pada tahun 2019 dan SMA Negeri 1 Sangkulirang pada tahun 2022. Setelah lulus SMA hingga pada penulisan skripsi ini , penulis masih terdaftar sebagai sebagai mahasiswa Program Studi S1 Informatika, Fakultas Teknik, Universitas Mulawarman di Kota Samarinda, Provinsi Kalimantan Timur. Pada tanggal 01 Juli – 01 September 2025 penulis melaksanakan kegiatan Praktik Kerja Lapangan (PKL) di CV Aras Jaya Berkah, Kecamatan Samarinda Ulu, Kalimantan Timur dan pada 22 November 2025 - 20 Agustus 2026 penulis melakukan pembuatan Skripsi. Kegiatan organisasi yang prenah diikuti selama masa perkuliahan yaitu Association Informatics (AI) sebagai anggota Bidang Pengembangan Sumberdaya Manusia (BPSDM) pada tahun 2024.

Muhammad Alpi Ashari
2209106017
Program Studi Informatika

Dosen Pembimbing
I. Ramadiani, S.Pd., M.Si., M.Kom., Ph.D
II. Anton Prafanto, S.Kom., M.T

OPTIMASI MODEL K-NEAREST NEIGHBOR (KNN) DALAM MEMPREDIKSI TINGKAT KEPARAHAN KECELAKAAN LALU LINTAS

ABSTRAK

Penelitian ini bertujuan untuk menganalisis dan mengoptimalkan kinerja metode K-Nearest Neighbors (KNN) dalam memprediksi tingkat keparahan kecelakaan lalu lintas menggunakan dataset *Road Traffic Severity Classification* yang diperoleh dari Kaggle. Dataset yang digunakan berbentuk data tabular dengan berbagai atribut yang merepresentasikan kondisi kecelakaan lalu lintas. Tahapan penelitian dimulai dari preprocessing data yang meliputi pembersihan data, penanganan missing value, transformasi data kategorikal, serta normalisasi data agar siap digunakan dalam proses pemodelan. Selanjutnya dilakukan pengujian parameter menggunakan K-Fold Cross Validation, jumlah tetangga (K), metric jarak, dan weight pada algoritma KNN. Penelitian ini juga menerapkan teknik Synthetic Minority Oversampling Technique (SMOTE) untuk mengatasi ketidakseimbangan data. Seluruh proses preprocessing dan pemodelan dilakukan menggunakan pipeline untuk menghindari data leakage selama proses validasi. Evaluasi model dilakukan menggunakan confusion matrix dan classification report dengan metrik accuracy, precision, recall, dan F1-Score. Model terbaik pada penelitian ini adalah Grid Search CV Non-SMOTE dengan Macro F1-Score sebesar 39,216% karena dinilai lebih mampu mengenali seluruh kelas dibandingkan model lainnya.

Kata Kunci : K-Nearest Neighbors, SMOTE, K-Fold Cross Validation, Grid Search, Klasifikasi, Kecelakaan Lalu Lintas

Muhammad Alpi Ashari
2209106017
Informatics Study Program

Supervisors
I. Ramadiani, S.Pd., M.Si., M.Kom., Ph.D
II. Anton Prafanto, S.Kom., M.T

Optimization of the K-Nearest Neighbor (KNN) Model in Predicting Traffic Accident Severity

ABSTRACT

This study aims to analyze and optimize the performance of the K-Nearest Neighbors (KNN) method in predicting road traffic accident severity using the Road Traffic Severity Classification dataset obtained from Kaggle. The dataset consists of tabular data containing various attributes representing traffic accident conditions. The research process began with data preprocessing, including data cleaning, handling missing values, categorical data transformation, and data normalization to prepare the dataset for modeling. Furthermore, parameter tuning was conducted using K-Fold Cross Validation, variations of the number of neighbors (K), distance metrics, and weight parameters in the KNN algorithm. This study also applied the Synthetic Minority Oversampling Technique (SMOTE) to address the problem of imbalanced data. All preprocessing and modeling stages were implemented using a pipeline to prevent data leakage during validation. Model evaluation was carried out using confusion matrix and classification report with accuracy, precision, recall, and F1-Score metrics. The best model in this study was the Grid Search CV Non SMOTE model with a Macro F1-Score of 39,216%, as it was considered more capable of recognizing all classes compared to the other models.

Keywords : K-Nearest Neighbors, SMOTE, K-Fold Cross Validation, Grid Search, Klasifikasi, Road Traffic Accidents

KATA PENGANTAR

Puji syukur kepada Allah SWT, Tuhan Yang Maha Esa sehingga dapat menyelesaikan skripsi dengan judul **"OPTIMASI MODEL K-NEAREST NEIGHBOR (KNN) DALAM MEMPREDIKSI TINGKAT KEPARAHAN KECELAKAAN LALU LINTAS**". Skripsi ini disusun sebagai salah satu tahapan dalam menyelesaikan skripsi pada Fakultas Teknik, Universitas Mulawarman. Oleh karena itu, pada kesempatan ini kami ingin mengucapkan terima kasih kepada semua pihak yang telah mendukung serta membantu saya selama proses penyusunan skripsi, kepada:

1. Orang tua dan Saudara-saudara saya atas do'a, bimbingan serta kasih sayangnya.
2. Bapak Prof. Dr. Ir. H. Tamrin, S.T., M.T., IPU, APEC Eng. Selaku Dekan Fakultas Teknik, Universitas Mulawarman.
3. Bapak Awang Harsa Kridalaksana, S.Kom., M.Kom. Selaku Koordinator Program Studi Informatika.
4. Ramadiani, S.Pd., M.Si., M.Kom., Ph.D Selaku Pembimbing I yang selalu memberikan arahan dan masukkan terhadap penelitian ini.
5. Anton Prafanto, S.Kom., M.T selaku Pembimbing II yang selalu memberikan arahan dan masukkan terhadap penelitian ini.
6. Medi Taruk, M.Cs selaku Penguji I atas saran dan masukkan terhadap penelitian ini.
7. Ir. Novianti Puspitasari, S.Kom., M.Eng selaku Penguji II atas saran dan masukkan terhadap penelitian ini.
8. Segenap Dosen Program Studi Informatika, yang telah memberikan ilmu pengetahuan selama mengikuti perkuliahan.
9. Rekan-rekan seperjuangan yang terus memberikan dukungan semangat demi terselesainya tugas ini.

Saya menyadari bahwa skripsi ini tidak luput dari berbagai kekurangan. Oleh karena itu, semua kritik dan saran yang bersifat memperbaiki demi kesempurnaan sangat diharapkan.

Samarinda, 11 Juni 2026

Penulis

DAFTAR ISI

halaman

HALAMAN JUDUL ii
PERNYATAAN KEASLIAN SKRIPSI iii
HALAMAN PENGESAHAN iv
HALAMAN PERSEMBAHAN v
RIWAYAT HIDUP vi
ABSTRAK vii
ABSTRACT viii
KATA PENGANTAR ix
DAFTAR ISI xi
DAFTAR TABEL xiii
DAFTAR GAMBAR xv
DAFTAR LAMPIRAN xvi
DAFTAR ISTILAH/LAMBANG xvii
DAFTAR SINGKATAN xix

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1
1.2 Rumusan Masalah 3
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 4
1.5 Manfaat Penelitian 4

BAB II TINJAUAN PUSTAKA 6

2.1 Penelitian Terkait 6
2.2 Kecelakaan Lalu Lintas 17
2.3 Machine Learning 17
2.4 *Supervised Learning* 18
2.5 *Python* 19
2.6 *Streamlit* 19
2.7 Pandas 20
2.8 *Scikit-Learn* 20
2.9 *Imbalanced-Learn* 21
2.10 *K-Nearest Neighbor* (KNN) 21
2.11 SMOTE 22
2.12 *Hyperparameter Tuning* 23
2.13 *Grid Search* CV 23
2.14 *K-Fold Cross Validation* 24
2.15 *Dummy Baseline* 25

2.16 *Pipeline* 25
2.17 *Confusion Matrix Multi Class* 26

BAB III METODOLOGI PENELITIAN 29

3.1 Tahapan Pelaksanaan Penelitian 29
3.2 Pengumpulan Data 30
3.3 Perancangan Data 32
3.4 Perancangan Proses/Algoritma 35
3.5 Perancangan Tampilan 39
3.6 Perancangan Pengujian 42

BAB IV HASIL DAN PEMBAHASAN 45

4.1 Pengolahan Data 45
4.2 Penerapan Proses 45
4.3 Penerapan Tampilan 58
4.4 Hasil Pengujian 63
4.5 Pembahasan 84

BAB V KESIMPULAN DAN SARAN 88

5.1 Kesimpulan 88
5.2 Saran 89

DAFTAR PUSTAKA 91
LAMPIRAN 95

DAFTAR TABEL

Halaman

Tabel 2. 1 Penelitian Terkait 15
Tabel 2. 2 Confusion Matrix Multiclass 27
Tabel 3. 1 Deskripsi Atribut Dataset 31
Tabel 3. 2 Contoh Dataset Tingkat Keparahan Kecelakaan Lalu Lintas 32
Tabel 3. 3 Fitur Waktu (*Temporal*) 33
Tabel 3. 4 Fitur Cuaca (*Weather*) 33
Tabel 3. 5 Fitur Pengemudi (*Driver*) 34
Tabel 3. 6 Fitur Kondisi Jalan (*Road Condition*) 34
Tabel 3. 7 Fitur Jenis Kendaraan (*Vehicle*) 34
Tabel 3. 8 Fitur Lingkungan (*Environtment*) 35
Tabel 3. 9 Fitur Kecelakaan (*Accident Detail*) 35
Tabel 3. 10 Fitur Korban (*Casualty*) 35
Tabel 4. 1 Hasil Penanganan Nilai Null 46
Tabel 4. 2 Hasil Penanganan Data Duplikat 47
Tabel 4. 3 Hasil Penanganan *Outlier* 47
Tabel 4. 4 Contoh Hasil Transformasi Data 48
Tabel 4. 5 Contoh Hasil Normalisasi Data 49
Tabel 4. 6 Data Hasil *Preprocessing* 50
Tabel 4. 7 Jumlah data Skema Data tanpa SMOTE 51
Tabel 4. 8 Data Sampel SMOTE 52
Tabel 4. 9 Perhitungan SMOTE 52
Tabel 4. 10 Data Hasil SMOTE 53
Tabel 4. 11 Jumlah Data Kecelakaan Setelah SMOTE 53
Tabel 4. 12 Hasil Jumlah data pembagian K-Fold K = 5 54
Tabel 4. 13 Data KNN 55
Tabel 4. 14 Hasil Perhitungan Jarak Euclidean 56
Tabel 4. 15 Hasil Perhitungan Manhattan Distance 57
Tabel 4. 16 Hasil Pengujian Kfold KNN Non SMOTE 65
Tabel 4. 17 Hasil Pengujian Kfold KNN SMOTE 66
Tabel 4. 18 Hasil Pengujian K tetangga KNN Non SMOTE 67
Tabel 4. 19 Hasil Pengujian K Tetangga KNN SMOTE 67
Tabel 4. 20 Hasil Pengujian Metrik KNN Non SMOTE 68
Tabel 4. 21 Hasil Pengujian Metric Jarak KNN SMOTE 69
Tabel 4. 22 Hasil Pengujian Bobot KNN Non SMOTE 70
Tabel 4. 23 Hasil Pengujian Bobot KNN SMOTE 70
Tabel 4. 24 Hasil Classification Report KNN Manual Non SMOTE 73
Tabel 4. 25 Classification Report KNN Manual SMOTE 75
Tabel 4. 26 Classification Report KNN Grid Search CV (Non SMOTE) 77
Tabel 4. 27 Confusion Matrix KNN Grid Search CV (SMOTE) 79

Tabel 4. 28 Confusion Matrix *Dummy Baseline*..... 81
Tabel 4. 29 Perbandingan Performa seluruh model..... 82

DAFTAR GAMBAR

Halaman

Gambar 3. 1 Tahapan Pelaksanaan Penelitian 29
Gambar 3. 2 Tahapan Proses 36
Gambar 3. 3 Perancangan Tampilan 40
Gambar 4. 1 Halaman Dataset 59
Gambar 4. 2 Halaman Preprocessing 60
Gambar 4. 3 Halaman EDA 61
Gambar 4. 4 Halaman Hasil Pengujian 62
Gambar 4. 5 Halaman Evaluasi Model 63
Gambar 4. 6 Confusion Matrix KNN (Non SMOTE) 72
Gambar 4. 7 Confusion Matrix KNN (SMOTE) 74
Gambar 4. 8 Confusion Matrix KNN GridSearch CV (Non SMOTE) 76
Gambar 4. 9 Grafik Perbandingan seluruh Model 82

DAFTAR LAMPIRAN

halaman

Lampiran 1 Arsip Keseluruhan 95

DAFTAR ISTILAH/LAMBANG

	Makna/Arti
Machine Learning	Cabang kecerdasan buatan yang memungkinkan komputer belajar dari data tanpa diprogram secara eksplisit.
Supervised Learning	Metode pembelajaran mesin yang menggunakan data berlabel untuk melatih model.
Severity	Tingkat keparahan suatu kejadian atau kondisi berdasarkan kategori tertentu.
Synthetic Minority Over-sampling Technique (SMOTE)	Teknik penyeimbangan data dengan membuat data sintetis pada kelas minoritas.
Hyperparameter Tuning	Proses pencarian parameter terbaik untuk meningkatkan kinerja model.
Grid Search CV	Metode pencarian hyperparameter dengan menguji seluruh kombinasi parameter menggunakan cross validation.
Library	Kumpulan fungsi atau modul siap pakai untuk mempermudah pengembangan program.
Confusion Matrix	Tabel evaluasi yang menunjukkan perbandingan hasil prediksi dan data aktual.
Real-time	Kondisi pemrosesan data yang dilakukan secara langsung tanpa jeda waktu signifikan.
Deployment	Proses penerapan model atau aplikasi ke lingkungan produksi agar dapat digunakan.
Support Vector Machine	Algoritma klasifikasi yang mencari hyperplane terbaik untuk memisahkan data.
Bootstrap Aggregating	Teknik ensemble yang menggabungkan beberapa model untuk meningkatkan akurasi prediksi.
Radial Basis Function	Fungsi kernel yang digunakan untuk menangani data non-linear.
Graph Neural Network	Jaringan saraf yang dirancang untuk memproses data berbentuk graf.
Convolutional Neural Network	Jaringan saraf yang umum digunakan untuk pengolahan citra dan data visual.
Ant Colony Optimization	Algoritma optimasi yang meniru perilaku semut dalam mencari jalur terbaik.
Genetic Algorithm	Algoritma optimasi berbasis proses evolusi biologis seperti seleksi dan mutasi.

Harmony Search	Algoritma optimasi yang meniru proses improvisasi dalam musik.
Univariate Feature Selection	Teknik pemilihan fitur berdasarkan pengujian statistik pada tiap fitur secara individu.
Logistic Regression	Algoritma klasifikasi yang memprediksi probabilitas suatu kelas.
RandomizedSearch CV	Metode pencarian hyperparameter dengan pengambilan kombinasi parameter secara acak.
Multilayer Perceptron	Jaringan saraf tiruan dengan satu atau lebih lapisan tersembunyi.
K-Nearest Neighbor (KNN)	Algoritma klasifikasi yang menentukan kelas berdasarkan mayoritas K tetangga terdekat.
Random Forest	Algoritma ensemble berbasis banyak decision tree untuk meningkatkan akurasi.
Extreme Gradient Boosting	Algoritma boosting yang mengoptimalkan performa dan kecepatan pelatihan model.
Unsupervised Learning	Metode pembelajaran mesin tanpa menggunakan data berlabel.
Reinforcement Learning	Metode pembelajaran mesin berbasis interaksi dan pemberian reward.
K-Fold Cross Validation	Teknik evaluasi model dengan membagi data menjadi K bagian secara bergantian.
Naive Bayes	Algoritma klasifikasi probabilistik berdasarkan Teorema Bayes.
Output	Hasil akhir yang dihasilkan oleh suatu sistem atau model.
Confusion Matrix Multiclass	Confusion matrix yang digunakan untuk evaluasi klasifikasi dengan lebih dari dua kelas.
Weights	Nilai bobot yang menentukan pengaruh fitur dalam suatu model.

DAFTAR SINGKATAN

	Arti
KNN	*K-Nearest Neighbor*
SVM	*Support Vector Machine*
ADASYN	*Adaptive Synthetic Sampling Approach for Imbalanced Learning*
CNN	*Convolutional Neural Network*
ACO	*Ant Colony Optimization*
GA	*Genetic Algorithm*
GNN	*Graph Neural Network*
BPS	*Badan Pusat Statistik*
RBF	*Radial Basis Function*
HS	*Harmony Search*
LR	*Logistic Regression*
MLP	*Multilayer Perceptron*
RF	*Random Forest*
XGBoost	*Extreme Gradient Boosting*
SMOTE	*Synthetic Minority Oversampling Technique*
TP	*True Positive*
TN	*True Negative*
FP	*False Positive*
FN	*False Negative*

BAB I
PENDAHULUAN

1.1 Latar Belakang

Tingkat keparahan kecelakaan lalu lintas merupakan indikator yang digunakan untuk menggambarkan dampak yang ditimbulkan akibat suatu kecelakaan, baik terhadap korban maupun kerugian yang terjadi. Fenomena kecelakaan lalu lintas masih menjadi permasalahan yang dihadapi oleh berbagai negara di dunia karena menyebabkan tingginya angka korban luka maupun korban meninggal setiap tahunnya. Tingkat keparahan kecelakaan umumnya diklasifikasikan menjadi tiga kategori, yaitu *Slight Injury* (cedera ringan), *Serious Injury* (cedera serius), dan *Fatal Injury* (cedera fatal). Klasifikasi tersebut penting karena dapat digunakan untuk mengidentifikasi tingkat risiko kecelakaan serta menjadi dasar dalam penyusunan strategi keselamatan lalu lintas (Syahriza, 2019).

Kecelakaan lalu lintas merupakan salah satu permasalahan serius yang berdampak besar terhadap keselamatan manusia serta menimbulkan kerugian ekonomi dan sosial. Kecelakaan dapat menyebabkan luka ringan hingga berat, kecacatan permanen, bahkan kematian, sehingga menjadi perhatian utama dalam bidang transportasi. Tingginya angka kecelakaan lalu lintas menunjukkan bahwa diperlukan upaya yang lebih sistematis dan berbasis data untuk memahami faktor penyebab serta tingkat keparahan kecelakaan yang terjadi (C. A. Salsabila et al., 2025).

Tingkat keparahan kecelakaan lalu lintas dipengaruhi oleh berbagai faktor, seperti kondisi jalan, cuaca, waktu kejadian, karakteristik kendaraan, dan pengemudi. Kompleksitas hubungan antar faktor tersebut menyebabkan prediksi tingkat keparahan kecelakaan menjadi sulit dilakukan dengan metode analisis konvensional. Prediksi tingkat keparahan kecelakaan penting dilakukan untuk membantu pihak terkait dalam mengambil langkah pencegahan, meningkatkan kesiapan dalam penanganan korban, serta mendukung perencanaan kebijakan keselamatan lalu lintas yang lebih efektif. Oleh karena itu, diperlukan pendekatan Machine Learning yang mampu mengolah data dalam

jumlah besar dan menemukan pola yang kompleks untuk menghasilkan prediksi yang lebih optimal (N. A. Salsabila, 2023).

Seiring dengan keterbatasan analisis konvensional tersebut, penerapan *Machine Learning* menjadi solusi yang relevan dalam analisis kecelakaan lalu lintas. *Machine Learning* mampu mempelajari pola dari data historis dan menghasilkan model prediksi yang adaptif terhadap kompleksitas data. Dalam bidang keselamatan lalu lintas, penerapan *Machine Learning* diharapkan dapat membantu memprediksi tingkat keparahan kecelakaan secara lebih akurat sehingga dapat mendukung upaya mitigasi risiko dan peningkatan keselamatan jalan (Fauzan et al., 2024).

Salah satu pendekatan dalam *Machine Learning* yang banyak digunakan adalah *Supervised learning*, yaitu metode pembelajaran yang memanfaatkan data berlabel sebagai dasar pembentukan model prediksi. Pendekatan ini dinilai efektif karena mampu mempelajari hubungan antara variabel input dan target secara langsung berdasarkan data yang tersedia. Berbagai algoritma *Supervised learning* telah diterapkan dalam bidang transportasi untuk melakukan klasifikasi maupun prediksi tingkat keparahan kecelakaan lalu lintas (Sinaga et al., 2025).

K-Nearest Neighbor (KNN) merupakan salah satu algoritma *Supervised learning* yang sering digunakan karena konsepnya yang sederhana dan kemampuannya dalam menangani data dengan banyak atribut. Algoritma KNN bekerja dengan menentukan kelas suatu data berdasarkan kedekatannya dengan sejumlah data lain yang memiliki kemiripan tertinggi berdasarkan ukuran jarak tertentu. KNN telah banyak diterapkan dalam berbagai penelitian karena fleksibilitasnya serta kemampuannya menghasilkan performa yang cukup baik dalam permasalahan klasifikasi (Febrianto et al., 2024).

Penelitian terdahulu menunjukkan bahwa penerapan *Machine Learning* dalam klasifikasi dan prediksi kecelakaan lalu lintas mampu memberikan hasil yang cukup akurat. Penelitian oleh N. A. Salsabila, (2023) mengklasifikasikan tingkat keparahan korban kecelakaan lalu lintas dan membuktikan bahwa algoritma *Supervised learning* dapat menghasilkan prediksi yang kompetitif. Penelitian lain oleh Fauzan et al., (2024) juga menunjukkan bahwa pendekatan berbasis data efektif dalam memprediksi klasifikasi kecelakaan lalu lintas.

Selain itu, penelitian oleh Sinaga et al., (2025) menunjukkan bahwa performa model *Machine Learning* dalam bidang transportasi sangat dipengaruhi oleh pemilihan algoritma dan parameter yang digunakan. Penelitian lain oleh Salsabila dan rekan juga menegaskan bahwa konfigurasi dan parameter pada algoritma klasifikasi, termasuk KNN, memiliki pengaruh signifikan terhadap kinerja model. Oleh karena itu, penelitian ini berfokus pada optimasi algoritma KNN dalam memprediksi tingkat keparahan kecelakaan lalu lintas guna memperoleh hasil yang lebih optimal dan akurat.

1.2 Rumusan Masalah

Berdasarkan latar belakang penelitian, maka yang menjadi rumusan masalah dalam penelitian ini adalah "Bagaimana mengoptimalkan kinerja model KNN dalam memprediksi tingkat keparahan kecelakaan lalu lintas? ".

1.3 Batasan Masalah

Penelitian ini disusun berdasarkan data yang diperoleh dari website Kaggle, yaitu dataset Road Traffic Severity Classification tahun 2023 dengan jumlah data sebanyak 12.316 data kecelakaan. Target yang digunakan dalam penelitian ini adalah variabel Accident_severity yang bersifat kategorikal dan telah terlabel dalam dataset, terdiri atas tiga kelas, yaitu Slight Injury (kecelakaan dengan tingkat cedera ringan), Serious Injury (kecelakaan dengan tingkat cedera serius), dan Fatal Injury (kecelakaan dengan tingkat cedera fatal). Prediksi tingkat keparahan kecelakaan dilakukan berdasarkan atribut-atribut yang tersedia dalam dataset, meliputi kondisi cuaca, kondisi jalan, pencahayaan, waktu kejadian, karakteristik kendaraan, serta karakteristik pengemudi. Model klasifikasi yang digunakan dalam penelitian ini adalah K-Nearest Neighbors (KNN) dengan optimasi parameter menggunakan Hyperparameter Tuning berbasis Grid Search Cross Validation (Grid Search CV) serta penerapan metode Synthetic Minority Over-sampling Technique (SMOTE) untuk menangani ketidakseimbangan kelas. Evaluasi kinerja model dilakukan menggunakan metrik Accuracy, Precision, Recall, F1-Score, dan Confusion Matrix. Penelitian ini hanya berfokus pada proses pemodelan dan evaluasi performa model

sehingga tidak mencakup analisis secara *real-time* maupun implementasi (*deployment*) model ke dalam sistem produksi.

1.4 Tujuan Penelitian

Tujuan yang ingin dicapai dalam penelitian ini adalah untuk mengoptimalkan kinerja model KNN dalam memprediksi tingkat keparahan kecelakaan lalu lintas.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat pada berbagai pihak, khususnya:

1. Penulis

   Penelitian ini diharapkan dapat bermanfaat bagi penulis untuk mengembangkan wawasan dan kemampuan dalam mengimplementasikan serta menerapkan algoritma KNN dalam memprediksi tingkat keparahan kecelakaan lalu lintas berbasis data.

2. Mahasiswa

   Penelitian ini dapat memberikan pengetahuan kepada mahasiswa mengenai bagaimana penerapan metode *Machine Learning*, khususnya algoritma KNN, dilakukan dalam proses prediksi tingkat kecelakaan, serta dapat menjadi referensi bagi mahasiswa di bidang Informatika dalam mendukung proses pembelajaran dan penelitian sejenis.

3. Instansi Kepolisian

   Manfaat yang diharapkan bagi instansi kepolisian adalah dengan adanya hasil analisis prediksi tingkat keparahan kecelakaan lalu lintas, diharapkan dapat membantu pihak kepolisian dalam menganalisis pola kecelakaan, mendukung pengambilan keputusan, serta meningkatkan upaya pencegahan dan keselamatan lalu lintas.

1.6 Kontribusi Penelitian

Kontribusi yang dapat diberikan dari penelitian ini antara lain:

1. Bidang Informatika

Penelitian ini diharapkan dapat menambah wawasan mahasiswa dalam penerapan algoritma *Machine Learning*, khususnya KNN, dalam melakukan klasifikasi untuk memprediksi tingkat keparahan kecelakaan lalu lintas.

2. Bidang Keselamatan Lalu Lintas

   Penelitian ini diharapkan dapat memberikan gambaran awal mengenai bagaimana pemanfaatan data kecelakaan lalu lintas dapat digunakan untuk memprediksi tingkat keparahan kecelakaan sebagai upaya pendukung dalam analisis dan peningkatan keselamatan lalu lintas

BAB II
TINJAUAN PUSTAKA

2.1 Penelitian Terkait

Kajian mengenai penelitian-penelitian terkait yang telah dilakukan sebelumnya diperlukan dalam rangka mendukung penelitian ini. Daftar penelitian terkait antara lain:

1. Penelitian terkait optimasi kinerja model *Machine Learning* dalam klasifikasi tingkat keparahan kecelakaan lalu lintas. **Metode yang digunakan:** *Support Vector Machine* (SVM) dan *Bootstrap Aggregating* (Bagging) dengan penanganan data tidak seimbang. **Temuan Penelitian:** Penelitian ini bertujuan untuk menganalisis dan membandingkan kinerja algoritma SVM dan metode ensemble Bagging dalam mengklasifikasikan tingkat keparahan korban kecelakaan lalu lintas di Kabupaten Lombok Timur. Dataset yang digunakan berasal dari data kecelakaan lalu lintas yang mencakup berbagai atribut seperti kondisi kejadian, karakteristik kendaraan, serta informasi korban, dengan label tingkat keparahan korban yang terdiri dari beberapa kelas. Pada tahap preprocessing, data melalui proses pembersihan data kosong, transformasi data kategorikal ke dalam bentuk numerik, serta normalisasi data. Mengingat distribusi data antar kelas tidak seimbang, penelitian ini menerapkan teknik penanganan imbalance class seperti under sampling dan *Synthetic Minority Oversampling Technique* (SMOTE). Setelah tahap preprocessing, data dibagi menjadi data training dan data testing dengan perbandingan 80% data pelatihan dan 20% data pengujian. Evaluasi model dilakukan menggunakan metrik akurasi serta analisis *Confusion Matrix* untuk melihat kemampuan model dalam mengklasifikasikan setiap kelas tingkat keparahan. Hasil penelitian menunjukkan bahwa model SVM dengan teknik under sampling menghasilkan akurasi tertinggi sebesar 93,66%, sedangkan model Bagging dengan penerapan SMOTE memperoleh akurasi sebesar 87,97%. Analisis *Confusion Matrix* menunjukkan bahwa model SVM lebih stabil dalam mengklasifikasikan kelas mayoritas maupun minoritas

dibandingkan Bagging, sehingga disimpulkan bahwa SVM lebih optimal dalam memprediksi tingkat keparahan kecelakaan lalu lintas pada dataset yang digunakan (Susanti et al., 2024).

2. Penelitian terkait optimasi kinerja model *Machine Learning* dalam mengidentifikasi tingkat keparahan kecelakaan lalu lintas. **Metode yang digunakan:** *Support Vector Machine* (SVM) multiclass dengan pendekatan one-against-one serta perbandingan dua fungsi kernel (Linear dan RBF). **Temuan Penelitian:** Penelitian ini bertujuan untuk menganalisis efektivitas metode Support Vector Machine dalam mengklasifikasikan tingkat keparahan korban kecelakaan lalu lintas dengan kategori luka ringan, luka berat, dan meninggal dunia berdasarkan data kecelakaan yang dialami pengendara sepeda motor di Kota Padang pada periode Januari–Maret 2022. Dalam penelitian ini, data yang digunakan berasal dari databse Badan Pusat Statistik (BPS) dan diproses melalui tahap pra-pemrosesan termasuk pengelompokan level keparahan serta transformasi fitur yang relevan untuk kebutuhan model. Selanjutnya, model SVM multiclass dikembangkan menggunakan dua fungsi kernel berbeda, yaitu kernel Linear dan kernel *Radial Basis Function* (RBF), dan diuji menggunakan metrik evaluasi klasifikasi seperti akurasi, APER, serta F1-score untuk masing-masing kategori kelas. Hasil penelitian menunjukkan bahwa SVM dengan kernel RBF memberikan performa yang lebih baik dibanding kernel Linear, dengan tingkat akurasi sebesar 94,62%, nilai APER sebesar 5,38%, serta nilai F1-score untuk kategori luka ringan 97,07%, luka berat 59,90%, dan kategori meninggal dunia 80% (Rahmi et al., 2024).

3. Penelitian terkait optimasi kinerja model *Machine Learning* dalam klasifikasi tingkat keparahan kecelakaan lalu lintas dengan penanganan imbalance class. **Metode yang digunakan:** Support Vector Machine (SVM), Forward Selection, dan ADASYN (Adaptive Synthetic Sampling Approach for Imbalanced Learning). **Temuan Penelitian:** Penelitian ini bertujuan untuk mengevaluasi performa model Support Vector Machine dalam mengklasifikasikan data kecelakaan lalu lintas dengan menerapkan teknik seleksi fitur Forward Selection dan teknik penanganan ketidakseimbangan kelas ADASYN untuk meningkatkan

akurasi prediksi. Data yang digunakan dalam penelitian ini mencakup kasus kecelakaan lalu lintas di Indonesia, yang menunjukkan tren peningkatan insiden pada tahun-tahun terakhir, dan mencakup berbagai variabel yang berkaitan dengan kejadian kecelakaan (misalnya kondisi jalan, faktor cuaca, jenis kendaraan, dan kondisi korban). Dalam proses penelitian, data awal terlebih dahulu mengalami tahapan pra-pemrosesan termasuk pembersihan data dan transformasi variabel, kemudian diterapkan Forward Selection untuk memilih fitur paling relevan terhadap target klasifikasi. Selanjutnya, dataset yang memiliki distribusi kelas tidak seimbang mendapat penanganan melalui metode ADASYN untuk mensintesis contoh minoritas sehingga memperbaiki keseimbangan antar kelas. Setelah tahapan data siap, model SVM dilatih dan diuji dengan menggunakan dataset terpisah antara data training dan data testing. Hasil penelitian menunjukkan bahwa kombinasi Forward Selection dan ADASYN meningkatkan performa klasifikasi SVM dibandingkan SVM tanpa teknik imbalance, dengan evaluasi akurasi model yang lebih tinggi serta peningkatan kemampuan dalam memprediksi kelas kelas minoritas dalam dataset. Temuan ini menunjukkan bahwa penggunaan teknik pemilihan fitur dan penanganan imbalance dapat secara signifikan meningkatkan kinerja model SVM dalam mengklasifikasikan kecelakaan lalu lintas berbasis data *Machine Learning* (Ramadhani et al., 2025).

4. Penelitian terkait optimasi kinerja model *Machine Learning* dalam klasifikasi tingkat keparahan kecelakaan lalu lintas. **Metode yang digunakan:** *K-Nearest Neighbor* (KNN) **Temuan Penelitian:** Penelitian ini bertujuan untuk mengklasifikasikan tingkat keparahan kecelakaan lalu lintas di Kota Samarinda dengan menggunakan model K-Nearest Neighbor, yang merupakan algoritme berbasis jarak yang efektif dalam tugas klasifikasi data. Dataset penelitian diperoleh dari Kepolisian Sektor Samarinda Kota dan mencakup berbagai atribut yang berpengaruh terhadap kejadian kecelakaan, seperti kondisi cahaya, tipe dan kelas jalan, serta batas kecepatan kendaraan. Dalam prosesnya, data terlebih dahulu diproses dan kemudian dibagi menjadi beberapa kelompok melalui teknik *K-Fold Cross Validation* untuk pelatihan dan pengujian model agar hasil evaluasi

tidak bias terhadap satu pasangan train-test saja. Evaluasi model menggunakan metrik akurasi menunjukkan bahwa dengan pemilihan parameter optimal, model KNN mampu mencapai akurasi sebesar 92,54%, sehingga menunjukkan bahwa metode KNN dapat digunakan secara efektif untuk mengklasifikasikan tingkat kecelakaan lalu lintas dan menjadi referensi bagi otoritas terkait dalam peningkatan keselamatan jalan (Anitasari et al., 2025).

5. Penelitian terkait optimasi kinerja model *Machine Learning* melalui penyesuaian hyperparameter model klasifikasi. **Metode yang digunakan:** *Support Vector Machine* (SVM) dan *GridSearchCV* untuk hyperparameter tuning. **Temuan Penelitian:** Penelitian ini bertujuan untuk meningkatkan performa model stance detection dalam memprediksi keberpihakan teks tweet terhadap Undang-Undang Cipta Kerja dengan menggunakan teknik penyesuaian hyperparameter melalui GridSearchCV yang diimplementasikan dalam kerangka kerja CRISP-DM. Dataset penelitian berasal dari data tweet Twitter yang telah dianotasi manual dan diproses melalui tahap preprocessing teks lalu dibagi menjadi data pelatihan dan pengujian menggunakan rasio 80:20. Model-model yang diuji mencakup beberapa konfigurasi SVM dengan berbagai kombinasi hyperparameter seperti C dan jumlah fitur TF-IDF. Evaluasi menggunakan 10-Fold Cross Validation serta micro f1-score menunjukkan bahwa model SVM terbaik memiliki nilai micro f1-score sebesar 78,4%, meningkat sekitar 6,6% dibandingkan model sebelumnya tanpa tuning hyperparameter, dan menunjukkan bahwa model tidak mengalami overfitting dengan selisih skor yang kecil antara hasil pada train-set dan test-set. *Confusion Matrix* atau classification report dalam penelitian ini juga menunjukkan bahwa metrik micro f1-score pada model terbaik mencerminkan kombinasi dari precision, recall, dan akurasi yang seimbang dalam klasifikasi multiclass teks tersebut (Nababan & Hutagalung, 2023).
6. Penelitian terkait optimasi kinerja model *Machine Learning* melalui hyperparameter tuning untuk meningkatkan performa klasifikasi. **Metode yang digunakan:** *Graph Neural Network* (GNN) dengan penerapan random search hyperparameter tuning. **Temuan Penelitian:** Penelitian ini bertujuan untuk meningkatkan kinerja model klasifikasi senyawa inhibitor SARS-CoV-2 dengan

mengoptimalkan parameter model Graph Neural Network. Dataset yang digunakan terdiri dari data senyawa kimia dalam bentuk representasi graf molekul (SMILES) yang dikelompokkan ke dalam dua kelas, yaitu inhibitor dan non-inhibitor. Tahapan penelitian dimulai dengan preprocessing data graf, pembentukan model GNN, serta proses hyperparameter tuning menggunakan metode random search untuk memperoleh konfigurasi parameter terbaik. Evaluasi performa model dilakukan menggunakan metrik akurasi serta analisis *Confusion Matrix* pada data uji. Berdasarkan hasil *Confusion Matrix*, model GNN dengan hyperparameter terbaik mampu mengklasifikasikan sebagian besar data inhibitor dan non-inhibitor secara tepat, yang ditunjukkan oleh tingginya jumlah true positive dan true negative, serta rendahnya kesalahan klasifikasi berupa false positive dan false negative. Hasil ini menunjukkan bahwa penerapan hyperparameter tuning tidak hanya meningkatkan nilai akurasi model, tetapi juga memperbaiki kemampuan model dalam membedakan kedua kelas secara lebih seimbang. Dengan demikian, penelitian ini membuktikan bahwa optimasi parameter memiliki peran penting dalam meningkatkan kinerja model *Machine Learning* berbasis graf pada permasalahan klasifikasi (Himawan et al., 2023).

7. Penelitian terkait optimasi kinerja model *Machine Learning* melalui pendekatan metaheuristik untuk hyperparameter tuning pada model klasifikasi. **Metode yang digunakan:** *Convolutional Neural Network* (CNN) yang dioptimalkan melalui tiga algoritma metaheuristik, yaitu *Ant Colony Optimization* (ACO), *Genetic Algorithm* (GA), dan *Harmony Search* (HS). **Temuan Penelitian:** Penelitian ini bertujuan untuk mencari kombinasi hyperparameter terbaik pada model CNN yang digunakan untuk klasifikasi citra dengan mengaplikasikan pendekatan metaheuristik sehingga ruang solusi yang sangat besar dapat dieksplorasi secara efisien. Dataset yang digunakan dalam penelitian ini adalah MNIST, sebuah dataset citra angka tulisan tangan yang umum dipakai untuk evaluasi klasifikasi citra. Proses penelitian dimulai dengan preprocessing data citra dan pembentukan arsitektur CNN, kemudian dilakukan pencarian kombinasi hyperparameter terbaik menggunakan masing-masing algoritma ACO, GA, dan HS untuk menghasilkan konfigurasi parameter yang optimal. Evaluasi performa model

dilakukan menggunakan metrik akurasi, serta pengamatan terhadap *Confusion Matrix* dan karakterisasi *precision–recall curve*, walaupun angka detail *Confusion Matrix* tidak dilaporkan secara eksplisit di dalam artikel, penelitian ini menyatakan bahwa model dengan hyperparameter hasil metaheuristik menunjukkan kinerja yang sangat baik dalam membedakan kelas citra pada dataset MNIST. Secara kuantitatif, hasil eksperimen menunjukkan bahwa metode ACO mencapai akurasi tertinggi sebesar 99,7%, diikuti oleh GA sebesar 97,7%, serta HS sebesar 89,9%, yang mengindikasikan bahwa model CNN mampu meminimalkan kesalahan klasifikasi (yang juga tercermin pada pola diagonal dominan dalam *Confusion Matrix* kelas) serta memiliki kombinasi *precision* dan *recall* yang tinggi bagi setiap kelas angka yang diprediksi. Hasil ini membuktikan bahwa pendekatan metaheuristik sangat menjanjikan dalam tuning hyperparameter CNN untuk meningkatkan performa klasifikasi pada permasalahan yang kompleks (Purnomo et al., 2024).

8. Penelitian terkait optimasi kinerja model *Machine Learning* melalui hyperparameter tuning dan pemilihan fitur untuk meningkatkan akurasi klasifikasi. **Metode yang digunakan:** Gaussian Naïve Bayes dengan Hyperparameter Tuning menggunakan *GridSearch CV* dan *Univariate Feature Selection* (ANOVA-f test). **Temuan Penelitian:** Penelitian ini bertujuan untuk meningkatkan kinerja model Gaussian Naïve Bayes dalam memprediksi kondisi cuaca harian di Kota Palembang berdasarkan data historis harian sejumlah 6.454 sampel. Proses penelitian dimulai dengan tahapan preprocessing data cuaca, dilanjutkan dengan penerapan Univariate Feature Selection untuk memilih fitur-fitur yang paling relevan terhadap target prediksi, serta penggunaan GridSearchCV untuk menyesuaikan hyperparameter model guna menghasilkan kombinasi parameter terbaik. Model kemudian dievaluasi menggunakan data uji sebanyak 644 sampel dengan metrik akurasi serta ditinjau dari pola prediksi yang dihasilkan dalam *Confusion Matrix*. Hasil penelitian menunjukkan bahwa optimasi model berhasil meningkatkan performa klasifikasi secara signifikan, dengan akurasi tertinggi setelah optimasi mencapai 98,33%, meningkat dari akurasi awal sebelum optimasi sebesar 96,95%. Analisis *Confusion Matrix*

memperlihatkan pengurangan jumlah kesalahan klasifikasi untuk kategori cuaca yang jarang terjadi (misalnya hujan sangat lebat), di mana prediksi yang salah turun dari 20 kasus menjadi hanya 7 kasus setelah optimasi. Hal ini menunjukkan bahwa kombinasi pemilihan fitur relevan dan penyesuaian hyperparameter melalui GridSearchCV mampu meningkatkan kemampuan model dalam membedakan kelas-kelas target secara lebih akurat dan konsisten (Lindawati et al., 2023).

9. Penelitian terkait optimasi kinerja model *Machine Learning* melalui hyperparameter tuning pada model klasifikasi. **Metode yang digunakan:** *Logistic Regression*, SVM, *Decision Tree*, dan KNN dengan penerapan *GridSearch CV* dan *RandomizedSearch CV* untuk tuning parameter sekaligus cross-validation. **Temuan Penelitian:** Penelitian ini bertujuan untuk meningkatkan efektivitas model klasifikasi dalam memprediksi risiko penyakit jantung dengan mengevaluasi dampak hyperparameter tuning pada beberapa algoritma *Machine Learning* menggunakan dataset Cleveland Clinic Heart Disease yang diperoleh dari Kaggle. Dataset diproses melalui tahapan prakualifikasi, pembagian data training dan testing, serta penerapan teknik tuning untuk setiap model agar kombinasi parameter terbaik ditemukan. Evaluasi dilakukan menggunakan metrik seperti akurasi, precision, recall, dan F1-score yang ditinjau melalui *Confusion Matrix*, dan hasilnya menunjukkan bahwa setelah dilakukan tuning, model *Logistic Regression*, KNN, dan SVM masing-masing mencapai akurasi tertinggi sebesar 84%, sedangkan Decision Tree menghasilkan akurasi lebih rendah yaitu 80%. Selain itu, laporan evaluasi menunjukkan peningkatan nilai precision, recall, dan F1-score terutama pada model *Logistic Regression* dan KNN yang menunjukkan keseimbangan performa terbaik antara benar positif dan benar negatif sehingga memperkuat kemampuan model dalam memprediksi risiko penyakit jantung secara lebih akurat. Dengan demikian, penelitian ini menunjukkan bahwa hyperparameter tuning berperan penting dalam meningkatkan kinerja model klasifikasi, khususnya pada model KNN dan *Logistic Regression* dalam prediksi risiko penyakit jantung berbasis *Machine Learning* (Putri et al., 2025).

10. Penelitian terkait optimasi kinerja model machine learning dalam klasifikasi tingkat keparahan kecelakaan lalu lintas dan identifikasi segmen jalan berisiko tinggi. **Metode yang digunakan:** *Logistic Regression* (LR), *Support Vector Machines* (SVM), *Multilayer Perceptron* (MLP), *Random Forest* (RF), dan *Extreme Gradient Boosting* (XGBoost) dengan penanganan ketidakseimbangan kelas melalui *Synthetic Minority Oversampling Technique* (SMOTE). **Temuan Penelitian:** Penelitian ini bertujuan untuk membangun dan membandingkan kinerja beberapa algoritma machine learning dalam mengklasifikasikan tingkat keparahan kecelakaan pada dataset kecelakaan dari provinsi Şırnak, Turki, yang berisi 29 variabel terkait kondisi kecelakaan, geometrik jalan, dan operasi lalu lintas. Dataset yang sangat tidak seimbang antara kecelakaan cedera dan kecelakaan fatal diolah dengan metode SMOTE untuk meningkatkan sensitivitas model terhadap kelas minoritas. Evaluasi model dilakukan menggunakan metrik akurasi, F1-score, dan analisis *Confusion Matrix* untuk melihat kemampuan model dalam membedakan antara kecelakaan cedera dan fatal. Hasil penelitian menunjukkan bahwa sebelum SMOTE, model-model mengalami kesulitan dalam mendeteksi kecelakaan fatal secara akurat karena dominasi kelas cedera, dengan F1-score paling tinggi yang diperoleh Random Forest sebesar 0,55 dan Logistic Regression sebesar 0,42 pada data asli. Setelah penerapan SMOTE, performa peningkatan terlihat jelas, di mana algoritma XGBoost mencapai F1-score tertinggi sebesar 0,61 dengan tingkat False Alarm terendah (0,01), diikuti oleh Random Forest dan MLP yang masing-masing menunjukkan F1-score yang kompetitif, sementara SVM dan LR meskipun lebih cepat secara komputasi memberikan performa yang lebih rendah. Analisis *Confusion Matrix* setelah SMOTE mengindikasikan bahwa XGBoost dan Random Forest memiliki jumlah *true positive* fatal crash yang lebih tinggi sambil tetap mempertahankan False Alarm rendah, sedangkan SVM dan MLP mencapai keseimbangan moderat antara sensitivitas dan spesifisitas. Temuan ini menegaskan bahwa model ensemble seperti XGBoost dan Random Forest mampu mengatasi kompleksitas data kecelakaan yang heterogen dan membantu dalam pengambilan keputusan berbasis risiko pada perencanaan keselamatan jalan (Yumak et al., 2025).

Berdasarkan referensi penelitian terdahulu yang telah dibahas, sebagian besar penelitian sebelumnya berfokus pada penerapan machine learning untuk klasifikasi tingkat keparahan kecelakaan lalu lintas dengan menggunakan berbagai algoritma seperti *Decision Tree, Random Forest, Support Vector Machine, Logistic Regression*, serta KNN (Silvia & Wilandari, 2022; Sinaga et al., 2025; Fauzan et al., 2024). Beberapa penelitian menitikberatkan pada perbandingan antar algoritma atau penanganan ketidakseimbangan kelas menggunakan teknik oversampling seperti SMOTE (Firmansyah et al., 2023; Ahmet Yumak et al., 2025), sementara penelitian lain lebih berfokus pada optimasi model secara umum melalui hyperparameter tuning tanpa penekanan khusus pada KNN (Putri et al., 2025). Selain itu, sebagian penelitian menggunakan dataset berskala lokal atau regional dengan jumlah atribut yang terbatas, serta belum secara spesifik mengeksplorasi faktor-faktor penyebab kecelakaan seperti kondisi cuaca, waktu kejadian, dan karakteristik infrastruktur jalan secara komprehensif menggunakan dataset publik dari Kaggle, seperti *Road Traffic Severity Classification* Dataset.

Penelitian ini difokuskan pada optimasi model (KNN) dalam memprediksi tingkat keparahan kecelakaan lalu lintas dengan memanfaatkan dataset publik *Road Traffic Severity Classification* dari Kaggle yang memiliki cakupan data luas serta beragam variabel penyebab kecelakaan. Berbeda dengan penelitian sebelumnya, penelitian ini tidak hanya membangun model KNN secara konvensional, tetapi juga menerapkan hyperparameter tuning untuk menentukan konfigurasi parameter terbaik, seperti nilai k, metode pengukuran jarak, dan skema pembobotan, guna meningkatkan kinerja model. Selain itu, penelitian ini menekankan pada analisis faktor-faktor penyebab kecelakaan, meliputi kondisi cuaca, waktu kejadian, kondisi lingkungan, serta karakteristik infrastruktur jalan yang berpotensi memengaruhi tingkat keparahan kecelakaan. Evaluasi kinerja model dilakukan menggunakan metrik akurasi, precision, recall, F1-score, serta confusion matrix. Dengan pendekatan tersebut, penelitian ini diharapkan dapat memberikan kontribusi baru dalam pengembangan model prediksi kecelakaan lalu lintas berbasis KNN, khususnya dalam menunjukkan efektivitas *hyperparameter tuning* berbasis *Gridsearch CV* serta peran faktor-faktor penyebab kecelakaan dalam meningkatkan akurasi prediksi tingkat keparahan kecelakaan lalu lintas.

Tabel 2. 1 Penelitian Terkait

No	Penelitian	Metode	Objek Penelitian	Hasil Penelitian
1	Susanti et al. (2024)	SVM, Bagging, SMOTE	Klasifikasi tingkat keparahan kecelakaan lalu lintas	SVM dengan *under sampling* memperoleh akurasi tertinggi sebesar 93,66%, sedangkan Bagging + SMOTE memperoleh 87,97%.
2	Rahmi et al. (2024)	Multiclass SVM (Kernel Linear & RBF)	Tingkat keparahan kecelakaan sepeda motor	Kernel RBF memberikan performa terbaik dengan akurasi 94,62%.
3	Ramadhani et al. (2025)	SVM, Forward Selection, ADASYN	Klasifikasi tingkat keparahan kecelakaan	Kombinasi Forward Selection dan ADASYN meningkatkan performa model dibandingkan SVM tanpa optimasi.
4	Anitasari et al. (2025)	K-Nearest Neighbors (KNN)	Klasifikasi kecelakaan di Kota Samarinda	Model KNN memperoleh akurasi 92,54% menggunakan K-Fold Cross Validation.
5	Nababan & Hutagalung (2023)	SVM + GridSearchCV	Stance Detection Twitter	Hyperparameter tuning meningkatkan nilai Micro F1-Score menjadi 78,4%.
6	Himawan et al. (2023)	Graph Neural Network + Random Search	Klasifikasi inhibitor SARS-CoV-2	Hyperparameter tuning meningkatkan akurasi dan kemampuan klasifikasi model GNN.

Tabel 2. 1 Penelitian Terkait (Lanjutan)

No	Penelitian	Metode	Objek Penelitian	Hasil Penelitian
7	Purnomo et al. (2024)	CNN + ACO, GA, Harmony Search	Klasifikasi citra MNIST	Metode ACO menghasilkan akurasi tertinggi sebesar 99,7%.
8	Lindawati et al. (2023)	Gaussian Naïve Bayes + GridSearchCV	Prediksi cuaca	Optimasi meningkatkan akurasi dari 96,95% menjadi 98,33%.
9	Putri et al. (2025)	Logistic Regression, SVM, DT, KNN + GridSearchCV	Prediksi penyakit jantung	Hyperparameter tuning meningkatkan performa model, dengan akurasi tertinggi 84%.
10	Yumak et al. (2025)	LR, SVM, MLP, RF, XGBoost + SMOTE	Tingkat keparahan kecelakaan di Turki	XGBoost menghasilkan performa terbaik dengan F1-Score 0,61 setelah SMOTE.

Sumber: Diolah dari berbagai penelitian (2023–2025).

Berdasarkan Tabel 2.1 dapat diketahui bahwa penelitian mengenai prediksi tingkat keparahan kecelakaan lalu lintas telah banyak dilakukan dengan memanfaatkan berbagai algoritma machine learning, seperti Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Random Forest, Logistic Regression, Decision Tree, hingga Extreme Gradient Boosting (XGBoost). Selain itu, beberapa penelitian menerapkan teknik penanganan ketidakseimbangan data, seperti SMOTE dan ADASYN, serta optimasi model menggunakan Hyperparameter Tuning berbasis GridSearchCV maupun Random Search untuk meningkatkan performa klasifikasi.

Meskipun demikian, belum banyak penelitian yang secara khusus mengoptimalkan model K-Nearest Neighbors (KNN) menggunakan kombinasi Hyperparameter Tuning berbasis Grid Search Cross Validation (Grid Search CV) dan Synthetic Minority Oversampling Technique (SMOTE) pada dataset publik Road Traffic Severity Classification dari Kaggle. Oleh karena itu, penelitian ini berfokus pada optimasi model

KNN melalui pencarian parameter terbaik serta penanganan ketidakseimbangan kelas guna menghasilkan performa klasifikasi yang lebih optimal dalam memprediksi tingkat keparahan kecelakaan lalu lintas.

2.2 Kecelakaan Lalu Lintas

Kecelakaan lalu lintas merupakan fenomena yang kompleks dan multidimensional yang dipengaruhi oleh berbagai faktor yang saling berkaitan, termasuk perilaku manusia, kondisi jalan, serta kondisi lingkungan sekitar**.** Sebagai salah satu masalah kesehatan masyarakat yang serius, kecelakaan lalu lintas tidak hanya menyebabkan kematian dan luka berat, tetapi juga berdampak signifikan terhadap biaya sosial dan ekonomi, seperti biaya perawatan, kehilangan produktivitas, serta dampak psikologis pada korban dan keluarga. Di Indonesia, kecelakaan lalu lintas terus meningkat seiring dengan pertumbuhan jumlah kendaraan bermotor dan mobilitas masyarakat, sehingga menjadi fokus penting dalam upaya peningkatan keselamatan jalan dan kebijakan lalu lintas yang efektif (Syahriza, 2019).

Secara empiris, berbagai studi menunjukkan bahwa faktor utama penyebab kecelakaan lalu lintas dapat dikategorikan menjadi aspek manusia (*human factors*), kendaraan, dan lingkungan fisik jalan. Faktor manusia seperti ketidaktertiban berkendara, kelelahan, pengemudi lengah, serta kurangnya disiplin terhadap aturan lalu lintas merupakan kontributor dominan dalam banyak kejadian kecelakaan, terutama di wilayah urban maupun kawasan tol (Mega Sari & Lestari, 2024). Selain itu, penelitian karakteristik kecelakaan menunjukkan bahwa kecelakaan cenderung terjadi pada jam-jam padat aktivitas masyarakat, perilaku ceroboh terhadap lalu lintas, serta kelompok usia produktif, menekankan perlunya pemahaman faktor penyebab sebagai dasar intervensi keselamatan yang lebih tepat sasaran (Surahman et al., 2025).

2.3 Machine Learning

Machine learning merupakan cabang dari kecerdasan buatan yang fokus pada pengembangan sistem komputer yang mampu belajar dari data dan mengenali pola tanpa

diprogram secara eksplisit berdasarkan aturan tetap, sehingga dapat memprediksi atau mengklasifikasikan data baru dengan akurasi yang semakin baik seiring pengalaman atau jumlah data tersedia. Dalam kajian sistematis, machine learning didefinisikan sebagai bidang penelitian ilmiah yang mempelajari algoritma dan model statistik yang digunakan oleh sistem komputer untuk menyelesaikan tugas-tugas tertentu tanpa instruksi eksplisit, melainkan dengan memanfaatkan pola serta inferensi dari data yang tersedia sehingga mampu mengurangi kerja manual dan menghasilkan keputusan yang lebih efisien dan akurat dalam berbagai konteks aplikasi (Yumak et al., 2025).

Machine learning umumnya dibagi menjadi beberapa paradigma utama, yaitu *Supervised learning*, *Unsupervised learning*, dan *reinforcement learning*, yang masing-masing memiliki tujuan dan pendekatan berbeda dalam menangani data untuk prediksi, klasifikasi, atau pengelompokan. *Supervised learning* menggunakan data berlabel untuk melatih model agar dapat memetakan input ke output yang benar, sedangkan un *Supervised learning* tidak memerlukan label dan lebih fokus menemukan pola tersembunyi dalam data. Pendekatan-pendekatan ini memungkinkan machine learning diaplikasikan pada berbagai domain nyata seperti kesehatan, kecelakaan lalu lintas, ekonomi, dan sistem cerdas lainnya, dengan pemilihan algoritma tergantung pada karakteristik data dan tujuan penelitian (Sarker, 2021).

2.4 *Supervised Learning*

Supervised learning adalah salah satu paradigma utama dalam *machine learning* di mana model dilatih menggunakan data berlabel yang sudah memiliki kategori atau nilai target tertentu untuk membentuk fungsi pemetaan antara input dan output. Pendekatan ini memungkinkan model mempelajari hubungan antara fitur dan label melalui contoh-contoh yang telah diketahui, sehingga ketika menghadapi data baru model dapat memprediksi hasilnya berdasarkan pola yang sudah dipelajari. *Supervised learning* sering digunakan dalam permasalahan klasifikasi maupun regresi, termasuk tugas-tugas seperti prediksi penyakit dan klasifikasi kualitas (Rahmawati et al., 2023).

Dalam studi-studi terapan, *Supervised learning* terbukti efektif dalam menangani masalah yang memerlukan pemetaan eksplisit antara atribut dan label target karena kemampuan model untuk generalisasi dari data pelatihan ke data uji yang belum pernah dilihat. *Supervised learning* mengintegrasikan berbagai algoritma seperti *K-Nearest Neighbor* (KNN), *Support Vector Machine* (SVM), *Logistic Regression*, dan lainnya untuk menghasilkan model prediksi yang dapat dievaluasi melalui metrik seperti akurasi dan *confusion matrix* berdasarkan data berlabel. Pendekatan ini mendasari banyak aplikasi klasifikasi data nyata termasuk aplikasi kesehatan, pendidikan, dan sistem prediksi berbasis data historis (Adenia et al., 2024).

2.5 *Python*

Python merupakan salah satu bahasa pemrograman tingkat tinggi yang populer digunakan dalam pengembangan aplikasi *data science* dan *machine learning* karena sintaksisnya sederhana, mudah dibaca, dan memiliki ekosistem pustaka yang kaya seperti *scikit-learn, Pandas, NumPy, TensorFlow, dan PyTorch*, yang memudahkan peneliti dan pengembang mengimplementasikan proses pra-pemrosesan data, pembangunan model, serta evaluasi kinerja model secara efisien dan dapat diandalkan. *Python* juga mendukung integrasi berbagai *library visualisasi* dan analisis statistik sehingga menjadi pilihan utama dalam penelitian ilmiah untuk analisis data, pemodelan prediksi, dan eksperimen *machine learning* karena mengurangi kompleksitas pemrograman dan memungkinkan fokus pada algoritma dan interpretasi hasil (Suryati et al., 2025).

2.6 *Streamlit*

Streamlit merupakan *framework* berbasis *Python* yang dirancang untuk mempermudah pengembangan aplikasi web interaktif, khususnya dalam bidang data *science* dan *machine learning*. Dengan pendekatan deklaratif, *Streamlit* memungkinkan pengguna untuk mengubah *script Python* menjadi aplikasi web tanpa memerlukan keahlian dalam HTML, CSS, atau JavaScript. Hal ini menjadikan Streamlit sebagai solusi yang efisien bagi peneliti dan praktisi dalam menyajikan hasil analisis data, visualisasi,

serta model prediksi dalam bentuk dashboard yang interaktif dan mudah digunakan (Syafarina & Zaenuddin, 2023).

Selain itu, *Streamlit* juga mendukung integrasi berbagai pustaka populer seperti *Matplotlib*, Seaborn, dan Plotly untuk menghasilkan visualisasi data yang dinamis dan informatif. Kemampuannya dalam melakukan pembaruan tampilan secara real-time memungkinkan pengguna untuk melakukan eksplorasi data secara lebih fleksibel dan responsif. Oleh karena itu, Streamlit banyak dimanfaatkan dalam pengembangan dashboard analitik dan sistem berbasis data karena kemudahan implementasi serta kemampuannya dalam meningkatkan pemahaman pengguna terhadap informasi yang disajikan (Syafarina & Zaenuddin, 2023).

2.7 Pandas

Pandas merupakan pustaka Python yang umum digunakan dalam pengolahan dan manipulasi data karena menyediakan struktur data seperti *DataFrame* yang memudahkan dalam menangani data tabular secara efisien, termasuk pemfilteran, penggabungan, dan pembersihan data sebelum analisis atau pemodelan lanjutan. Dalam konteks *data science*, Pandas membantu menyederhanakan proses eksplorasi dan transformasi data sehingga memungkinkan peneliti untuk mengelola dataset besar dengan lebih terstruktur dalam lingkungan Python (Candra, 2025).

2.8 *Scikit-Learn*

*Scikit-L*earn adalah pustaka *open-source* Python yang menyediakan berbagai algoritma machine learning untuk tugas seperti klasifikasi, regresi, dan clustering, serta alat pra-pemrosesan data yang terintegrasi untuk mempermudah pembangunan model prediktif. Pustaka ini dirancang untuk efisiensi dan kemudahan penggunaan oleh peneliti maupun praktisi, serta sering digunakan dalam penelitian untuk membangun dan mengevaluasi model machine learning pada berbagai domain aplikasi (Sieg et al., 2024).

2.9 *Imbalanced-Learn*

Imbalanced-learn adalah library Python yang sering digunakan dalam *machine learning* untuk mengatasi masalah data tidak seimbang (*imbalanced data*) dengan menyediakan teknik *resampling* seperti *oversampling*, *undersampling*, serta kombinasi keduanya agar model klasifikasi dapat belajar secara lebih adil dari semua kelas. Teknik-teknik ini sangat penting ketika distribusi kelas tidak seimbang karena model cenderung bias terhadap kelas yang dominan, sehingga library ini membantu meningkatkan performa prediksi terutama pada kelas minoritas (Kurniawati, 2019).

2.10 *K-Nearest Neighbor* (KNN)

Algoritma *K-Nearest Neighbor* (KNN) adalah metode untuk mengklasifikasikan objek berdasarkan data pembelajaran yang paling dekat dengan objek tersebut. Data tersebut diproyeksikan ke dalam banyak ruang dimensi dimana setiap dimensi mempresentasikan fitur-fitur data *K-Nearest Neighbor* melakukan klasifikasi berdasarkan data latih dan data uji yang paling dekat dengan objek berdasarkan K- tetangga (Hariadhy et al., 2021).

KNN merupakan algoritma yang paling umum digunakan dalam klasifikasi pada bidang kecerdasan buatan. Secara dasar ketika terdapat data baru yang akan diklasifikasikan harus ditentukan berdasarkan kategori dari sampel data lainnya. Data yang akan diklasifikasikan wajib dilakukan ekstraksi dan dibandingkan dengan karakteristik setiap data yang ada pada data uji. Kemudian akan diambil tetangga terdekat K dari data uji untuk menghitung kategori dimana data tersebut akan berada (Wang et al., 2021).

2.10.1 *Eucliedan Distance*

Euclidean distance adalah metrik pengukuran jarak yang paling umum digunakan dalam algoritma *K-Nearest Neighbor* (KNN), di mana jarak dihitung sebagai akar dari jumlah kuadrat selisih antara setiap atribut data uji dan data latih, sehingga data dengan

nilai jarak terkecil dianggap sebagai tetangga terdekat yang paling mirip (Setiawan, 2022). Hal ini dapat dilihat pada Persamaan 2.1.

$$D(p,q) = \sqrt{\sum_{i=1}^{n}(p_i - q_i)^2} \quad (2.1)$$

Dimana :

D = Jarak antara dua data

p = Data Uji

q = Sampel Data

n = Dimensi data

2.10.2 *Manhattan Distance*

Manhattan distance, juga dikenal sebagai *City Block distance*, adalah metrik jarak yang digunakan dalam algoritma KNN untuk mengukur kedekatan antar titik data dengan cara menjumlahkan nilai absolut selisih setiap atribut dari dua titik data, sehingga lebih robust terhadap perbedaan besar kecilnya nilai atribut dibandingkan dengan jarak Euclidean (Setiawan, 2022). Hal ini dapat dilihat pada Persamaan 2.2.

$$D(p,q) = \sum_{i=1}^{n} |p_i - q_i| \quad (2.2)$$

Dimana :

D = Jarak antara dua data

p = Data Uji

q = Sampel Data

n = Dimensi data

2.11 SMOTE

Synthetic Minority Over-sampling Technique (SMOTE) adalah metode yang digunakan untuk menyeimbangkan jumlah data setiap *class*. SMOTE bekerja dengan cara menyeimbangkan distribusi data sample pada *class* minoritas atau bisa juga dilakukan dengan menyeimbangkan distribusi data sample pada *class* mayoritas juga. Hal ini dilakukan dengan menyeleksi data *training* kemudian menyeimbangkannya dengan *class*

minoritas dan juga sebaliknya, sehingga kedua *class* seimbang dalam hal distribusi data dan juga jumlah data *training*. Metode SMOTE ini dilakukan dikarenakan banyak data yang jumlah *class* tiap data tidak seimbang dan hal ini juga akan mempengaruhi kinerja dari model *machine learning* yang kita buat. Dalam penggunaan SMOTE, hal yang pertama kali dilakukan adalah menghitung jarak antara kedua *class* yang minoritas atau yang mayoritas, selanjutnya akan menentukan nilai presentase SMOTE dan menentukan jumlah K terdekat, sehingga akan menciptakan data sintesis dengan persamaan 2.4. (Sulistiyono et al., 2021).

$$x_{syn} = x_i + \delta(x_{knn} - x_i) \quad \text{.....(2.4)}$$

Keterangan :

a. x_{syn} adalah data sintesis yang dihasilkan
b. x_i adalah titik data asli
c. x_{knn} adalah tetangga terdekat dari x_i
d. δ adalah nilai random antara 0 dan 1.

2.12 *Hyperparameter Tuning*

Hyperparameter tuning adalah proses pencarian konfigurasi parameter terbaik pada model machine learning yang tidak dipelajari langsung dari data, tetapi ditetapkan sebelum model dilatih, sehingga dapat meningkatkan performa prediksi model secara signifikan jika dibandingkan dengan nilai parameter *default*. Dalam praktiknya, teknik-teknik seperti *Grid Search, Random Search*, atau optimasi berbasis algoritma heuristik digunakan untuk mengevaluasi berbagai kombinasi hyperparameter, sehingga model dapat mencapai metrik evaluasi seperti akurasi atau F1-score yang lebih tinggi di data uji (Nababan & Hutagalung, 2023).

2.13 *Grid Search* CV

Grid Search CV adalah metode yang digunakan untuk mengoptimalkan nilai akurasi dari suatu model *machine learning*. Hal ini dilakukan dengan memillih kombinasi model dan menentukan parameter terbaik dari suatu model *machine learning* dengan

menguji parameter dengan kombinasi pembagian data dan memvalidasi setiap kombinasi tersebut. Tujuan dari *Grid Search* adalah untuk menentukan parameter terbaik dari berbagai kombinasi pembagian data yang sudah dilakukan untuk menghasilkan performa model *machine learning* terbaik yang dapat digunakan sebagai model prediksi. *Grid Search CV* dikombinasikan dengan *K-Fold Cross Validarion* untuk menciptakan hasil evaluasi pada model klasifikasi (Toha et al., 2022).

2.14 *K-Fold Cross Validation*

K-Fold Cross Validation adalah salah satu teknik evaluasi model dalam *machine learning* yang membagi dataset menjadi *k* bagian (*folds*) yang sama besar, kemudian model dilatih dan diuji secara bergantian sebanyak *k* kali sehingga setiap bagian menjadi data uji sekali dan data latih *k-1* kali, sehingga hasil evaluasi model tidak tergantung pada satu pembagian data saja. Teknik ini membantu menghasilkan estimasi kinerja model yang lebih stabil dan mengurangi kemungkinan bias akibat pembagian data yang tidak representatif, serta umum digunakan untuk mengukur performa algoritma klasifikasi seperti K-Nearest Neighbor, Naive Bayes, atau lainnya dalam berbagai studi aplikatif. Hal ini dapat diimplementasikan dalam persamaan 2.5 (Hafid, 2023).

$$Jumlah\ Data\ per\ fold = \frac{N}{K} \quad \text{.....(2.5)}$$

Keterangan :

N = Jumlah data Awal

K = Nilai K untuk tiap pembagian.

Dalam praktiknya, *K-Fold Cross Validation* memungkinkan evaluasi yang lebih andal karena setiap titik data digunakan baik sebagai data pelatihan maupun data pengujian sehingga mengatasi masalah pembagian data tunggal yang dapat menyebabkan variabilitas hasil yang tinggi; hasil metrik seperti akurasi, presisi, recall, dan F1-score kemudian dirata-rata antar fold untuk memberikan gambaran kinerja model secara keseluruhan. Studi-studi evaluasi model klasifikasi yang menggunakan *K-Fold Cross Validation* menunjukkan bahwa metode ini sangat berguna untuk membandingkan

performa berbagai algoritma dan merancang model yang lebih generalisasi terhadap data belum terlihat, misalnya pada penelitian komparatif kinerja *Naive Bayes* dan KNN di dataset publik (Abrori, 2025).

2.15 *Dummy Baseline*

Dummy Baseline merupakan model sederhana yang digunakan sebagai acuan awal untuk membandingkan performa model machine learning yang dikembangkan. Pada permasalahan klasifikasi, *dummy baseline* umumnya bekerja dengan cara memprediksi kelas mayoritas tanpa mempelajari pola dari data. Penggunaan *dummy baseline* penting dilakukan untuk mengetahui apakah model yang dibangun benar-benar memiliki kemampuan klasifikasi yang lebih baik dibandingkan prediksi sederhana atau tebakan naif (*naive prediction*). Jika performa model utama lebih rendah atau hampir sama dengan dummy baseline, maka model tersebut dianggap belum optimal dalam mempelajari pola data (Bijl et al., 2022).

Pada penelitian klasifikasi dengan data tidak seimbang (*imbalanced dataset*), dummy baseline juga berfungsi sebagai pembanding untuk mengevaluasi apakah model mampu mengenali kelas minoritas dengan baik. Oleh karena itu, evaluasi tidak hanya berfokus pada accuracy, tetapi juga menggunakan metrik seperti precision, recall, dan F1-score agar performa model dapat dinilai secara lebih menyeluruh. Dengan adanya *dummy baseline*, hasil evaluasi model menjadi lebih objektif dan dapat menunjukkan peningkatan performa yang dihasilkan oleh metode klasifikasi yang digunakan (Koziarski et al., 2021).

2.16 *Pipeline*

Pipeline dalam machine learning merupakan rangkaian proses terstruktur yang menggabungkan tahapan preprocessing data dan proses pelatihan model ke dalam satu alur kerja yang terintegrasi. Penggunaan *pipeline* bertujuan untuk memastikan bahwa setiap tahapan, seperti imputasi missing value, normalisasi data, oversampling menggunakan SMOTE, hingga proses klasifikasi dilakukan secara konsisten pada data latih maupun data uji. Selain itu, pipeline juga membantu mengurangi risiko terjadinya data leakage karena seluruh proses preprocessing hanya dilakukan berdasarkan data

pelatihan pada setiap iterasi validasi silang. Dengan demikian, hasil evaluasi model menjadi lebih valid dan dapat merepresentasikan performa model secara lebih objektif (Ichwani et al., 2026).

Dalam implementasi *machine learning, pipeline* juga mempermudah proses optimasi model karena seluruh tahapan dapat dijalankan secara otomatis dan terintegrasi dengan metode validasi seperti *Cross Validation* maupun *Grid Search* CV. Penggunaan pipeline memungkinkan proses preprocessing dan pemodelan dilakukan secara berulang dengan parameter yang berbeda tanpa perlu menjalankan tahapan secara manual. Hal ini membuat proses eksperimen menjadi lebih efisien, konsisten, serta meningkatkan reproduktibilitas penelitian. Selain itu, pipeline dapat membantu meningkatkan kualitas pemodelan karena setiap transformasi data diterapkan dengan urutan yang tepat sebelum model melakukan proses klasifikasi (Ichwani et al., 2026).

2.17 *Confusion Matrix Multi Class*

Confusion Matrix merupakan sebuah cara untuk mempresentasikan hasil daripada *accuracy* dari model yang sudah dibuat. *Confusion Matrix* merangkum performa dalam mengklasifikasi sesuai dengan banyaknya kategori yang diklasifikasi dari nilai benar dari suatu kelas dari objek yang diprediksi (Aldean et al., 2022). Pada pengukuran *Confusion Matrix* terdapat 4 istilah untuk mempresentasikan hasil proses klasifikasi, keempat istilah adalah *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN). Nilai *True Negative* (TN) merupakan jumlah data negatif yang terdeteksi dengan benar, sedangkan *False Positive* (FP) merupakan data negatif namun terdeteksi sebagai data positif. Sementara itu, *True Positive* (TP) merupakan data positif yang terdeteksi benar. *False Negative* (FN) merupakan kebalikan dari *True Positive,* sehingga data positif, namun terdeteksi sebagai data negatif. Pada penelitian ini menggunakan *confusion matrix multiclass* karena *output* yang dimiliki sebanyak 3 *output* (Aldean et al., 2022).

Matriks pada *confusion Matrix Multiclass* memberikan hasil seberapa baik performa dari suatu model. Diagonal utama menunjukkan jumlah prediksi yang benar untuk setiap. Bentuk dari *confusion matrix multiclass* pada Tabel 2.2 (Irmanda & Astriratma, 2020).

Tabel 2. 2 Confusion Matrix Multiclass

Hasil Aktual	Hasil Prediksi		
	A	**B**	**C**
A	TA	FB1	FC1
B	FA1	TB	FC2
C	FA2	FB2	TC

Berdasarkan Tabel 2.2 *confusion matrix multiclass* terdapat 2 bagian, yaitu kolom hasil prediksi dan baris hasil aktual. Hasil prediksi adalah hasil dari klasifikasi suatu model. Sedangkan hasil aktual adalah hasil yang didapatkan berdasarkan kenyataan. Hasil aktual terdiri dari 3 kelas, yaitu A, B, dan C. Dari ketiga kelas dibagi menjadi 9 kategori, yaitu kasus-kasus tersebut dibagi menjadi sembilan kategori: TA, FA1, FA2, FB1, TB, FB2, FC1, FC2, dan TC. TA diklasifikasikan dengan benar sebagai kelas A, TB diklasifikasikan dengan benar sebagai kelas B, TC diklasifikasikan dengan benar sebagai kelas C. FA1 adalah kelas B, diklasifikasikan sebagai kelas A, FA2 adalah kelas C, diklasifikasikan sebagai kelas A. FB1 adalah kelas A, diklasifikasikan sebagai Kelas B, FB2 adalah kelas C, diklasifikasikan dalam kelas B. FC1 adalah kelas A, diklasifikasikan dalam kelas A hingga kelas C, FC2 adalah kelas B, diklasifikasikan dalam kelas C (Irmanda & Astriratma, 2020). Setelah dibuat *confusion matrix multiclass* nilai *accuracy* akan dihitung untuk melihat perhitungan jarak yang memiliki *accuracy* paling tertinggi dengan persamaan 2.5 (Irmanda & Astriratma, 2020).

$$Accuracy = \frac{\sum_{i=1}^{k} TP_i}{Total\ Data} \quad \text{.....(2.7)}$$

Nilai *accuracy* merupakan nilai yang didapatkan dari hasil bagi semua jumlah data uji yang benar dengan jumlah data uji keseluruhan (Irmanda & Astriratma, 2020).

Precision digunakan untuk menggambarkan tingkat keakuratan antara semua kelas positif yang diprediksi dengan kelas positif yang sebenarnya. Untuk persamaan rumus menghitung *precision* dapat dilihat pada persamaan 2.6 (Irmanda & Astriratma, 2020).

$$Precision = \frac{TP_i}{TP_i + FP_i} \quad \text{.....(2.8)}$$

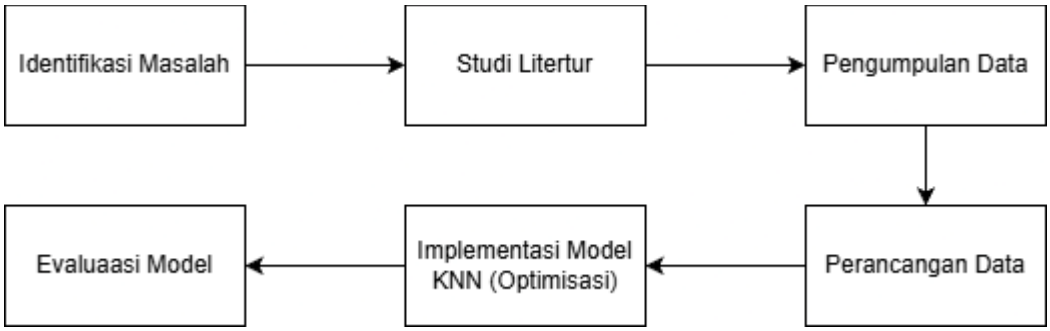
R*ecall* digunakan untuk mengetahui seberapa baik model dalam mengenali kelas positif yang sebenarnya. Persamaan rumus *recall* dapat dilihat pada persamaan 2.7 (Irmanda & Astriratma, 2020).

$$Recal = \frac{TP_i}{TP_i+FN_i} \text{.....(2.9)}$$

BAB III
METODOLOGI PENELITIAN

3.1 Tahapan Pelaksanaan Penelitian

Adapun tahapan penelitian ini meliputi identifikasi masalah, studi literatur, pengumpulan data, perancangan data, implementasi model KNN, dan evaluasi dapat dilihat di Gambar 3.1.



Gambar 3. 1 Tahapan Pelaksanaan Penelitian

1. Identifikasi Masalah

Tahap identifikasi masalah dilakukan untuk mengenali permasalahan yang terjadi pada proses klasifikasi tingkat keparahan kecelakaan lalu lintas. Permasalahan utama dalam penelitian ini adalah rendahnya performa model klasifikasi akibat data berskala besar, bersifat multi-class, serta memiliki distribusi kelas yang tidak seimbang. Oleh karena itu, diperlukan optimasi algoritma KNN agar mampu memberikan hasil prediksi yang lebih akurat dan stabil.

2. Studi Literatur

Studi literatur dilakukan dengan menelaah jurnal ilmiah, prosiding konferensi, dan referensi akademik yang berkaitan dengan kecelakaan lalu lintas, machine learning, algoritma KNN, optimasi hyperparameter, serta teknik evaluasi model seperti confusion matrix dan K-Fold *Cross Validation*. Studi literatur bertujuan untuk memperoleh landasan teoritis dan metodologis yang mendukung penelitian ini.

3. Pengumpulan Data

Pengumpulan data dilakukan dengan memanfaatkan dataset publik yang tersedia di situs Kaggle. Dataset yang digunakan adalah *Road Traffic Severity Classification* yang berisi data kecelakaan lalu lintas dengan berbagai atribut seperti waktu kejadian, lokasi, kondisi cuaca, dan tingkat keparahan kecelakaan.

4. Perancangan Data

Perancangan data dilakukan dengan menyeleksi atribut yang relevan, mengubah nama atribut agar lebih mudah dipahami, serta menentukan variabel yang digunakan dalam proses pemodelan. Atribut yang tidak memiliki keterkaitan dengan tujuan penelitian akan dieliminasi.

5. Implementasi Model KNN

Tahap implementasi meliputi preprocessing data, normalisasi fitur, pembagian data menggunakan K-Fold *Cross Validation*, serta penerapan algoritma KNN. Selain model baseline, dilakukan optimasi model melalui tuning hyperparameter dan seleksi fitur untuk memperoleh performa terbaik.

6. Evaluasi Model

Evaluasi model dilakukan menggunakan confusion matrix dan classification report untuk mengukur nilai *accuracy*, *precision*, *recall*, dan *F1-score*. Evaluasi dilakukan pada beberapa skenario eksperimen untuk membandingkan performa model KNN sebelum dan sesudah optimasi.

3.2 Pengumpulan Data

Data yang digunakan dalam penelitian ini bersumber dari situs penyedia dataset publik, yaitu Kaggle. Dataset yang digunakan berjudul **"***Road Traffic Severity Classification*" yang dipublikasikan oleh pengguna Kaggle dengan nama "*Avikumart*". Dataset ini berjumlah 12.316 data yang berisi data kecelakaan lalu lintas yang digunakan untuk mengklasifikasikan tingkat keparahan kecelakaan berdasarkan berbagai faktor yang memengaruhinya. Dataset ini terdiri dari 31 atribut dan 1 label target.

Dataset ini tersedia dalam format file CSV dan terdiri dari sejumlah fitur yang merepresentasikan kondisi saat terjadinya kecelakaan, seperti kondisi cuaca, kondisi

jalan, waktu kejadian, karakteristik kendaraan, serta faktor pengemudi. Variabel target dalam dataset ini adalah tingkat keparahan kecelakaan (*accident severity*), yang diklasifikasikan ke dalam beberapa kategori.

Deskripsi mengenai dataset dapat dilihat pada Tabel 3.1.

Tabel 3. 1 Deskripsi Atribut Dataset

No	Nama Atribut	Tipe Data	Deskripsi
1	*Time*	Kategorikal	Waktu terjadinya kecelakaan
2	*Day_of_week*	Kategorikal	Hari terjadinya kecelakaan
3	*Age_band_of_driver*	Kategorikal	Rentang usia pengemudi
4	*Sex_of_driver*	Kategorikal	Jenis kelamin pengemudi
5	*Educational_level*	Kategorikal	Tingkat pendidikan pengemudi
6	*Vehicle_driver_relation*	Kategorikal	Hubungan pengemudi dengan kendaraan
7	*Driving_experience*	Kategorikal	Lama pengalaman mengemudi
8	*Type_of_vehicle*	Kategorikal	Jenis kendaraan yang digunakan
9	*Owner_of_vehicle*	Kategorikal	Status kepemilikan kendaraan
10	*Service_year_of_vehicle*	Kategorikal	Lama penggunaan kendaraan
11	*Defect_of_vehicle*	Kategorikal	Kondisi kerusakan kendaraan
12	*Area_accident_occured*	Kategorikal	Lokasi terjadinya kecelakaan
13	*Lanes_or_Medians*	Kategorikal	Kondisi jalur atau median jalan
14	*Road_allignment*	Kategorikal	Kondisi kelurusan atau kemiringan jalan
15	*Types_of_Junction*	Kategorikal	Jenis persimpangan jalan
16	*Road_surface_type*	Kategorikal	Jenis permukaan jalan
17	*Road_surface_conditions*	Kategorikal	Kondisi permukaan jalan
18	*Light_conditions*	Kategorikal	Kondisi pencahayaan saat kecelakaan
19	*Weather_conditions*	Kategorikal	Kondisi cuaca saat kecelakaan
20	*Type_of_collision*	Kategorikal	Jenis tabrakan yang terjadi
21	*Number_of_vehicles_involved*	Numerik	Jumlah kendaraan yang terlibat
22	*Number_of_casualties*	Numerik	Jumlah korban dalam kecelakaan
23	*Vehicle_movement*	Kategorikal	Pergerakan kendaraan saat kecelakaan
24	*Casualty_class*	Kategorikal	Kategori korban
25	*Sex_of_casualty*	Kategorikal	Jenis kelamin korban
26	*Age_band_of_casualty*	Kategorikal	Rentang usia korban
27	*Casualty_severity*	Kategorikal	Tingkat keparahan korban
28	*Work_of_casuality*	Kategorikal	Pekerjaan korban
29	*Fitness_of_casuality*	Kategorikal	Kondisi fisik korban
30	*Pedestrian_movement*	Kategorikal	Pergerakan pejalan kaki
31	*Cause_of_accident*	Kategorikal	Penyebab utama kecelakaan
32	*Accident_severity*	Kategorikal	Tingkat keparahan kecelakaan (label/target klasifikasi)

Selain dari struktur dataset, isi dari dataset tingkat keparahan kecelakaan lalu lintas menampilkan 5 data teratas dan 5 data terbawah dan beberapa atribut, dapat dilihat pada Tabel 3.2.

Tabel 3. 2 Contoh Dataset Tingkat Keparahan Kecelakaan Lalu Lintas

Waktu	Hari	Kondisi Cuaca	...	Jumlah Kendaraan Terlibat	Jumlah Korban	Tingkat Keparahan Kecelakaan
17:02:00	Senin	Normal	...	2	2	Cedera Ringan
17:02:00	Senin	Normal	...	2	2	Cedera Ringan
17:02:00	Senin	Normal	...	2	2	Cedera Berat
01:06:00	Minggu	Normal	...	2	2	Cedera Ringan
01:06:00	Minggu	Normal	...	2	2	Cedera Ringan
...	...	...	...	...	...	...
16:15:00	Rabu	Normal	...	2	1	Cedera Ringan
18:00:00	Minggu	Normal	...	2	1	Cedera Ringan
13:55:00	Minggu	Normal	...	1	1	Cedera Berat
13:55:00	Minggu	Normal	...	2	1	Cedera Ringan
13:55:00	Minggu	Normal	...	2	1	Cedera Ringan

3.3 Perancangan Data

Tabel 3.2 merupakan data yang diperoleh dari dataset "*Road Traffic Severity Classification"* yang bersumber dari Kaggle. Dataset ini berisi data nyata terkait kecelakaan lalu lintas yang digunakan untuk mengklasifikasikan tingkat keparahan kecelakaan berdasarkan berbagai faktor yang memengaruhinya.

Pada penelitian ini, Pembatasan data dilakukan bertujuan agar proses *training* dan pengujian model *machine learning* dapat dilakukan dengan lebih cepat dan optimal. Selain itu, dengan menggunakan jumlah data yang terkontrol, proses eksperimen seperti perbandingan algoritma dapat dilakukan secara lebih efektif dan terukur. Kemudian variabel *Accident_Severity* dipilih sebagai variabel target karena secara langsung merepresentasikan tingkat dampak dari kecelakaan yang terjadi (Yumak et al., 2025).

Faktor seperti kondisi cuaca, kondisi jalan, pencahayaan, waktu kejadian, serta karakteristik kendaraan dan pengemudi memiliki pengaruh besar terhadap tingkat

keparahan kecelakaan. Kondisi cuaca seperti hujan atau kabut dapat menurunkan visibilitas dan meningkatkan risiko kecelakaan fatal. Selain itu, kondisi jalan dan pencahayaan juga memengaruhi kontrol kendaraan. Jumlah kendaraan yang terlibat dan jumlah korban turut mencerminkan tingkat keparahan kecelakaan. Oleh karena itu, pemilihan fitur dalam penelitian ini disesuaikan dengan faktor-faktor tersebut (Bekelcho et al., 2023).

Sehingga berdasarkan hal tersebut, fitur yang akan digunakan dalam penelitian ini adalah sebagai berikut:

1. Variabel Target

   Variabel *Accident_Severity* akan dijadikan sebagai target dalam penelitian ini. Fitur ini menunjukkan tingkat keparahan kecelakaan dengan kategori sebagai berikut :

   1. *Slight Injury*
   2. *Serious Injury*
   3. *Fatal Injury*

2. Fitur Waktu (Temporal)

   Fitur waktu digunakan untuk mengetahui kapan kecelakaan terjadi, yang berkaitan dengan aktivitas lalu lintas dan visibilitas (Bekelcho et al., 2023). Fitur Waktu dapati dilihat pada Tabel 3.3.

Tabel 3. 3 Fitur Waktu (*Temporal*)

Atribut	Tipe Data	RentangNilai/ Keterangan
Day_of_Week	Kategorikal	Hari kejadian (Senin–Minggu)
Time	Kategorikal	Waktu kejadian (Pagi, Siang, Sore, Malam)

3. Fitur Cuaca (*Weather*)

   Faktor cuaca memiliki korelasi terhadap frekuensi dan keparahan kecelakaan, sebab kondisi lingkungan memengaruhi kontrol kendaraan dan visibilitas pengemudi (Bekelcho et al., 2023). Fitur Cuaca dapat dilihat pada Tabel 3.4.

Tabel 3. 4 Fitur Cuaca (*Weather*)

Atribut	Tipe Data	RentangNilai/ Keterangan
Weather_conditions	Kategorikal	Normal, Hujan, Badai, Berawan, Berkabut

4. Fitur Pengemudi (*Driver*)

Fitur pengemudi digunakan karena perilaku dan pengalaman pengemudi sangat berpengaruh terhadap keselamatan berkendara (Bekelcho et al., 2023). Fitur Jarak dapat dilihat pada Tabel 3.6.

Tabel 3. 5 Fitur Pengemudi (*Driver*)

Atribut	Tipe Data	Keterangan
Age_band_of_driver	Kategorikal	Rentang usia pengemudi
Sex_of_driver	Kategorikal	Jenis kelamin pengemudi
Educational_level	Kategorikal	Tingkat pendidikan
Driving_experience	Kategorikal	Lama pengalaman mengemudi
Vehicle_driver_relation	Kategorikal	Hubungan dengan kendaraan

5. Fitur Kondisi Jalan (*Road Condition*)

Fitur kondisi jalan digunakan karena kondisi permukaan jalan memengaruhi kestabilan kendaraan (Bekelcho et al., 2023). Fitur Jarak dapat dilihat pada Tabel 3.7.

Tabel 3. 6 Fitur Kondisi Jalan (*Road Condition*)

Atribut	Tipe Data	RentangNilai / Keterangan
Road_surface_type	Kategorikal	Jenis permukaan jalan
Road_surface_conditions	Kategorikal	Kondisi permukaan jalan
Road_allignment	Kategorikal	Kelurusan atau kemiringan jalan
Lanes_or_Medians	Kategorikal	Kondisi jalur atau median

6. Fitur Kendaraan (*Vehicle*)

Fitur Jenis Kendaraan digunakan karena Jenis kendaraan berpengaruh terhadap tingkat risiko dan dampak kecelakaan (Bekelcho et al., 2023). Fitur Jarak dapat dilihat pada Tabel 3.8.

Tabel 3. 7 Fitur Jenis Kendaraan (*Vehicle*)

Atribut	Tipe Data	RentangNilai / Keterangan
Type_of_vehicle	Kategorikal	Jenis kendaraan
Owner_of_vehicle	Kategorikal	Kepemilikan kendaraan
Service_year_of_vehicle	Kategorikal	Lama penggunaan kendaraan
Defect_of_vehicle	Kategorikal	Kondisi kerusakan kendaraan
Number_of_vehicles_involved	Numerik	Jumlah kendaraan terlibat

7. Fitur Lingkungan dan Lokasi (*Environtment*)

Lingkungan jalan berpengaruh terhadap Kondisi lingkungan sekitar saat kecelakaan terjadi (Bekelcho et al., 2023). Fitur Jarak dapat dilihat pada Tabel 3.9.

Tabel 3. 8 Fitur Lingkungan (*Environtment*)

Atribut	Tipe Data	RentangNilai / Keterangan
Area_accident_occured	Kategorikal	Lokasi kecelakaan
Types_of_Junction	Kategorikal	Jenis persimpangan
Light_conditions	Kategorikal	Kondisi pencahayaan

8. Fitur Kecelakaan (*Accident Detail*)

Fitur ini berhubungan langsung dengan karakteristik kejadian kecelakaan seperti jenis tabrakan, penyebab kecelakaan dan jumlah korban (Bekelcho et al., 2023). Fitur ini dapat dilihat pada Tabel 3.10.

Tabel 3. 9 Fitur Kecelakaan (*Accident Detail*)

Atribut	Tipe Data	RentangNilai / Keterangan
Type_of_vehicle	Kategorikal	Jenis kendaraan
Owner_of_vehicle	Kategorikal	Kepemilikan kendaraan
Service_year_of_vehicle	Kategorikal	Lama penggunaan kendaraan
Defect_of_vehicle	Kategorikal	Kondisi kerusakan kendaraan
Number_of_vehicles_involved	Numerik	Jumlah kendaraan terlibat

9. Fitur Korban (*Casualty*)

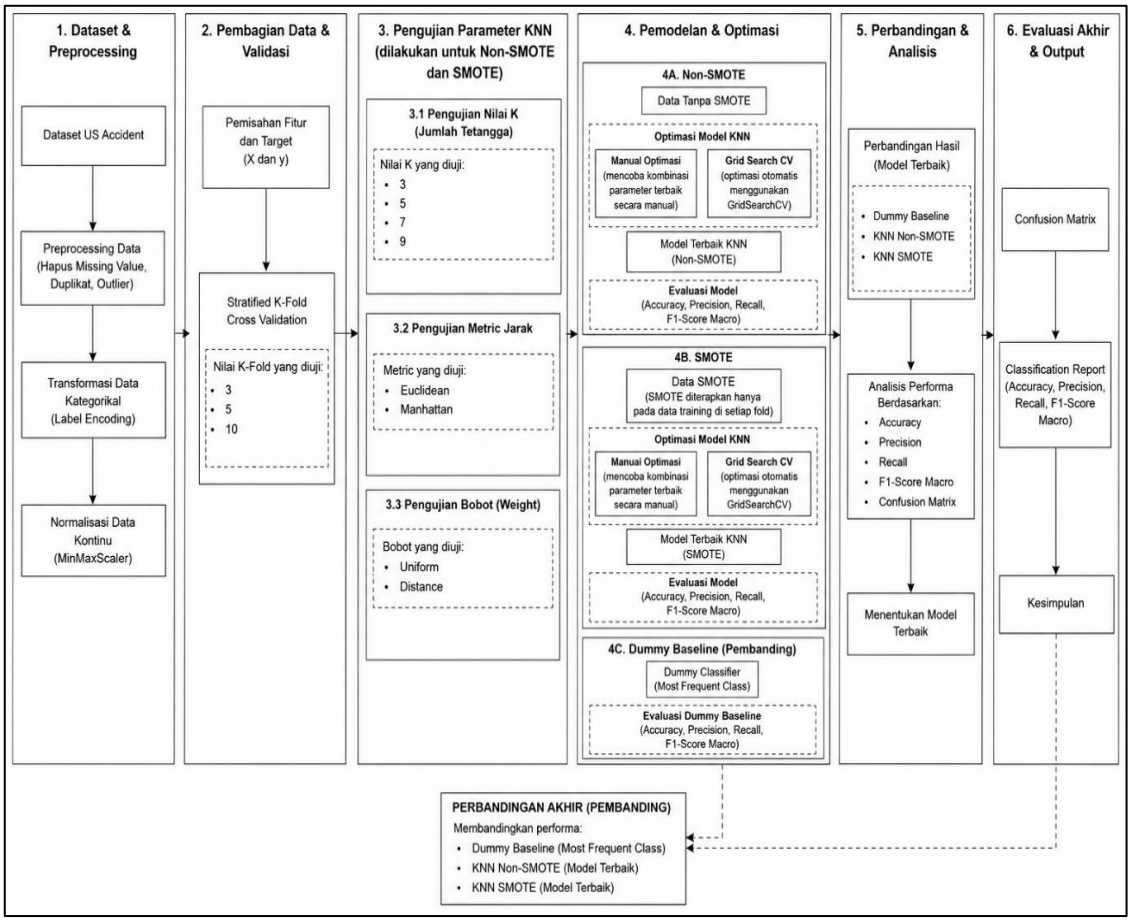
Fitur ini memberikan karakteristik tentang korban dalam kecelakaan (Bekelcho et al., 2023). Fitur ini dapat dilihat pada Tabel 3.11.

Tabel 3. 10 Fitur Korban (*Casualty*)

Atribut	Tipe Data	RentangNilai / Keterangan
Casualty_class	Kategorikal	Kategori korban
Sex_of_casualty	Kategorikal	Jenis kelamin korban
Age_band_of_casualty	Kategorikal	Rentang usia korban
Casualty_severity	Kategorikal	Tingkat keparahan korban
Work_of_casuality	Kategorikal	Pekerjaan korban
Fitness_of_casuality	Kategorikal	Kondisi fisik korban

3.4 Perancangan Proses/Algoritma

Tahapan Proses dalam penelitian ini meliputi, Preprocessing Data, Transformasi Data dan Normalisasi Data, pembagian Data, Implementasi Model, Evaluasi Model. Seperti yang terlihat pada Gambar 3.2.



Gambar 3. 2 Tahapan Proses

Berdasarkan Gambar 3.2, gambaran proses dapat diuraikan sebagai berikut:

1. *Data Processing*

   Pada tahap ini, dataset *Road Traffic Severity Classification* dipersiapkan sebelum digunakan dalam pemodelan. Proses preprocessing meliputi pembersihan data dari nilai kosong (*missing values*), penanganan data duplikat, serta penyesuaian format data agar konsisten. Selain itu, dilakukan *handling outlier* menggunakan metode Z-Score, di mana nilai data yang memiliki jarak lebih dari ambang batas tertentu terhadap rata-rata, dalam penelitian ini lebih dari 3, maka akan dianggap sebagai *outlier* (Yaro et al., 2023). Penanganan *outlier* ini bertujuan untuk mengurangi pengaruh nilai ekstrem yang dapat menyebabkan bias pada proses pembelajaran model, terutama pada algoritma yang sensitif terhadap jarak seperti KNN.

2. Transformasi dan Normalisasi Data

   Pada tahap transformasi, fitur kategorikal diubah menjadi bentuk numerik menggunakan Label Encoder. Metode ini memberikan label numerik unik pada setiap kategori sehingga data dapat diproses oleh algoritma machine learning. Selanjutnya, fitur numerik dinormalisasi menggunakan metode Min-Max Normalization, yang menskalakan nilai data ke dalam rentang 0 hingga 1 (Kumar & Bhardwaj, 2025). Normalisasi ini penting untuk memastikan setiap fitur memiliki kontribusi yang seimbang dalam proses perhitungan jarak, khususnya pada algoritma KNN yang bergantung pada jarak antar data.

3. Pembagian Data

   Data yang telah melalui tahap preprocessing kemudian dipisahkan menjadi fitur (X) dan target (y). Setelah itu, dilakukan pembagian data menggunakan metode Stratified K-Fold Cross Validation dengan nilai fold sebanyak 3, 5, dan 10. Penggunaan Stratified K-Fold bertujuan untuk menjaga proporsi distribusi kelas pada setiap fold sehingga distribusi data tetap seimbang selama proses validasi. Pada penelitian ini dilakukan dua skema pengujian, yaitu data tanpa penanganan ketidakseimbangan kelas (Non-SMOTE) dan data dengan penanganan ketidakseimbangan kelas menggunakan Synthetic Minority Oversampling Technique (SMOTE). Penerapan SMOTE dilakukan hanya pada data latih di setiap fold untuk menghindari data leakage serta memastikan evaluasi model dilakukan secara valid. Teknik SMOTE digunakan untuk menghasilkan data sintetis pada kelas minoritas sehingga distribusi kelas menjadi lebih seimbang.

4. Pengujian parameter KNN

   Tahap berikutnya adalah melakukan pengujian parameter pada algoritma K-Nearest Neighbor (KNN) untuk memperoleh kombinasi parameter terbaik dalam proses klasifikasi tingkat keparahan kecelakaan. Pengujian dilakukan pada dua skema, yaitu data tanpa penanganan ketidakseimbangan kelas (Non-SMOTE) dan data dengan penerapan Synthetic Minority Oversampling Technique (SMOTE). Parameter yang diuji meliputi nilai K-Fold Cross Validation sebesar 3, 5, dan 10, jumlah tetangga (K) sebesar 3, 5, 7, dan 9, metric distance berupa Euclidean dan Manhattan, serta bobot (weight) berupa uniform dan distance.

Proses pengujian dilakukan secara bertahap agar diperoleh konfigurasi parameter yang optimal. Tahap pertama dimulai dengan pengujian nilai K-Fold Cross Validation untuk menentukan jumlah fold terbaik berdasarkan performa evaluasi model. Selanjutnya, nilai K-Fold terbaik digunakan pada tahap pengujian jumlah tetangga (K) untuk menentukan nilai K yang menghasilkan performa paling baik. Setelah itu, dilakukan pengujian metric distance menggunakan kombinasi parameter terbaik yang telah diperoleh sebelumnya. Pada tahap terakhir, dilakukan pengujian parameter bobot (weight) untuk menentukan metode pembobotan terbaik antara uniform dan distance.

Selain pengujian parameter secara manual, penelitian ini juga menggunakan Grid Search Cross Validation sebagai metode optimasi otomatis untuk mencari kombinasi parameter terbaik secara menyeluruh. Hasil performa model KNN kemudian dibandingkan dengan Dummy Baseline Classifier sebagai pembanding dasar untuk mengetahui apakah model yang dibangun memiliki performa yang lebih baik dibandingkan prediksi sederhana berdasarkan kelas mayoritas. Seluruh proses pengujian dilakukan pada skema Non-SMOTE dan SMOTE untuk melihat pengaruh penanganan imbalance terhadap performa klasifikasi model.

5. Implementasi Model Dan Optimasi

   Pada tahap ini dilakukan implementasi algoritma KNN menggunakan dua pendekatan optimasi, yaitu manual optimization dan Grid Search Cross Validation (GridSearchCV). Pada manual optimization, pengujian parameter dilakukan secara bertahap berdasarkan kombinasi parameter yang telah ditentukan sebelumnya. Sedangkan pada Grid Search CV, proses pencarian parameter terbaik dilakukan secara otomatis dengan mencoba seluruh kombinasi parameter yang tersedia untuk memperoleh performa terbaik berdasarkan hasil validasi silang. Selain model KNN, penelitian ini juga menggunakan Dummy Baseline dengan strategi most frequent class sebagai model pembanding. Dummy baseline digunakan untuk mengetahui apakah model KNN yang dibangun benar-benar memiliki performa yang lebih baik dibandingkan model sederhana yang hanya memprediksi kelas mayoritas.

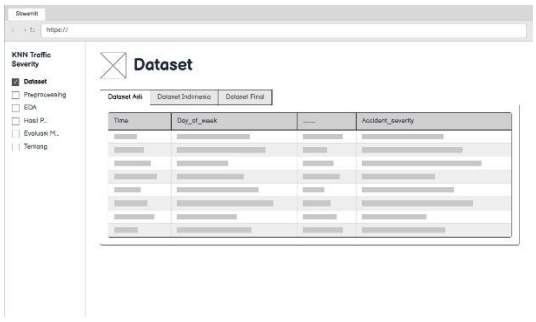
6. Evaluasi Model

   Evaluasi performa model dilakukan menggunakan Confusion Matrix dan Classification Report. Confusion Matrix digunakan untuk melihat distribusi prediksi benar dan salah pada setiap kelas, sedangkan Classification Report digunakan untuk menampilkan nilai accuracy, precision, recall, dan F1-score macro. Pada penelitian ini, F1-score macro digunakan sebagai salah satu metrik utama evaluasi karena dataset memiliki distribusi kelas yang tidak seimbang. Penggunaan metrik ini bertujuan agar performa model pada seluruh kelas, termasuk kelas minoritas, dapat dievaluasi secara lebih adil dan menyeluruh. Hasil evaluasi dari model Dummy Baseline, KNN Non-SMOTE, dan KNN SMOTE kemudian dibandingkan untuk menentukan model terbaik dalam melakukan klasifikasi tingkat keparahan kecelakaan.

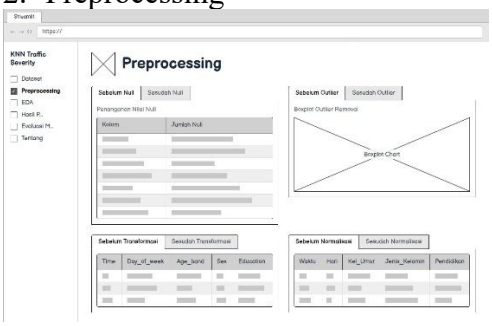
3.5 Perancangan Tampilan

Perancangan tampilan sistem pada penelitian ini dilakukan menggunakan framework berbasis web, yaitu Streamlit. Pemilihan Streamlit didasarkan pada kemampuannya dalam menyajikan hasil analisis data dan evaluasi model machine learning secara interaktif, sederhana, serta mudah diintegrasikan dengan bahasa pemrograman Python. Tampilan sistem dirancang untuk mendukung proses analisis data dan pengujian model K-Nearest Neighbor (KNN) dalam memprediksi tingkat keparahan kecelakaan lalu lintas, sehingga pengguna dapat memahami hasil penelitian secara lebih mudah dan sistematis. Hal ini dapat dilihat pada Gambar 3.3.

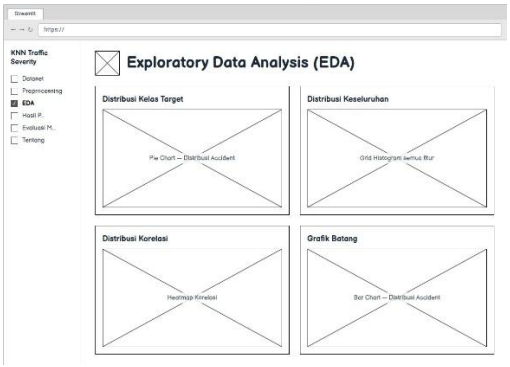
1. Halaman Dataset



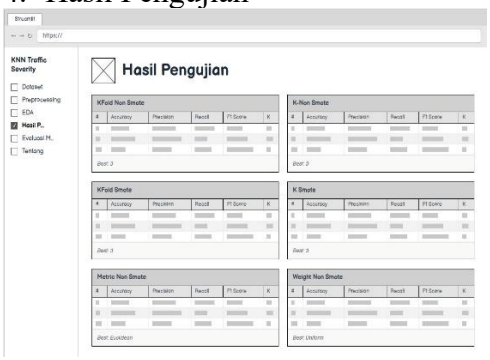
2. Preprocessing



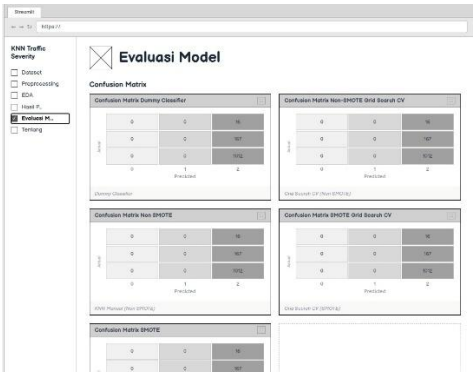
3. EDA



4. Hasil Pengujian



5. Evaluasi Model



Gambar 3. 3 Perancangan Tampilan

Berdasarkan Gambar 3.3, halaman tampilan dapat dibagi sebagai sebagai berikut:

1. Halaman Dataset

   Halaman dataset merupakan tampilan awal yang berfungsi untuk memberikan gambaran umum mengenai data yang digunakan dalam penelitian. Pada halaman ini ditampilkan beberapa baris pertama dari dataset sebagai bentuk pratinjau data, serta informasi terkait jumlah data dan struktur variabel yang tersedia. Penyajian ini bertujuan agar pengguna dapat memahami karakteristik dasar dari dataset sebelum dilakukan proses analisis lebih lanjut. Dengan adanya tampilan ini, pengguna dapat

mengetahui jenis data yang digunakan serta memastikan bahwa data telah sesuai untuk digunakan dalam proses pemodelan.

2. Halaman Preprocessing

   Halaman preprocessing ini ditampilkan hasil dari setiap tahap preprocessing, yaitu penanganan nilai null, penanganan outlier, transformasi data, dan normalisasi data. Masing-masing tahapan disajikan dalam bentuk tab sebelum dan sesudah sehingga pengguna dapat melihat perbandingan kondisi data pada setiap proses yang dilakukan. Penyajian ini bertujuan agar pengguna dapat memahami perubahan data yang terjadi selama proses preprocessing. Tampilan ini memastikan bahwa data telah melalui tahapan prapemrosesan dengan baik dan siap digunakan dalam proses pemodelan.

3. Halaman *Exploratory Data Analysis* (EDA)

   Halaman Exploratory Data Analysis (EDA) digunakan untuk menampilkan hasil eksplorasi data guna memahami pola dan karakteristik dataset. Pada halaman ini disajikan berbagai visualisasi seperti grafik distribusi kelas, histogram untuk melihat sebaran data, boxplot untuk mendeteksi outlier, serta heatmap untuk melihat hubungan antar variabel. Visualisasi tersebut membantu pengguna dalam memahami kondisi data secara lebih mendalam sebelum dilakukan proses pemodelan, sehingga dapat mendukung pengambilan keputusan yang lebih tepat dalam tahap preprocessing dan pemilihan model.

4. Halaman Hasil Pengujian

   Halaman hasil pengujian merupakan tampilan yang berfungsi untuk menampilkan hasil evaluasi model K-Nearest Neighbor (KNN) berdasarkan beberapa skenario pengujian yang dilakukan dalam penelitian. Pada halaman ini ditampilkan hasil pengujian menggunakan K-Fold Cross Validation, variasi nilai K, metrik jarak, dan bobot (weight) pada kondisi tanpa SMOTE dan dengan SMOTE. Setiap skenario disajikan dalam bentuk tabel yang memuat nilai accuracy, precision, recall, dan F1-score, sehingga pengguna dapat membandingkan performa model pada setiap kombinasi parameter yang diuji. Penyajian ini bertujuan agar pengguna dapat memahami hasil pengujian serta pengaruh setiap parameter terhadap kinerja model. Dengan adanya tampilan ini, pengguna dapat mengidentifikasi kombinasi parameter

yang menghasilkan performa terbaik dan digunakan sebagai dasar dalam pemilihan model yang optimal.

5. Halaman Evaluasi Model

   Halaman evaluasi model digunakan untuk menampilkan hasil dari proses pengujian yang telah dilakukan. Pada halaman ini disajikan confusion matrix yang menggambarkan hasil klasifikasi model, serta classification report yang berisi nilai akurasi, presisi, recall, dan F1-score. Selain itu, halaman ini juga menampilkan perbandingan performa antara model yang menggunakan teknik SMOTE dan yang tidak menggunakan SMOTE. Informasi ini sangat penting dalam menentukan model terbaik yang akan digunakan dalam penelitian.

3.6 Perancangan Pengujian

Perancangan pengujian dalam penelitian ini bertujuan untuk mengevaluasi performa model KNN secara komprehensif dengan mempertimbangkan variasi skema validasi, teknik penanganan ketidakseimbangan data, serta kombinasi parameter model. Pengujian dilakukan menggunakan beberapa skenario agar diperoleh konfigurasi model yang paling stabil dan optimal dalam memprediksi tingkat keparahan kecelakaan lalu lintas. Berikut Skenario pengujian yang akan dilakukan dalam penelitian ini:

1. Pengujian Variasi K-Fold

   Pengujian dilakukan dengan menerapkan beberapa nilai K-Fold Cross Validation**,** yaitu K = 3, 5, dan 10. Tujuan dari variasi ini adalah untuk melihat pengaruh jumlah lipatan data terhadap performa model. Nilai K yang kecil, seperti K=2 dan K=3, memungkinkan evaluasi model dengan proporsi data latih yang lebih besar, namun berpotensi menghasilkan variansi yang tinggi. Sebaliknya, nilai K yang lebih besar seperti K=5 dan K=10 memberikan evaluasi yang lebih stabil karena setiap data memiliki kesempatan yang lebih seimbang untuk menjadi data latih dan data uji. Dengan membandingkan hasil dari keempat nilai K tersebut, dapat diketahui skema validasi mana yang memberikan performa model paling konsisten.

2. Pengujian pengaruh SMOTE

Pada skenario ini, pengujian dilakukan dengan membandingkan dua kondisi data, yaitu data tanpa SMOTE dan data dengan SMOTE. Data tanpa SMOTE digunakan sebagai baseline untuk melihat performa model pada distribusi kelas asli yang tidak seimbang. Selanjutnya, SMOTE diterapkan untuk menghasilkan data sintetis pada kelas minoritas sehingga distribusi kelas menjadi lebih seimbang. Perbandingan hasil dari kedua kondisi ini bertujuan untuk mengetahui sejauh mana SMOTE mampu meningkatkan kemampuan model dalam mengenali kelas dengan jumlah data yang lebih sedikit, khususnya pada prediksi tingkat keparahan kecelakaan.

3. Pengujian variasi perhitungan jarak

   Pengujian juga dilakukan dengan menggunakan beberapa metode perhitungan jarak pada algoritma KNN, yaitu *Euclidean Distance, Manhattan Distance. Euclidean Distance* digunakan untuk mengukur jarak lurus antar data dalam ruang multidimensi, sedangkan *Manhattan Distance* menghitung jarak berdasarkan penjumlahan selisih absolut antar fitur. Variasi ini bertujuan untuk mengidentifikasi metode jarak yang paling sesuai dengan karakteristik data kecelakaan lalu lintas yang telah dinormalisasi.

4. Pengujian variasi bobot tetangga (*weights*)

   Dalam skenario ini, pengujian dilakukan dengan menggunakan dua jenis pembobotan tetangga pada KNN, yaitu uniform dan distance. Pada bobot uniform, seluruh tetangga memiliki kontribusi yang sama dalam proses klasifikasi. Sebaliknya, pada bobot distance, tetangga yang memiliki jarak lebih dekat terhadap data uji akan memiliki pengaruh yang lebih besar. Pengujian ini bertujuan untuk mengetahui apakah pemberian bobot berdasarkan jarak mampu meningkatkan akurasi prediksi dibandingkan dengan pembobotan yang seragam.

5. Pengujian K tetangga KNN

   Dalam Skenario ini algoritma KNN akan diuji menggunakan beberapa variasi nilai parameter K, yaitu 3, 5, 7, dan 9, untuk menentukan jumlah tetangga terdekat yang paling optimal dalam proses klasifikasi. Pemilihan nilai K ganjil bertujuan untuk menghindari kemungkinan terjadinya nilai voting yang sama pada saat penentuan kelas, serta untuk memperoleh keseimbangan antara kompleksitas model dan kemampuan generalisasi. Setiap nilai K akan dievaluasi menggunakan metode pengujian seperti cross validation guna membandingkan performa model berdasarkan

metrik evaluasi seperti accuracy, precision, recall, dan F1-score, sehingga dapat ditentukan nilai K yang memberikan hasil terbaik terhadap dataset yang digunakan (Uddin et al., 2022).

6. Pengujian Kombinasi Parameter melalui *Grid Search* CV

   Seluruh variasi parameter yang meliputi jumlah lipatan K-Fold, metode jarak, jenis bobot tetangga, serta penggunaan SMOTE dikombinasikan dan diuji menggunakan Grid Search CV. Pendekatan ini memungkinkan pencarian konfigurasi parameter terbaik secara sistematis berdasarkan hasil evaluasi pada setiap kombinasi skenario. Dengan demikian, model yang dihasilkan tidak hanya memiliki performa terbaik pada satu kondisi tertentu, tetapi juga menunjukkan kestabilan performa pada berbagai skema pengujian.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Pengolahan Data

Pengolahan data pada penelitian ini dilakukan melalui beberapa tahapan teknis sebelum dataset digunakan pada proses pemodelan. Dataset publik *Road Traffic Severity Classification* yang bersumber dari Kaggle terlebih dahulu melalui tahap *preprocessing*, yang meliputi penanganan nilai *null* menggunakan metode *imputation*, pemeriksaan data duplikat, serta penanganan *outlier* menggunakan metode *Z-Score*. Selanjutnya, data kategorikal ditransformasikan menjadi bentuk numerik menggunakan metode *Label Encoding*, kemudian dinormalisasi menggunakan metode *Min-Max Scaling* agar seluruh fitur memiliki rentang skala yang seragam. Setelah tahap pengolahan data selesai dilakukan, dataset dibagi menjadi dua skema, yaitu data tanpa penerapan SMOTE dan data dengan penerapan SMOTE, untuk selanjutnya digunakan pada proses implementasi dan optimasi model algoritma K-Nearest Neighbor (KNN). Rincian dari setiap teknik pengolahan data tersebut diuraikan pada sub-bab berikut.

4.2 Penerapan Proses

Proses yang dilakukan pada penelitian ini meliputi, *preprocessing data* seperti pengolahan nilai duplikat dan nilai null. Mengatasi nilai Outlier, normalisasi dan standarisasi data. Adapun hasil penerapan proses dapat dilihat pada uraian berikut:

1. *Data Processing*

Data Processing data merupakan tahapan awal yang dilakukan untuk meningkatkan kualitas dataset sebelum digunakan pada proses pemodelan. Tahapan ini bertujuan untuk membersihkan data, mengatasi permasalahan yang terdapat pada dataset, serta mengubah data agar sesuai dengan kebutuhan algoritma *K-Nearest Neighbors* (KNN). Pada penelitian ini, tahapan *preprocessing* meliputi penanganan nilai *null*, penanganan data duplikat, penanganan *outlier*, transformasi data, dan normalisasi data.

1. Penanganan Nilai Null

Penanganan nilai null dilakukan karena keberadaan data yang kosong dapat memengaruhi proses analisis dan pelatihan model. Pada penelitian ini digunakan metode *imputation*, yaitu pengisian nilai median untuk atribut numerik dan nilai yang paling sering muncul (*most frequent*) untuk atribut kategorikal sehingga informasi pada dataset tetap dipertahankan.

Proses *imputation* dilakukan dengan membagi fitur ke dalam dua jenis, yaitu fitur numerik dan fitur kategorikal. Untuk fitur numerik, nilai null diisi menggunakan nilai median dari masing-masing kolom karena lebih tahan terhadap pengaruh outlier dibandingkan nilai rata-rata (*mean*). Sementara itu, untuk fitur kategorikal digunakan nilai yang paling sering muncul (*most frequent*) agar distribusi data tetap terjaga. Setelah proses imputasi selesai dilakukan, dilakukan pengecekan kembali untuk memastikan seluruh nilai kosong telah berhasil ditangani.

Tabel 4. 1 Hasil Penanganan Nilai Null

Keterangan	Sebelum Penanganan	Sesudah Penanganan
Jumlah Data	12.316	12.316
Jumlah Atribut	32	32
Nilai Null	Terdapat 16 kolom	Tidak terdapat nilai null (0)

Berdasarkan Tabel 4.1, sebelum dilakukan preprocessing masih terdapat beberapa atribut yang memiliki nilai kosong (*missing value*). Dataset yang digunakan terdiri dari 32 kolom, yaitu 31 kolom fitur (variabel independen) dan 1 kolom label target (*Accident_Severity*), sehingga jumlah atribut yang ditampilkan pada Tabel 4.1 mencakup keseluruhan kolom tersebut. Setelah dilakukan proses imputation, seluruh nilai kosong berhasil ditangani sehingga tidak terdapat lagi nilai null pada dataset. Jumlah data tetap sebanyak 12.316 data karena metode imputasi mengisi nilai yang kosong tanpa menghapus baris data. Dengan demikian, dataset menjadi lebih lengkap, konsisten, dan siap digunakan pada tahapan preprocessing berikutnya.

2. Penanganan Nilai Duplikat

Tahapan berikutnya adalah pemeriksaan data duplikat. Data duplikat dapat menyebabkan informasi yang sama dihitung lebih dari satu kali sehingga berpotensi

memengaruhi hasil analisis dan proses pelatihan model. Oleh karena itu, pemeriksaan data duplikat dilakukan untuk memastikan bahwa setiap data pada *dataset* bersifat unik dan tidak terjadi pengulangan data.

Tabel 4. 2 Hasil Penanganan Data Duplikat

Keterangan	Sebelum Penanganan	Sesudah Penanganan
Jumlah Data	12.316	12.316
Data Duplikat	0	0

Berdasarkan Tabel 4.2, hasil pemeriksaan menunjukkan bahwa *dataset* tidak memiliki data duplikat sehingga tidak dilakukan proses penghapusan data. Jumlah data tetap sebanyak 12.316 data karena seluruh data bersifat unik. Dengan demikian, *dataset* telah memenuhi kualitas data pada aspek duplikasi dan siap digunakan pada tahapan *preprocessing* berikutnya.

3. Penanganan *Outlier*

Penanganan *outlier* dilakukan menggunakan metode *Z-Score* dengan nilai ambang (*threshold*) sebesar 3. Data yang memiliki nilai *Z-Score* lebih dari 3 atau kurang dari -3 dianggap sebagai *outlier* sehingga dihapus dari *dataset*. Proses ini bertujuan untuk mengurangi pengaruh nilai ekstrem yang dapat memengaruhi kualitas data dan performa model klasifikasi.

Tabel 4. 3 Hasil Penanganan *Outlier*

Keterangan	Jumlah Data
Sebelum Penanganan *Outlier*	12.316
Setelah Penanganan *Outlier*	11.941
Data yang Dihapus	375

Berdasarkan Tabel 4.3, sebanyak 375 data teridentifikasi sebagai *outlier* sehingga dihapus dari *dataset*. Jumlah data berkurang dari 12.316 menjadi 11.941 data. Penghapusan *outlier* dilakukan dengan menghitung nilai *Z-Score* pada setiap fitur numerik, kemudian hanya data yang memiliki nilai *Z-Score* berada pada rentang yang ditentukan yang dipertahankan. Hasil tersebut menunjukkan bahwa terdapat sejumlah data dengan nilai ekstrem yang tidak sesuai dengan pola umum *dataset*. Setelah proses penanganan *outlier*, *dataset* menjadi lebih bersih, lebih representatif, dan lebih siap digunakan pada tahap pemodelan sehingga diharapkan dapat meningkatkan performa model dalam proses klasifikasi.

2. Transformasi Data

Proses selanjutnya adalah transformasi data kategorikal. Transformasi ini dilakukan karena sebagian besar fitur dalam *dataset* masih berbentuk data kategorikal (teks), sedangkan algoritma *K-Nearest Neighbors* (KNN) hanya dapat mengolah data dalam bentuk numerik. Oleh karena itu, data kategorikal perlu diubah menjadi representasi numerik agar dapat diproses oleh model.

Pada penelitian ini, proses transformasi dilakukan menggunakan metode *Label Encoding*. Metode ini mengubah setiap kategori pada suatu atribut menjadi kode numerik berupa bilangan bulat. Transformasi dilakukan pada seluruh atribut yang bertipe *object* sehingga seluruh data dapat digunakan pada proses pemodelan.

Tabel 4. 4 Contoh Hasil Transformasi Data

Sebelum Transformasi	Sesudah Transformasi
Senin	6
Kamis	1
Jumat	0
Hujan	3
Berkabut	1
Cedera Ringan	0
Cedera Berat	1
Cedera Fatal	2

Nilai pada kolom Sesudah Transformasi merupakan hasil proses *Label Encoding* menggunakan pustaka *scikit-learn*. Setiap kategori diubah menjadi kode numerik secara otomatis oleh sistem. Nilai kode yang dihasilkan dapat berbeda bergantung pada urutan kategori yang diproses oleh *LabelEncoder*.

Berdasarkan Tabel 4.4, seluruh data kategorikal berhasil ditransformasikan menjadi data numerik menggunakan metode *Label Encoding*. Proses ini memungkinkan seluruh atribut diproses oleh algoritma *K-Nearest Neighbors* (KNN) tanpa mengubah informasi yang terkandung pada setiap kategori. Setelah proses transformasi selesai dilakukan, seluruh atribut pada *dataset* telah berada dalam bentuk numerik sehingga siap digunakan pada tahap normalisasi dan pemodelan.

3. Normalisasi Data

Tahapan selanjutnya adalah normalisasi data. Normalisasi dilakukan karena perbedaan skala antar fitur dapat memengaruhi kualitas hasil analisis maupun proses pelatihan model. Fitur yang memiliki rentang nilai lebih besar berpotensi mendominasi perhitungan jarak pada algoritma *K-Nearest Neighbors* (KNN), sehingga dapat memengaruhi hasil klasifikasi.

Pada penelitian ini, normalisasi dilakukan menggunakan metode *Min-Max Scaling*. Metode ini mengubah nilai setiap atribut numerik ke dalam rentang 0 hingga 1 tanpa mengubah hubungan antar data secara signifikan. Sebelum proses normalisasi dilakukan, *dataset* dipisahkan menjadi variabel fitur (*X*) dan variabel target (*y*), di mana variabel *Accident_Severity* digunakan sebagai target. Proses normalisasi hanya diterapkan pada variabel fitur (*X*) menggunakan *MinMaxScaler*.

Tabel 4. 5 Contoh Hasil Normalisasi Data

Sebelum Normalisasi	Setelah Normalisasi
6	1.0
1	0.16
0	0.0
3	0.6
1	0.1
0	0.0
1	0.5

Nilai pada kolom Setelah Normalisasi merupakan hasil transformasi menggunakan metode *Min-Max Scaling*, sehingga seluruh data numerik berada pada rentang nilai 0 sampai 1.

Berdasarkan Tabel 4.5, proses normalisasi berhasil mengubah seluruh atribut numerik ke dalam rentang nilai 0–1 menggunakan metode *Min-Max Scaling*. Dengan demikian, setiap atribut memiliki skala yang seragam sehingga tidak ada atribut yang mendominasi perhitungan jarak pada algoritma *K-Nearest Neighbors* (KNN). Setelah proses normalisasi selesai dilakukan, *dataset* siap digunakan pada tahap pelatihan dan evaluasi model.

Setelah seluruh tahapan preprocessing selesai dilakukan, diperoleh dataset yang siap digunakan pada tahap pemodelan. Dataset hasil preprocessing terdiri dari 11.941 data

kecelakaan dengan 32 kolom, yang meliputi 31 fitur (variabel independen) dan 1 variabel target, yaitu Tingkat Keparahan Kecelakaan (*Accident Severity*).

Sebelum digunakan pada proses pemodelan, seluruh atribut yang bertipe kategorikal (object) diubah menjadi data numerik menggunakan metode Label Encoding dari pustaka *scikit-learn*. Proses tersebut dilakukan secara otomatis dengan fungsi LabelEncoder().fit_transform() pada setiap kolom kategorikal sehingga setiap kategori memperoleh kode numerik tertentu. Tujuan transformasi ini adalah agar seluruh data dapat diproses oleh algoritma K-Nearest Neighbor (KNN) yang hanya menerima masukan berupa data numerik.

Tabel 4. 6 Data Hasil *Preprocessing*

Hari dalam Seminggu	Kelompok Usia Pengemudi	Jenis Kelamin Pengemudi	...	Pergerakan Pejalan Kaki	Penyebab Kecelakaan	Tingkat Keparahan Kecelakaan
1.0	0.0	0.0	...	0.5	0.7	1
1.0	0.2	0.0	...	0.5	0.6	1
1.0	0.0	0.0	...	0.5	0.05	0
0.3	0.0	0.0	...	0.5	0.0	1
0.3	0.0	0.0	...	0.5	0.6	1
...	...	...	...	...	...	...
0.5	0.2	0.0	...	0.5	0.9	1
0.3	0.5	0.0	...	0.5	0.9	1
0.3	0.7	0.0	...	0.5	0.0	0
0.3	0.0	0.5	...	0.5	0.5	1
0.3	0.00	0.0	...	0.7	0.0	1

Tabel 4.6 merupakan hasil proses preprocessing menampilkan 5 data pertama dan 5 data terakhir dari 11.941 data hasil preprocessing sebagai contoh struktur dataset yang akan digunakan pada proses pelatihan model.

4. Pembagian Data

Pembagian data dalam penelitian ini terbagi menjadi dua skema data, yaitu data kecelakaan tanpa SMOTE dan data kecelakaan SMOTE. Setelah data dibagi menjadi dua skema ini, data akan dibagi menggunakan pembagian *Kfold Cross Validation*. Pembagian data dapat diuraikan sebagai berikut:

1. Skema Data Tanpa SMOTE

Skema data kecelakaan tanpa penerapan metode *Synthetic Minority Over-sampling Technique* (*SMOTE*) merupakan hasil dari proses *preprocessing* tanpa dilakukan penyeimbangan jumlah data pada setiap kelas. Sebelum proses pembagian data, variabel target Accident_Severity yang semula berbentuk data kategorikal telah ditransformasikan menggunakan metode *Label Encoding*. Proses tersebut mengubah setiap kategori tingkat keparahan kecelakaan menjadi kode numerik sehingga dapat diproses oleh algoritma *K-Nearest Neighbors* (KNN). Hasil transformasi tersebut menghasilkan tiga label, yaitu 0, 1, dan 2, yang masing-masing merepresentasikan kategori tingkat keparahan kecelakaan. Selanjutnya dilakukan perhitungan jumlah data pada setiap label untuk mengetahui distribusi kelas sebelum penerapan metode *SMOTE*. Rincian jumlah data pada masing-masing label dapat dilihat pada Tabel 4.7.

Tabel 4. 7 Jumlah data Skema Data tanpa SMOTE

Label (*Label Encoding*)	Kategori Tingkat Keparahan	Jumlah Data
0	Cedera Fatal	152
1	Cedera Serius	1.672
2	Cedera Ringan	10.117

Berdasarkan Tabel 4.7, hasil *Label Encoding* mengubah variabel target Accident_Severity menjadi tiga label numerik. Distribusi data menunjukkan bahwa label 2 (Cedera Ringan) memiliki jumlah data paling banyak, sedangkan label 0 (Cedera Fatal) memiliki jumlah data paling sedikit. Kondisi tersebut menunjukkan adanya ketidakseimbangan kelas (*class imbalance*), sehingga pada penelitian ini diterapkan metode *SMOTE* untuk menyeimbangkan jumlah data pada setiap kelas.

2. Penerapan SMOTE dan Skema Data SMOTE

Metode SMOTE merupakan teknik yang digunakan untuk menghasilkan data sintetis baru pada kelas minoritas. Tujuan dari metode ini adalah untuk menyeimbangkan jumlah data antar kelas sehingga distribusi data menjadi lebih merata. Pada penelitian ini, penerapan SMOTE dilakukan pada dataset *road traffic severity classification* yang memiliki ketidakseimbangan jumlah data pada setiap label tingkat keparahan kecelakaan. Sebagai ilustrasi perhitungan manual, digunakan data sampel sederhana yang diambil dari dataset, seperti ditunjukkan pada Tabel 4.8.

Tabel 4. 8 Data Sampel SMOTE

Data	Waktu	Hari	Kelompok Usia Pengemudi	…	Pengalaman Mengemudi	Jenis Kendaraan	Kepemilikan Kendaraan	Tingkat Keparahan
D1	0.4	1.0	0.0	…	0.0	0.3	0.3	1
D2	0.4	1.0	0.2	…	0.7	0.3	0.3	1
D3	0.4	1.0	0.0	…	0.0	0.3	0.3	0
D4	0.5	0.3	0.0	…	0.3	1.0	1.0	1
D5	0.5	0.3	0.0	…	0.2	0.3	0.3	1

Tabel 4.8 merupakan lima data pertama hasil normalisasi data menggunakan metode Label Encoding. Data tersebut diambil langsung dari dataset penelitian dan digunakan sebagai ilustrasi proses penerapan metode SMOTE. Nilai numerik pada setiap atribut merupakan hasil proses pengkodean kategori ke dalam bentuk bilangan bulat.

Diketahui:

Pencarian tetangga terdekat dari dokumen ke-4 menggunakan Persamaan Euclidean Distance pada Persamaan 2.1 adalah sebagai berikut.

$d(D4, D5) = \sqrt{(0.5 - 0.5)^2 + (0.3 - 0.3)^2 + (0.0 - 0.0)^2 + (0.3 - 0.2)^2}$

$= \sqrt{0 + 0 + 0 + 0.01}$

$= \sqrt{0.1}$

$= 0.3$

Sehingga diperoleh:

$x_1 = D4$

$x_{neighbor} = D5$

Selanjutnya dilakukan proses pembangkitan data sintetis menggunakan Persamaan 2.4 sebagai berikut.

$x_{new} = x_1 + \text{rand}(0,5) \times (x_{neighbor} - x_1)$

Misalkan nilai acak (*random*) = 0,5.

Maka hasil perhitungan SMOTE dapat dilihat Tabel 4.9.

Tabel 4. 9 Perhitungan SMOTE

Fitur	x_1 (D4)	x_{tetangga} (D5)	Data Sintetis
Waktu	0.5	0.5	0.5 + 0.5 × (0.5 – 0.5) = 0.5
Hari	0.3	0.3	0.3 + 0.5 × (0.3 – 0.3) = 0.3
Kelompok Usia Pengemudi	0.0	0.0	0.0 + 0.5 × (0.0 – 00.) = 0
Pengalaman Mengemudi	0.3	0.2	0.3 + 0.5 × (0.2 – 0.3) = 0.25

Setelah dilakukan perhitungan SMOTE pada semua fitur, maka hasil akhir SMOTE dapat dilihat pada Tabel 4.10.

Tabel 4. 10 Data Hasil SMOTE

Data	Waktu	Hari	Kelompok Usia Pengemudi	Pengalaman Mengemudi	Tingkat Keparahan
D1	0.4	1.0	0.0	0.0	1
D2	0.4	1.0	0.2	0.7	1
D3	0.4	1.0	0.0	0.0	0
D4	0.5	0.3	0.0	0.3	1
D5	0.5	0.3	0.0	0.2	1
Duji	0.5	0.3	0.0	0.25	1

Setelah dilakukan penerapan metode *Synthetic Minority Oversampling Technique* (*SMOTE*), jumlah data pada kelas minoritas berhasil diseimbangkan dengan menghasilkan data sintetis pada kelas yang memiliki jumlah data lebih sedikit, yaitu label 0 dan label 1. Proses ini membuat distribusi data menjadi lebih seimbang sehingga dapat mengurangi bias model terhadap kelas mayoritas. Hasil penerapan *SMOTE* menunjukkan bahwa jumlah data pada setiap kelas menjadi sama, yaitu masing-masing sebanyak 10.117 data. Hasil tersebut dapat dilihat pada Tabel 4.11.

Tabel 4. 11 Jumlah Data Kecelakaan Setelah SMOTE

Label	Jumlah
0	10.117
1	10.117
2	10.117

3. Penerapan dan Pembagian K-Fold Cross Validation

Setelah data dibagi menjadi dua skema, yaitu skema data tanpa *SMOTE* dan data dengan *SMOTE*, masing-masing skema dievaluasi menggunakan metode *Stratified K-Fold Cross Validation*. Berdasarkan implementasi pada program, nilai *K-Fold* yang digunakan adalah K = 3, K = 5, dan K = 10. Pemilihan beberapa nilai *K* tersebut bertujuan untuk memperoleh model dengan performa terbaik melalui proses validasi silang.

Jumlah data awal adalah: N = 12.316 Data

Perhitungan Jumlah Data Jumlah data pada setiap fold dihitung dengan persamaan 2.4:

K = 3 => $\frac{12.316}{3} = 4.105$

Data Uji = 4.105

Data Latih = 12.316 – 4.105 = 8.211

- K = 5 => $\frac{12.316}{5} = 2.463$

  Data Uji = 2.463

  Data Latih = 12.316 – 2.463 = 9.853

- K = 10 => $\frac{12.316}{10} = 1.231$

  Data Uji = 1.231

  Data Latih = 12.316 - 1.231 = 11.085

Sehingga setelah jumlah data dibagi, maka hasil pembagian data dapat dilihat pada Tabel 4.12, sebagai contoh untuk K=5.

Tabel 4. 12 Hasil Jumlah data pembagian K-Fold K = 5

Iterasi	Data Uji	Data Latih
1	Fold 1 (2.463)	Fold 2–5 (9.853)
2	Fold 2 (2.463)	Fold 1,3,4,5
3	Fold 3 (2.463)	Fold 1,2,4,5
4	Fold 4 (2.463)	Fold 1,2,3,5
5	Fold 5 (2.463)	Fold 1-4

5. Implementasi Model dan Optimasi

Pada tahap ini dilakukan perhitungan manual K-Nearest Neighbor untuk memperjelas proses klasifikasi yang digunakan dalam penelitian. Berdasarkan notebook penelitian, dataset awal terdiri dari 32 kolom, di mana kolom Tingkat Keparahan Kecelakaan dipisahkan sebagai target (y), sedangkan 31 kolom lainnya digunakan sebagai fitur (X). Sebelum dilakukan normalisasi, seluruh data kategorikal telah diubah menjadi numerik menggunakan Label Encoding, termasuk kolom target, sehingga label kelas yang muncul berupa angka 0 dan 1, dengan 0 merepresentasikan Cedera Berat dan 1 merepresentasikan Cedera Ringan. Setelah itu, seluruh fitur dinormalisasi menggunakan MinMaxScaler sehingga nilainya berada pada rentang 0 sampai 1.

Karena jumlah fitur yang digunakan sebanyak 31 kolom tidak memungkinkan untuk ditampilkan seluruhnya dalam satu tabel, maka pada bagian ini hanya ditampilkan beberapa fitur representatif dengan format horizontal yang sama seperti Tabel 4.8. Adapun proses perhitungan jarak pada implementasi model tetap menggunakan seluruh 31 fitur hasil preprocessing. Data D1 digunakan sebagai data uji, sedangkan D2 sampai D5 digunakan sebagai data latih. Data sampel yang digunakan ditunjukkan pada Tabel 4.13.

Tabel 4. 13 Data KNN

Data	Waktu	Hari	Kelompok Usia Pengemudi	…	Pengalaman Mengemudi	Jenis Kendaraan	Kepemilikan Kendaraan	Tingkat Keparahan
D1	0.4	1.0	0.0	…	0.0	0.3	0.3	1
D2	0.4	1.0	0.2	…	0.7	0.3	0.3	1
D3	0.4	1.0	0.0	…	0.0	0.3	0.3	0
D4	0.5	0.3	0.0	…	0.3	1.0	1.0	1
D5	0.5	0.3	0.0	…	0.2	0.3	0.3	1
Uji	0.5	0.3	0.0	…	1,5	0.0	0.3	?

Label kelas pada Duji belum diketahui, sehingga ditentukan melalui perhitungan jarak antara D1 dengan D2 sampai D5.

A. Perhitungan Euclidean Distance

Euclidean Distance digunakan untuk menghitung tingkat kedekatan antara data uji dan data latih berdasarkan selisih kuadrat pada setiap atribut. Semakin kecil nilai jarak yang dihasilkan, maka semakin dekat hubungan antara data uji dan data latih tersebut. Perhitungan dilakukan terhadap seluruh 31 fitur hasil preprocessing, namun untuk ilustrasi berikut ditampilkan perhitungan menggunakan beberapa fitur pada Tabel 4.13. Rumus Euclidean Distance dapat dilihat pada persamaan 2.1 dan didapatkan hasil seperti berikut:

$$d(UjiD1) = \sqrt{\begin{matrix}(0.5-0.4)^2+(0.3-1.0)^2+(0.0-0.0)^2 \\ +(0.1-0.0)^2+(0.4-0.0)^2+(0.0-0.0)^2 \\ +(1.5-0.0)^2+(0.0-0.3)^2+(0.3-0.3)^2\end{matrix}} = 1.8$$

$$d(\text{UjiD2}) = \sqrt{\begin{matrix}(0.5-0.4)^2+(0.3-1.0)^2+(0.0-0.2)^2 \\ +(0.1-0.0)^2+(0.4-0.3)^2+(0.0-0.0)^2 \\ +(1.5-0.7)^2+(0.0-0.3)^2+(0.3-0.3)^2\end{matrix}} = 1.2$$

$$d(\text{UjiD3}) = \sqrt{\begin{matrix}(0.5-0.4)^2+(0.3\ -\ 1.0)^2+(0.0-0.0)^2 \\ +(0.1-0.0)^2+(0.4-0.3)^2+(0.0-0.0)^2 \\ +(1.5-0.0)^2+(0.0\ -0.3)^2+(0.3-0.3)^2\end{matrix}} = 1.7$$

$$d(\text{UjiD4}) = \sqrt{\begin{matrix}(0.5-0.5)^2+(0.3\ -\ 0.3)^2+(0.0-0.0)^2 \\ +(0.1-0.0)^2+(0.4-0.3)^2+(0.0-0.0)^2 \\ +(1.5-0.3)^2+(0.0\ -1.0)^2+(0.3-1.0)^2\end{matrix}} = 1.7$$

$$d(\text{UjiD5}) = \sqrt{\begin{matrix}(0.5-0.5)^2+(0.3\ -\ 0.3)^2+(0.0-0.0)^2 \\ +(0.1-0.0)^2+(0.4-0.3)^2+(0.0-0.0)^2 \\ +(1.5-0.2)^2+(0.0\ -0.3)^2+(0.3-0.3)^2\end{matrix}} = 1.4$$

Hasil perhitungan jarak Euclidean disajikan pada Tabel 4.14.

Tabel 4. 14 Hasil Perhitungan Jarak Euclidean

Data	Jarak Euclidean	Peringkat	Label
D2	1.2	1	1
D5	1.4	2	1
D3	1.7	3	0
D4	1.7	3	1
D1	1.8	4	1

Berdasarkan Tabel 4.14, mayoritas tetangga terdekat memiliki label 1, sehingga data uji D2 diklasifikasikan ke dalam kelas 1.

B. Perhitungan Manhattan Distance

Selain menggunakan Euclidean Distance, penelitian ini juga menggunakan Manhattan Distance sebagai metode pengukuran jarak. Manhattan Distance menghitung jarak sebagai jumlah nilai absolut selisih setiap atribut antara data uji dan data latih. Rumus Manhattan distance dapat dilihat pada persamaan 2.2 dan didapatkan hasil seperti berikut:

$$d(\text{UjiD1}) = \begin{matrix}|\ 0.5-0.4\ |+|\ 0.3-1.0\ |+|\ 0.0-0.0\ | \\ +|\ 0.1-0.0\ |+|\ 0.4-0.0\ |+|\ 0.0-0.0\ | \\ +|\ 1.5-0.0\ |+|\ 0.0-0.3\ |+|\ 0.3-0.3\ |\end{matrix} = 3.1$$

$$d(\text{UjiD2}) = \begin{matrix}|\ 0.5-0.4\ |+|\ 0.3-1.0\ |+|\ 0.0-0.2\ | \\ +|\ 0.1-0.0\ |+|\ 0.4-0.3\ |+|\ 0.0-0.0\ | \\ +|\ 1.5-0.7\ |+|\ 0.0-0.3\ |+|\ 0.3-0.3\ |\end{matrix} = 2.3$$

$$d(\text{UjiD3}) = \begin{matrix}|\ 0.5-0.4\ |+|\ 0.3-1.0\ |+|\ 0.0-0.0\ | \\ +|\ 0.1-0.0\ |+|\ 0.4-0.3\ |+|\ 0.0-0.0\ | \\ +|\ 1.5-0.0\ |+|\ 0.0-0.3\ |+|\ 0.3-0.3\ |\end{matrix} = 2.8$$

$$d(\text{UjiD4}) = \begin{matrix} |\ 0.5-0.5\ |+|\ 0.3-0.3\ |+|\ 0.0-0.0\ | \\ +|\ 0.1-0.0\ |+|\ 0.4-0.3\ |+|\ 0.0-0.0\ | \\ +|\ 1.5-0.3\ |+|\ 0.0-1.0\ |+|\ 0.3-1.0\ | \end{matrix} = 3.1$$

$$d\text{U(jiD5)} = \begin{matrix} |\ 0.5-0.5\ |+|\ 0.3-0.3\ |+|\ 0.0-0.0\ | \\ +|\ 0.1-0.0\ |+|\ 0.4-0.3\ |+|\ 0.0-0.0\ | \\ +|\ 1.5-0.2\ |+|\ 0.0-0.3\ |+|\ 0.3-0.3\ | \end{matrix} = 1.8$$

Hasil perhitungan jarak Manhattan disajikan pada Tabel 4.15.

Tabel 4. 15 Hasil Perhitungan Manhattan Distance

Data	Jarak Manhattan	Rank	Label
D5	1.8	1	1
D2	2.3	2	1
D3	2.8	3	0
D1	3.1	4	1
D4	3.1	4	1

Berdasarkan Tabel 4.15, mayoritas tetangga terdekat juga memiliki label 1, sehingga data uji D5 diklasifikasikan ke dalam kelas 1.

Berdasarkan hasil perhitungan jarak pada Tabel 4.14 dan Tabel 4.15, data uji memiliki jarak paling kecil terhadap D2 dan D5 pada kedua metode pengukuran jarak. Konsistensi hasil ini menunjukkan bahwa keputusan klasifikasi data uji cukup stabil terhadap perubahan rumus pengukuran jarak, sehingga data uji dapat ditetapkan berada pada kelas 1.Pada hasil Euclidean Distance, tiga dari empat tetangga terdekat berada pada kelas 1, sehingga data uji diklasifikasikan ke kelas 1. Hasil Manhattan Distance juga menunjukkan pola yang sama, yaitu mayoritas tetangga terdekat tetap berada pada kelas 1. Konsistensi hasil dari dua metode jarak ini menguatkan bahwa keputusan klasifikasi untuk data uji cukup stabil terhadap perubahan rumus pengukuran jarak.

Hasil ini juga selaras dengan proses pemilihan parameter terbaik pada notebook, karena konfigurasi non-SMOTE terbaik menggunakan K = 5, metric euclidean, dan weight distance, sedangkan konfigurasi SMOTE terbaik menggunakan K = 3, metric manhattan, dan weight distance. Artinya, pemilihan tetangga terdekat dan cara menghitung jarak memang berpengaruh langsung terhadap performa model, sehingga pembahasan tabel sebaiknya tidak hanya menyebut “hasil jarak terkecil”, tetapi juga menjelaskan bahwa label akhir ditentukan oleh mayoritas kelas tetangga terdekat dan kualitas konfigurasi model.

Grid Search CV merupakan metode hyperparameter tuning yang digunakan untuk mencari kombinasi parameter terbaik pada model K-Nearest Neighbor (KNN) dengan menguji seluruh kombinasi parameter yang telah ditentukan, seperti nilai K (n_neighbors), metrik jarak (metric), dan bobot (weights). Setiap kombinasi parameter dievaluasi menggunakan K-Fold Cross Validation, sehingga model dilatih dan divalidasi secara berulang pada setiap fold untuk memperoleh nilai evaluasi yang lebih objektif dan stabil. Berdasarkan hasil evaluasi tersebut, Grid Search CV memilih kombinasi parameter yang memberikan performa terbaik sebagai parameter terbaik (best parameters). Pada penelitian ini, konfigurasi terbaik untuk model Non-SMOTE adalah K = 5, metric euclidean, dan weight distance, sedangkan konfigurasi terbaik untuk model SMOTE adalah K = 3, metric manhattan, dan weight distance. Hasil tersebut menunjukkan bahwa pemilihan jumlah tetangga, metode pengukuran jarak, dan skema pembobotan memiliki pengaruh langsung terhadap kemampuan model dalam mengklasifikasikan tingkat keparahan kecelakaan lalu lintas, sehingga penggunaan Grid Search CV membantu menghasilkan konfigurasi model yang lebih optimal.

6. Evaluasi Model

Evaluasi terhadap model KNN yang telah diimplementasikan dilakukan menggunakan *Confusion Matrix* dan *Classification Report* untuk mengukur nilai *accuracy*, *precision*, *recall*, dan *F1-score* pada setiap skema data (Non-SMOTE dan SMOTE).

4.3 Penerapan Tampilan

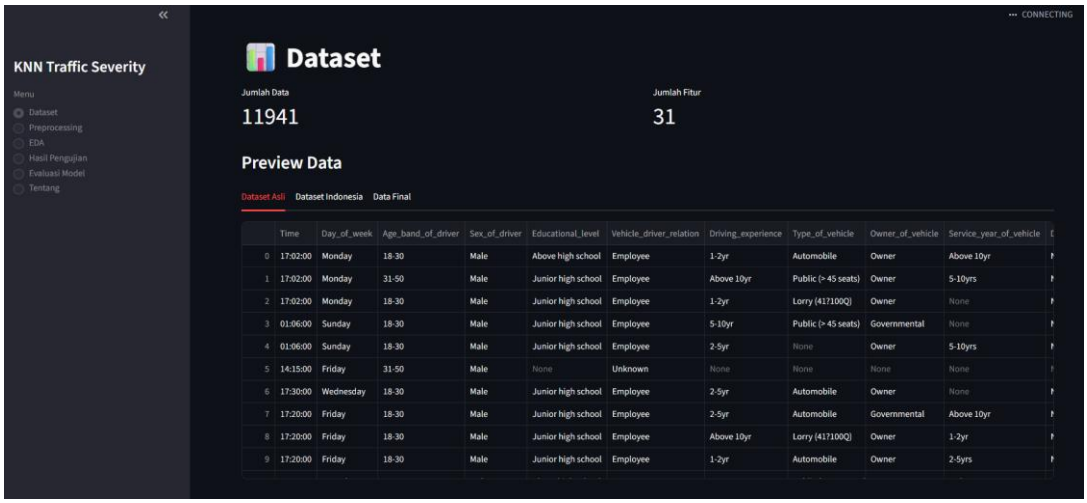
Pada penelitian ini, hasil implementasi sistem ditampilkan dalam bentuk aplikasi berbasis *web* menggunakan *framework* Streamlit. Aplikasi ini dirancang untuk mempermudah proses eksplorasi data, analisis, serta evaluasi model KNN dalam memprediksi tingkat keparahan kecelakaan lalu lintas. Antarmuka aplikasi dibagi menjadi beberapa halaman utama, yaitu halaman Dataset, halaman *Exploratory Data Analysis* (EDA), dan halaman Evaluasi Model.

1. Halaman Dataset

Halaman Dataset merupakan halaman yang digunakan untuk menampilkan informasi umum terkait data yang digunakan dalam penelitian. Pada halaman ini

ditampilkan jumlah keseluruhan data serta jumlah fitur yang digunakan dalam proses pemodelan. Informasi tersebut disajikan dalam bentuk *metric* sehingga memudahkan pengguna dalam memahami karakteristik dasar dataset.

Selain itu, halaman ini juga menampilkan *preview* data dalam bentuk tabel yang berisi beberapa baris pertama dari dataset. Tampilan ini bertujuan untuk memberikan gambaran awal mengenai struktur data, seperti nama atribut, tipe data, serta contoh isi dari masing-masing fitur. Dengan adanya fitur ini, pengguna dapat dengan mudah melakukan verifikasi terhadap data yang digunakan sebelum masuk ke tahap analisis lebih lanjut. Halaman ini dapat dilihat pada Gambar 4.1.



	Time	Day_of_week	Age_band_of_driver	Sex_of_driver	Educational_level	Vehicle_driver_relation	Driving_experience	Type_of_vehicle	Owner_of_vehicle	Service_year_of_vehicle
0	17:02:00	Monday	18-30	Male	Above high school	Employee	1-2yr	Automobile	Owner	Above 10yr
1	17:02:00	Monday	31-50	Male	Junior high school	Employee	Above 10yr	Public (> 45 seats)	Owner	5-10yrs
2	17:02:00	Monday	18-30	Male	Junior high school	Employee	1-2yr	Lorry (41?100Q)	Owner	None
3	01:06:00	Sunday	18-30	Male	Junior high school	Employee	5-10yr	Public (> 45 seats)	Governmental	None
4	01:06:00	Sunday	18-30	Male	Junior high school	Employee	2-5yr	None	Owner	5-10yrs
5	14:15:00	Friday	31-50	Male	None	Unknown	None	None	None	None
6	17:30:00	Wednesday	18-30	Male	Junior high school	Employee	2-5yr	Automobile	Owner	None
7	17:20:00	Friday	18-30	Male	Junior high school	Employee	2-5yr	Automobile	Governmental	Above 10yr
8	17:20:00	Friday	18-30	Male	Junior high school	Employee	Above 10yr	Lorry (41?100Q)	Owner	1-2yr
9	17:20:00	Friday	18-30	Male	Junior high school	Employee	1-2yr	Automobile	Owner	2-5yrs

Gambar 4. 1 Halaman Dataset

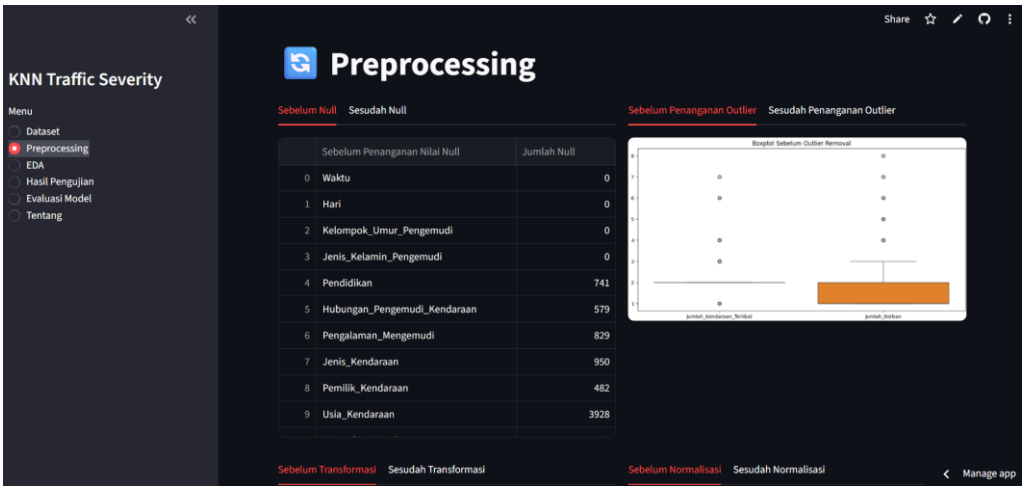
Selain menampilkan informasi umum mengenai dataset, halaman ini juga menyajikan hasil preprocessing data yang meliputi penanganan nilai *missing value*, data duplikat, *outlier*, transformasi data, dan normalisasi. Setiap tahapan ditampilkan dalam bentuk kondisi sebelum dan sesudah preprocessing, sehingga pengguna dapat melihat perubahan data pada setiap proses.

2. Halaman Preprocessing

Halaman Preprocessing merupakan halaman yang digunakan untuk menampilkan tahapan prapemrosesan data yang dilakukan sebelum proses pemodelan menggunakan algoritma K-Nearest Neighbor (KNN). Pada halaman ini ditampilkan hasil dari setiap tahapan preprocessing, yaitu penanganan nilai null,

penanganan outlier, transformasi data, dan normalisasi data. Setiap tahapan disajikan dalam bentuk tab sebelum dan sesudah sehingga pengguna dapat membandingkan kondisi data pada setiap proses preprocessing.

Selain itu, halaman ini juga menampilkan visualisasi dan tabel yang menggambarkan perubahan data setelah dilakukan preprocessing. Pada tahap penanganan nilai null ditampilkan jumlah nilai yang hilang pada setiap atribut, sedangkan pada tahap penanganan outlier ditampilkan boxplot untuk memperlihatkan distribusi data sebelum dan sesudah penghapusan outlier. Tahap transformasi dan normalisasi menampilkan perubahan bentuk data hasil proses encoding dan skala data setelah normalisasi menggunakan MinMaxScaler. Tampilan ini bertujuan untuk memberikan informasi mengenai proses persiapan data sehingga pengguna dapat memverifikasi hasil preprocessing sebelum data digunakan dalam proses pelatihan dan pengujian model. Halaman ini dapat dilihat pada Gambar 4.2.



Gambar 4. 2 Halaman Preprocessing

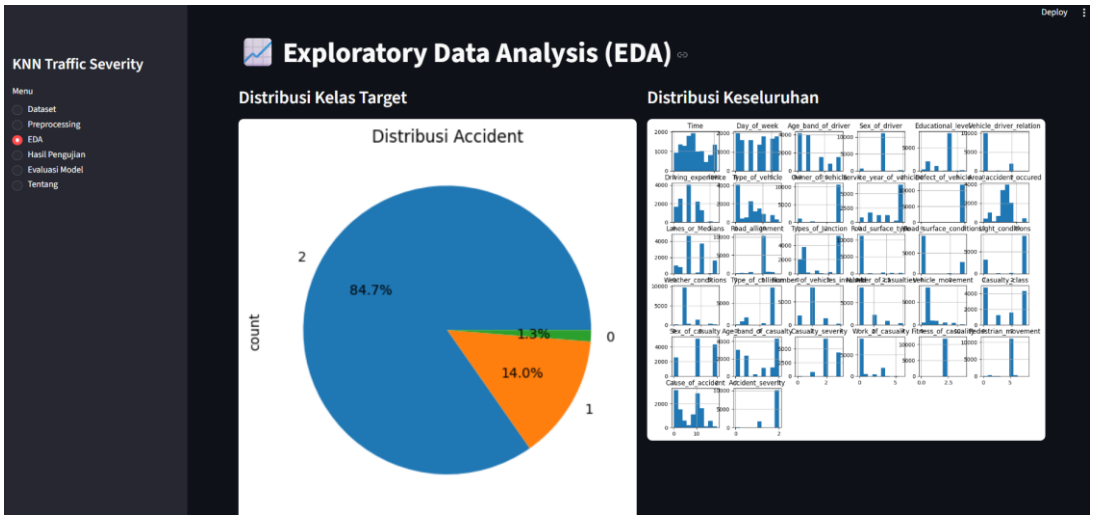
3. Halaman Exploratory Data Analysis (EDA)

Halaman *Exploratory Data Analysis* (EDA) digunakan untuk menampilkan hasil analisis eksploratif terhadap dataset yang bertujuan untuk memahami distribusi dan karakteristik data. Pada halaman ini, visualisasi ditampilkan dalam bentuk gambar yang merepresentasikan beberapa aspek penting dari data.

Visualisasi yang ditampilkan meliputi distribusi kelas target yang menunjukkan proporsi masing-masing kategori tingkat keparahan kecelakaan. Selain itu, ditampilkan

juga visualisasi distribusi usia pengemudi untuk melihat sebaran data berdasarkan usia. Halaman ini juga menyajikan *boxplot* yang digunakan untuk mengidentifikasi penyebaran data serta kemungkinan adanya *outlier* pada fitur tertentu.

Selanjutnya, ditampilkan pula visualisasi korelasi antar fitur yang bertujuan untuk mengetahui hubungan antar variabel dalam dataset. Dengan adanya analisis korelasi, dapat diketahui fitur-fitur yang memiliki hubungan kuat maupun lemah terhadap variabel target. Secara keseluruhan, halaman EDA membantu dalam memberikan pemahaman awal terhadap pola data sebelum dilakukan proses pemodelan. Halaman EDA dapat dilihat pada Gambar 4.3.



Gambar 4. 3 Halaman EDA

Halaman Evaluasi Model tidak hanya menampilkan hasil evaluasi seluruh model, tetapi juga menyajikan hasil terbaik dari skema Non-SMOTE dan SMOTE berdasarkan nilai *Macro F1-Score*. Untuk masing-masing skema ditampilkan parameter terbaik yang diperoleh dari proses *Grid Search Cross Validation*, *Confusion Matrix*, *Classification Report*, serta nilai *Accuracy*, *Precision*, *Recall*, dan *F1-Score*. Penyajian hasil terbaik ini memudahkan pengguna dalam membandingkan performa model sebelum dan sesudah penerapan metode *SMOTE*.

4. Hasil Pengujian

Halaman Hasil Pengujian merupakan halaman yang digunakan untuk menampilkan hasil pengujian model K-Nearest Neighbor (KNN) berdasarkan

berbagai skenario pengujian yang dilakukan dalam penelitian. Pada halaman ini ditampilkan hasil pengujian menggunakan K-Fold Cross Validation, variasi nilai K, metrik jarak (distance metric), dan bobot (weight) baik pada data tanpa SMOTE maupun dengan SMOTE. Setiap skenario disajikan dalam bentuk tab sehingga pengguna dapat membandingkan hasil pengujian pada masing-masing kondisi.

Selain itu, halaman ini juga menampilkan nilai accuracy, precision, recall, dan F1-score untuk setiap kombinasi parameter yang diuji. Hasil terbaik dari masing-masing skenario, seperti nilai K-Fold terbaik, nilai K terbaik, metrik jarak terbaik, dan bobot terbaik, ditampilkan di bawah tabel untuk memudahkan pengguna dalam mengidentifikasi parameter yang menghasilkan performa model paling optimal. Tampilan ini bertujuan untuk memberikan informasi mengenai proses evaluasi parameter KNN sehingga pengguna dapat memahami pengaruh setiap parameter terhadap kinerja model. Halaman ini dapat dilihat pada Gambar 4.4.



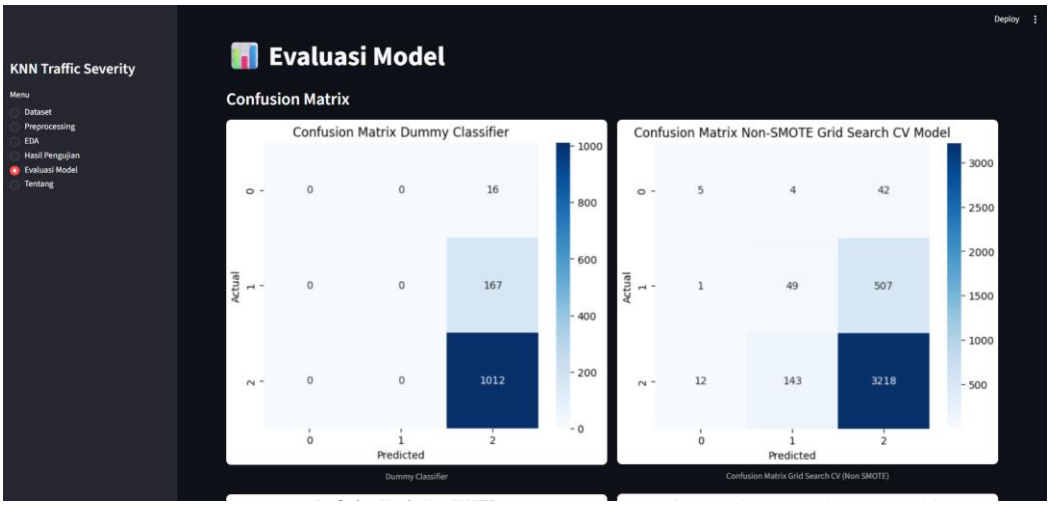
Gambar 4. 4 Halaman Hasil Pengujian

5. Halaman Evaluasi Model

Halaman Evaluasi Model merupakan bagian utama dalam aplikasi yang digunakan untuk menampilkan hasil kinerja model KNN yang telah dibangun. Pada halaman ini disajikan beberapa hasil evaluasi, antara lain *confusion matrix*, *classification report*, serta nilai akurasi dari model.

Confusion matrix ditampilkan untuk membandingkan hasil prediksi model dengan data aktual, baik pada skema tanpa SMOTE maupun dengan SMOTE. Selain itu,

ditampilkan juga hasil evaluasi dari model KNN yang menggunakan pendekatan manual serta hasil optimasi menggunakan *Grid Search Cross Validation*. Visualisasi ini membantu dalam memahami performa model dalam mengklasifikasikan setiap kelas. Dapat dilihat pada Gambar 4.5.



Gambar 4. 5 Halaman Evaluasi Model

Selanjutnya, ditampilkan *classification report* yang berisi nilai *precision* dan *recall* untuk masing-masing kelas. Informasi ini memberikan gambaran yang lebih detail mengenai kemampuan model dalam mengklasifikasikan setiap kategori tingkat keparahan kecelakaan. Selain itu, ditampilkan pula nilai akurasi model dalam bentuk *metric* sehingga memudahkan pengguna dalam melihat performa keseluruhan model secara ringkas. Pada bagian akhir, disajikan perbandingan hasil antara skema SMOTE dan non-SMOTE dalam bentuk visualisasi, yang bertujuan untuk menunjukkan pengaruh teknik penyeimbangan data terhadap performa model.Secara keseluruhan, halaman evaluasi model memberikan informasi yang komprehensif mengenai kinerja model KNN, baik sebelum maupun sesudah dilakukan optimasi dan penyeimbangan data.

4.4 Hasil Pengujian

Pengujian pada penelitian ini dilakukan terhadap model K-Nearest Neighbors (KNN) untuk mengevaluasi performa model dalam proses klasifikasi tingkat keparahan kecelakaan. Pengujian dilakukan menggunakan dua skenario data, yaitu data tanpa

penerapan SMOTE (Non-SMOTE) dan data dengan penerapan SMOTE. Selain itu, penelitian ini juga menggunakan Dummy Baseline sebagai model pembanding awal untuk mengetahui performa dasar klasifikasi sebelum dilakukan proses tuning parameter pada algoritma KNN.

Pada penelitian ini, seluruh proses preprocessing, normalisasi data, standarisasi data, serta penerapan SMOTE dilakukan menggunakan Pipeline. Penggunaan pipeline bertujuan untuk mencegah terjadinya data leakage karena seluruh proses transformasi data dilakukan hanya pada data latih di setiap fold cross validation. Dengan demikian, hasil evaluasi model menjadi lebih valid dan representatif terhadap performa model sebenarnya.

Tahapan pengujian dilakukan secara bertahap dimulai dari pencarian nilai K-Fold terbaik menggunakan variasi K-Fold 3, 5, dan 10. Selanjutnya dilakukan pengujian nilai K tetangga terbaik menggunakan variasi nilai K sebesar 3, 5, 7, dan 9 berdasarkan hasil K-Fold terbaik yang telah diperoleh sebelumnya. Setelah itu dilakukan pengujian metric distance terbaik menggunakan metric Euclidean, Manhattan, dan Cosine. Tahap terakhir pada proses manual tuning adalah pengujian bobot terbaik menggunakan bobot uniform dan distance. Seluruh proses pengujian tersebut dilakukan baik pada skenario Non-SMOTE maupun SMOTE.

Selain pengujian parameter secara manual, penelitian ini juga melakukan optimasi parameter menggunakan Grid Search Cross Validation (GridSearchCV). Metode ini digunakan untuk mencari kombinasi parameter terbaik secara otomatis dengan menguji seluruh kombinasi parameter yang telah ditentukan. Hasil dari GridSearchCV kemudian dibandingkan dengan hasil tuning manual untuk mengetahui metode optimasi yang memberikan performa terbaik. Hasil Pengujian dapat dilihat sebagai berikut:

1. Pengujian Kfold Terbaik

Pengujian K-Fold dilakukan untuk menentukan jumlah fold terbaik dalam proses validasi model K-Nearest Neighbors (KNN). Pengujian dilakukan menggunakan variasi K-Fold 3, 5, dan 10 dengan mengukur nilai accuracy, precision, recall, dan F1-score pada setiap fold. Seluruh proses pengujian dilakukan menggunakan pipeline agar tahapan preprocessing dan klasifikasi berjalan secara terintegrasi sehingga dapat mengurangi terjadinya data leakage selama proses validasi model. Selain itu, pengujian dilakukan

pada dua skenario data, yaitu data tanpa penerapan SMOTE (Non-SMOTE) dan data dengan penerapan SMOTE.

Penentuan K-Fold terbaik dilakukan berdasarkan nilai rata-rata antara accuracy dan F1-score. Penggunaan rata-rata kedua metrik tersebut bertujuan agar pemilihan parameter tidak hanya mempertimbangkan tingkat ketepatan prediksi secara keseluruhan, tetapi juga mempertimbangkan kemampuan model dalam mengenali distribusi kelas secara lebih seimbang. Dengan demikian, parameter yang dipilih diharapkan mampu menghasilkan performa model yang lebih optimal dan stabil pada proses klasifikasi. Pengujian K-fold dapat diuraikan sebagai berikut :

1. Pengujian Kfold terbaik Non SMOTE

Berdasarkan hasil pengujian pada skenario Non-SMOTE, diperoleh bahwa penggunaan K-Fold = 3 menghasilkan performa model yang lebih baik dibandingkan K-Fold lainnya. Hal ini ditunjukkan dengan nilai accuracy sebesar 81,116%, precision sebesar 38,779%, recall sebesar 36,429%, dan F1-score sebesar 36,437%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 58,776%, yang merupakan nilai tertinggi dibandingkan variasi K-Fold lainnya. Nilai evaluasi yang dihasilkan juga cenderung lebih stabil sehingga K-Fold = 3 dipilih sebagai parameter terbaik untuk tahap pengujian berikutnya pada skenario Non-SMOTE. Hal ini dapat dilihat pada Tabel 4.16.

Tabel 4. 16 Hasil Pengujian Kfold KNN Non SMOTE

Accuracy	***Precision***	***Recall***	***F1-Score***	***Rata-rata Accuracy dan F1-Score***	***K-Fold***
81,116%	38,779%	36,429%	36,437%	58,776%	3
81,367%	38,504%	35,900%	35,881%	58,624%	5
81,191%	38,894%	36,036%	36,130%	58,661%	10

2. Pengujian Kfold terbaik SMOTE

Berdasarkan hasil pengujian pada skenario SMOTE, diperoleh bahwa penggunaan K-Fold = 5 menghasilkan performa model terbaik dibandingkan K-Fold lainnya. Hal ini ditunjukkan dengan nilai accuracy sebesar 56,419%, precision sebesar 36,624%, recall sebesar 45,126%, dan F1-score sebesar 34,864%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 45,641%, yang merupakan nilai tertinggi dibandingkan variasi K-Fold lainnya. Hasil ini menunjukkan bahwa

penggunaan K-Fold = 5 mampu memberikan keseimbangan performa yang lebih baik pada data yang telah melalui proses SMOTE, sehingga dipilih sebagai parameter terbaik untuk tahap pengujian selanjutnya pada skenario SMOTE. Hal ini dapat dilihat pada Tabel 4.17.

Tabel 4. 17 Hasil Pengujian Kfold KNN SMOTE

Accuracy	*Precision*	*Recall*	*F1-Score*	*Rata-rata Accuracy dan F1-Score*	*K-Fold*
55,456%	36,235%	44,826%	34,380%	44,918%	3
56,419%	36,624%	45,126%	34,864%	45,641%	5
56,260%	36,490%	44,638%	34,732%	45,496%	10

2. Pengujian K Tetangga terbaik

Pengujian K tetangga dilakukan untuk menentukan jumlah tetangga terdekat (K) terbaik pada model K-Nearest Neighbors (KNN). Pengujian ini dilakukan menggunakan parameter K-Fold terbaik yang telah diperoleh sebelumnya, yaitu K-Fold = 3 pada skenario Non-SMOTE dan K-Fold = 5 pada skenario SMOTE. Variasi nilai K yang digunakan pada penelitian ini meliputi K = 3, 5, 7, dan 9. Evaluasi model dilakukan menggunakan nilai accuracy, precision, recall, dan F1-score. Selanjutnya, penentuan nilai K terbaik dilakukan berdasarkan rata-rata antara accuracy dan F1-score agar parameter yang dipilih tidak hanya memiliki tingkat akurasi yang tinggi, tetapi juga mampu memberikan keseimbangan performa klasifikasi pada setiap kelas data. Hal ini dapat diuraikan sebagai berikut:

1. Pengujian K tetangga terbaik Non SMOTE

Berdasarkan hasil pengujian pada skenario Non-SMOTE menggunakan K-Fold terbaik yaitu 3, diperoleh bahwa nilai K = 5 menghasilkan performa model terbaik dibandingkan variasi nilai K lainnya. Hal ini ditunjukkan dengan nilai accuracy sebesar 83,251%, precision sebesar 46,373%, recall sebesar 34,979%, dan F1-score sebesar 34,575%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 58,913%, yang merupakan nilai tertinggi dibandingkan nilai K lainnya. Hasil tersebut menunjukkan bahwa penggunaan K = 5 mampu memberikan keseimbangan performa yang lebih baik dibandingkan nilai K lainnya pada skenario Non-SMOTE, sehingga dipilih sebagai parameter terbaik untuk tahap pengujian selanjutnya. Hal ini dapat dilihat pada Tabel 4.18.

Tabel 4. 18 Hasil Pengujian K tetangga KNN Non SMOTE

Accuracy	*Precision*	*Recall*	*F1-Score*	*Rata-rata Accuracy dan F1-Score*	*K-Fold*
81,116%	38,779%	36,429%	36,437%	58,776%	3
83,251%	46,373%	34,979%	34,575%	58,913%	5
84,088%	47,882%	33,930%	32,262%	58,175%	7
84,440%	36,041%	33,488%	31,131%	57,786%	9

2. Pengujian K tetangga terbaik SMOTE

Berdasarkan hasil pengujian pada skenario SMOTE menggunakan K-Fold terbaik yaitu 5, diperoleh bahwa nilai K = 3 menghasilkan performa model terbaik dibandingkan variasi nilai K lainnya. Hal ini ditunjukkan dengan nilai accuracy sebesar 56,419%, precision sebesar 36,624%, recall sebesar 45,126%, dan F1-score sebesar 34,864%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 45,641%, yang merupakan nilai tertinggi dibandingkan nilai K lainnya. Hasil tersebut menunjukkan bahwa penggunaan nilai K yang lebih kecil memberikan performa yang lebih baik pada data hasil SMOTE karena model menjadi lebih sensitif dalam mengenali pola lokal antar data yang telah diseimbangkan. Oleh karena itu, nilai K = 3 dipilih sebagai parameter terbaik pada skenario SMOTE untuk tahap pengujian berikutnya. Hal ini dapat dilihat pada Tabel 4.19.

Tabel 4. 19 Hasil Pengujian K Tetangga KNN SMOTE

Accuracy	*Precision*	*Recall*	*F1-Score*	*Rata-rata Accuracy dan F1-Score*	*K-Fold*
56,419%	36,624%	45,126%	34,864%	45,641%	3
50,800%	36,383%	45,886%	32,844%	41,822%	5
46,529%	36,102%	45,990%	31,086%	38,807%	7
43,430%	35,941%	46,069%	29,722%	36,576%	9

3. Pengujian Metric Terbaik

Pengujian metric dilakukan untuk menentukan jenis perhitungan jarak (distance metric) terbaik pada model K-Nearest Neighbors (KNN). Pengujian ini dilakukan menggunakan parameter terbaik yang telah diperoleh sebelumnya, yaitu K-Fold terbaik dan K tetangga terbaik pada masing-masing skenario data. Pada skenario Non-SMOTE

digunakan K-Fold = 3 dan K = 5, sedangkan pada skenario SMOTE digunakan K-Fold = 5 dan K = 3. Metric yang diuji pada penelitian ini meliputi Euclidean dan Manhattan. Evaluasi dilakukan menggunakan nilai accuracy, precision, recall, dan F1-score. Selanjutnya, pemilihan metric terbaik dilakukan berdasarkan rata-rata antara accuracy dan F1-score untuk memperoleh metric yang mampu memberikan performa klasifikasi yang lebih optimal dan seimbang. Pengujian Metrik dapat diuraikan sebagai berikut:

1. Pengujian Metrik terbaik Non SMOTE

Berdasarkan hasil pengujian pada skenario Non-SMOTE menggunakan K-Fold terbaik yaitu 3 dan K tetangga terbaik yaitu 5, diperoleh bahwa metric Euclidean menghasilkan performa model terbaik dibandingkan metric Manhattan. Hal ini ditunjukkan dengan nilai accuracy sebesar 83,251%, precision sebesar 46,373%, recall sebesar 34,979%, dan F1-score sebesar 34,575%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 58,913%, yang merupakan nilai tertinggi dibandingkan metric lainnya. Hasil tersebut menunjukkan bahwa metric Euclidean lebih mampu merepresentasikan kedekatan antar data pada skenario Non-SMOTE sehingga dipilih sebagai metric terbaik untuk tahap pengujian selanjutnya. Hal ini dapat dilihat pada Tabel 4.20.

Tabel 4. 20 Hasil Pengujian Metrik KNN Non SMOTE

Accuracy	***Precision***	***Recall***	***F1-Score***	***Rata-rata Accuracy dan F1-Score***	***Metric***
83,251%	46,373%	34,979%	34,575%	58,913%	Euclidean
83,268%	41,936%	34,639%	33,927%	58,597%	Manhattan

2. Pengujian Metrik terbaik SMOTE

Berdasarkan hasil pengujian pada skenario SMOTE menggunakan K-Fold terbaik yaitu 5 dan K tetangga terbaik yaitu 3, diperoleh bahwa metric Manhattan menghasilkan performa model terbaik dibandingkan metric Euclidean. Hal ini ditunjukkan dengan nilai accuracy sebesar 68,663%, precision sebesar 38,199%, recall sebesar 42,763%, dan F1-score sebesar 39,036%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 53,850%, yang merupakan nilai tertinggi dibandingkan metric lainnya. Hasil tersebut menunjukkan bahwa metric Manhattan lebih sesuai digunakan

pada data hasil SMOTE karena mampu memberikan perhitungan jarak yang lebih optimal pada distribusi data yang telah diseimbangkan. Hal ini dapat dilihat pada Tabel 4.21.

Tabel 4. 21 Hasil Pengujian Metric Jarak KNN SMOTE

Accuracy	*Precision*	*Recall*	*F1-Score*	*Rata-rata Accuracy dan F1-Score*	*Metric*
56,419%	36,624%	45,126%	34,864%	45,641%	Euclidean
68,663%	38,199%	42,763%	39,036%	53,850%	Manhattan

4. Pengujian Bobot Terbaik

Pengujian bobot dilakukan untuk menentukan jenis pembobotan terbaik pada model K-Nearest Neighbors (KNN). Pengujian ini dilakukan menggunakan parameter terbaik yang telah diperoleh sebelumnya, yaitu K-Fold terbaik, K tetangga terbaik, dan metric terbaik pada masing-masing skenario data. Pada skenario Non-SMOTE digunakan K-Fold = 3, K = 5, dan metric Euclidean, sedangkan pada skenario SMOTE digunakan K-Fold = 5, K = 3, dan metric Manhattan. Jenis bobot yang diuji pada penelitian ini meliputi uniform dan distance. Evaluasi dilakukan menggunakan nilai accuracy, precision, recall, dan F1-score. Selanjutnya, penentuan bobot terbaik dilakukan berdasarkan rata-rata antara accuracy dan F1-score agar diperoleh parameter yang mampu memberikan performa klasifikasi yang lebih optimal dan stabil. Pengujian Bobot dapat diuraikan sebagai berikut :

1. Pengujian Bobot terbaik Non SMOTE

Berdasarkan hasil pengujian pada skenario Non-SMOTE menggunakan K-Fold terbaik yaitu 3, K tetangga terbaik yaitu 5, dan metric terbaik yaitu Euclidean, diperoleh bahwa bobot uniform menghasilkan performa model terbaik dibandingkan bobot distance. Hal ini ditunjukkan dengan nilai accuracy sebesar 83,251%, precision sebesar 46,373%, recall sebesar 34,979%, dan F1-score sebesar 34,575%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 58,913%, yang merupakan nilai tertinggi dibandingkan bobot lainnya. Hasil tersebut menunjukkan bahwa penggunaan bobot uniform lebih sesuai pada skenario Non-SMOTE karena seluruh tetangga memberikan kontribusi yang relatif seimbang dalam proses klasifikasi. Hal ini dapat dilihat pada Tabel 4.22.

Tabel 4. 22 Hasil Pengujian Bobot KNN Non SMOTE

Accuracy	*Precision*	*Recall*	*F1-Score*	*Rata-rata Accuracy dan F1-Score*	*Weight*
83,251%	46,373%	34,979%	34,575%	58,913%	Uniform
83,377%	47,791%	34,797%	34,223%	58,800%	Distance

2. Pengujian Bobot terbaik SMOTE

Berdasarkan hasil pengujian pada skenario SMOTE menggunakan K-Fold terbaik yaitu 5, K tetangga terbaik yaitu 3, dan metric terbaik yaitu Manhattan, diperoleh bahwa bobot distance menghasilkan performa model terbaik dibandingkan bobot uniform. Hal ini ditunjukkan dengan nilai accuracy sebesar 68,805%, precision sebesar 38,161%, recall sebesar 42,371%, dan F1-score sebesar 38,953%. Selain itu, hasil rata-rata antara accuracy dan F1-score memperoleh nilai sebesar 53,879%, yang merupakan nilai tertinggi dibandingkan bobot lainnya. Hasil tersebut menunjukkan bahwa penggunaan bobot distance lebih optimal pada data hasil SMOTE karena tetangga yang memiliki jarak lebih dekat memberikan pengaruh yang lebih besar dalam proses klasifikasi, sehingga model mampu mengenali pola data secara lebih baik. Hal ini dapat dilihat pada Tabel 4.23.

Tabel 4. 23 Hasil Pengujian Bobot KNN SMOTE

Accuracy	*Precision*	*Recall*	*F1-Score*	*Rata-rata Accuracy dan F1-Score*	*Weight*
68,663%	38,199%	42,763%	39,036%	53,850%	Uniform
68,805%	38,161%	42,371%	38,953%	53,879%	Distance

5. Pengujian Kombinasi GridSearch CV

Pengujian GridSearch Cross Validation (GridSearch CV) dilakukan untuk memperoleh kombinasi parameter terbaik pada model K-Nearest Neighbors (KNN) secara otomatis. Berbeda dengan pengujian sebelumnya yang dilakukan secara bertahap, pada tahap ini seluruh parameter diuji secara bersamaan menggunakan pipeline, sehingga proses preprocessing dan pelatihan model dilakukan secara terintegrasi. Parameter yang diuji meliputi jumlah tetangga (K), jenis metric, dan jenis weight. Pengujian dilakukan pada dua skenario data, yaitu data Non-SMOTE dan data SMOTE. Hasil pengujian GridSearch CV digunakan untuk menentukan konfigurasi parameter terbaik berdasarkan performa model yang diperoleh.

1. Gridsearch CV Non SMOTE

Berdasarkan hasil pengujian GridSearch CV pada skenario Non-SMOTE, diperoleh kombinasi parameter terbaik yaitu menggunakan metric *manhattan*, jumlah tetangga (*n_neighbors*) sebanyak 3, serta weight *distance*. Selain itu, model juga menggunakan tahapan preprocessing berupa *SimpleImputer* dengan strategi *most_frequent* dan *MinMaxScaler* dalam bentuk pipeline. Adapun model akhir yang diperoleh dapat dilihat sebagai berikut:

```
Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('scaler', MinMaxScaler()),
    ('knn',
     KNeighborsClassifier(
         metric='manhattan',
         n_neighbors=3,
         weights='distance'
     ))
])
```

2. Gridsearch CV SMOTE

Berdasarkan hasil pengujian GridSearch CV pada skenario SMOTE, diperoleh kombinasi parameter terbaik yaitu menggunakan metric *manhattan* dengan jumlah tetangga (*n_neighbors*) sebanyak 3. Pada model ini juga diterapkan metode SMOTE di dalam pipeline untuk menangani ketidakseimbangan data sebelum proses klasifikasi dilakukan. Selain itu, model menggunakan *SimpleImputer* dengan strategi *most_frequent* dan *MinMaxScaler* sebagai tahapan preprocessing. Adapun model akhir yang diperoleh dapat dilihat sebagai berikut:

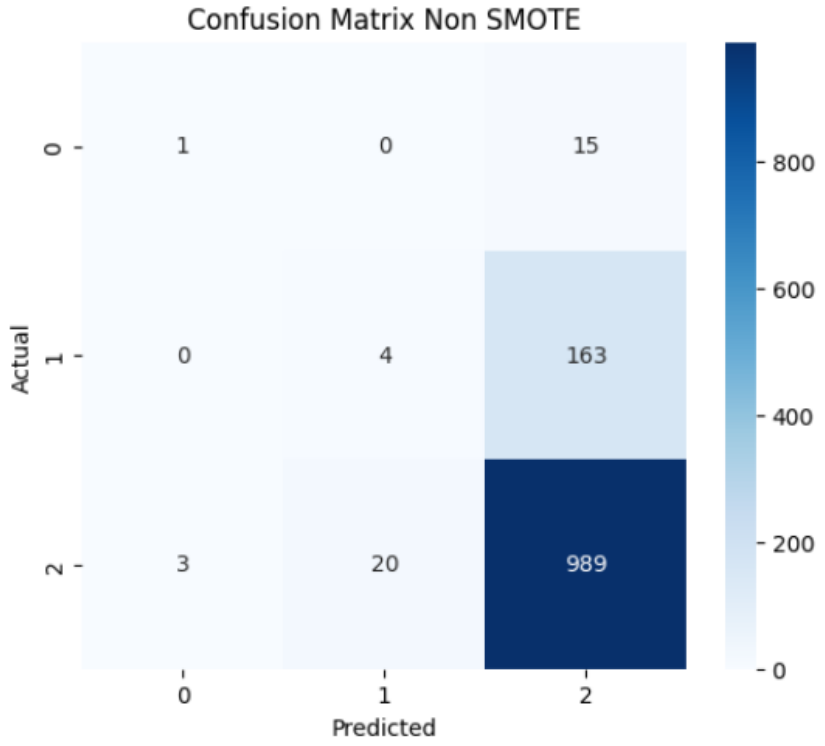
```
Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('scaler', MinMaxScaler()),
    ('smote', SMOTE(random_state=42)),
    ('knn',
     KNeighborsClassifier(
         metric='manhattan',
         n_neighbors=3
     ))
])
```

6. Pengujian Model

Pengujian menggunakan confusion matrix pada model KNN dilakukan dengan membandingkan dua skema data, yaitu skema tanpa implementasi SMOTE dan skema dengan implementasi SMOTE. Pengujian ini bertujuan untuk menganalisis perbedaan kinerja model KNN dalam melakukan klasifikasi pada kedua skenario data tersebut, khususnya dalam mendeteksi distribusi prediksi terhadap data aktual. Melalui confusion matrix, dapat diperoleh informasi yang lebih rinci mengenai performa model, seperti nilai accuracy, precision, dan recall pada masing-masing kelas. Adapun hasil pengujian confusion matrix pada model KNN untuk setiap skema data dapat diuraikan sebagai berikut:

1. Model KNN Tuning (Non SMOTE)

Pengujian dilakukan pada model KNN menggunakan skema data tanpa SMOTE dengan parameter terbaik yang telah diperoleh sebelumnya, yaitu K-Fold = 3, K tetangga = 5, metric = Euclidean, dan weight = uniform. Selanjutnya model dievaluasi menggunakan confusion matrix untuk mengetahui kemampuan model dalam melakukan klasifikasi pada setiap kelas data. Hasil pengujian model KNN tanpa SMOTE dapat dilihat pada Gambar 4.4.



Gambar 4. 6 Confusion Matrix KNN (Non SMOTE)

Berdasarkan Gambar 4.6 tersebut diperoleh hasil sebagai berikut:

1. Kelas prediksi dengan label 0 menghasilkan klasifikasi sesuai dengan kelas aktual sebanyak 1 data, serta klasifikasi yang tidak sesuai sebanyak 3 data dari kelas 2, dengan total prediksi label 0 sebanyak 4 data.
2. Kelas prediksi dengan label 1 menghasilkan klasifikasi sesuai dengan kelas aktual sebanyak 4 data, serta klasifikasi yang tidak sesuai sebanyak 20 data dari kelas 2, dengan total prediksi label 1 sebanyak 24 data.
3. Kelas prediksi dengan label 2 menghasilkan klasifikasi sesuai dengan kelas aktual sebanyak 989 data, serta klasifikasi yang tidak sesuai sebanyak 178 data (terdiri dari 15 data dari kelas 0 dan 163 data dari kelas 1), dengan total prediksi label 2 sebanyak 1.167 data.

Berdasarkan hasil tersebut, jumlah keseluruhan data adalah 1.195 data, dengan jumlah prediksi yang benar (diagonal confusion matrix) sebesar 994 data.

Nilai accuracy, precision, recall, dan F1-Score dapat dilihat pada Tabel 4.24 berikut:

Tabel 4. 24 Classification Report KNN Manual Non SMOTE

Kelas	Precision	Recall	F1-Score	Support
0	25,000%	6,250%	10,000%	16
1	16,667%	2,395%	4,188%	167
2	84,747%	97,727%	90,776%	1012
Accuracy	83,180%			
Macro Avg	42,138%	35,457%	34,988%	1195
Weighted Avg	74,433%	83,180%	77,594%	1195

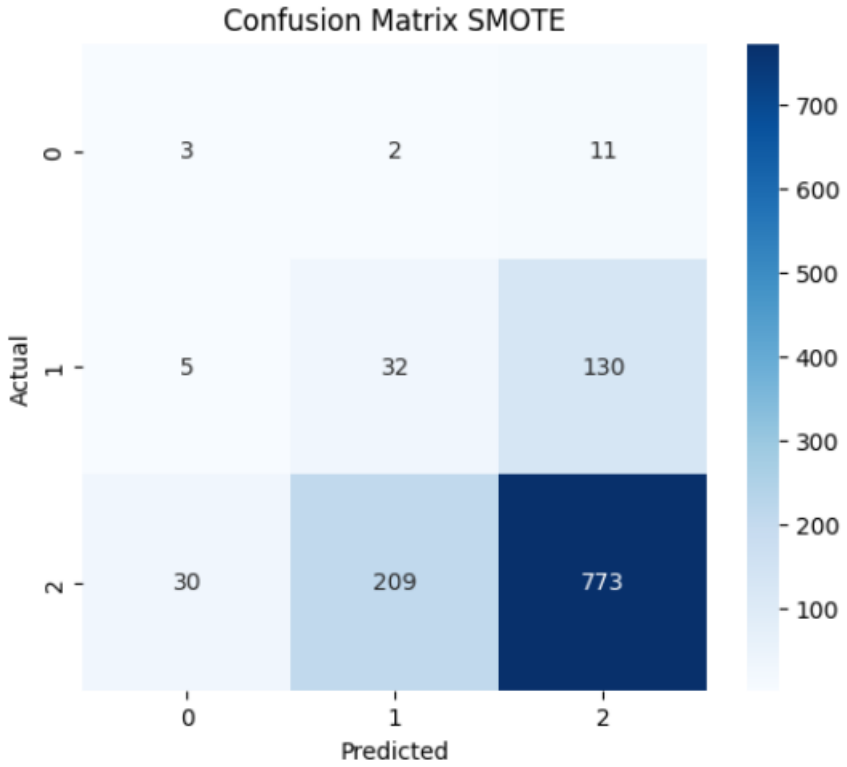
Berdasarkan hasil classification report pada model KNN Non-SMOTE, diperoleh nilai accuracy sebesar 83,180%. Nilai accuracy yang tinggi menunjukkan bahwa model mampu melakukan klasifikasi dengan baik secara keseluruhan. Namun, apabila ditinjau menggunakan nilai macro average, performa model masih tergolong rendah dengan precision sebesar 42,138%, recall sebesar 35,457%, dan F1-Score sebesar 34,988%.

Pada kelas 0, model menghasilkan precision sebesar 25,000%, recall sebesar 6,250%, dan F1-Score sebesar 10,000%. Nilai recall yang sangat rendah menunjukkan bahwa sebagian besar data pada kelas 0 gagal dikenali oleh model. Hal serupa juga terjadi pada kelas 1 dengan precision sebesar 16,667%, recall sebesar 2,395%, dan F1-Score sebesar 4,188%, yang menunjukkan bahwa model hampir tidak mampu mendeteksi kelas tersebut dengan baik. Sebaliknya, performa model pada kelas 2 sangat tinggi dengan

precision sebesar 84,747%, recall sebesar 97,727%, dan F1-Score sebesar 90,776%. Hal ini menunjukkan bahwa model sangat dominan dalam memprediksi kelas mayoritas dibandingkan kelas minoritas.

2. Model KNN Tuning (SMOTE)

Pengujian dilakukan pada model KNN menggunakan skema data dengan implementasi SMOTE menggunakan parameter terbaik yang telah diperoleh sebelumnya, yaitu K-Fold = 5, K tetangga = 3, metric = Manhattan, dan weight = distance. Selanjutnya model dievaluasi menggunakan confusion matrix untuk mengetahui kemampuan model dalam melakukan klasifikasi pada setiap kelas data. Hasil pengujian model KNN dengan skema data kecelakaan menggunakan SMOTE dapat dilihat pada Gambar 4.5.



Gambar 4. 7 Confusion Matrix KNN (SMOTE)

Berdasarkan Gambar 4.7 tersebut diperoleh hasil sebagai berikut:

1. Kelas prediksi dengan label 0 menghasilkan klasifikasi sesuai dengan kelas aktual sebanyak 3 data, serta klasifikasi yang tidak sesuai sebanyak 35 data (terdiri dari 5 data dari kelas 1 dan 30 data dari kelas 2), dengan total prediksi label 0 sebanyak 38 data.

2. Kelas prediksi dengan label 1 menghasilkan klasifikasi sesuai dengan kelas aktual sebanyak 32 data, serta klasifikasi yang tidak sesuai sebanyak 211 data (terdiri dari 2 data dari kelas 0 dan 209 data dari kelas 2), dengan total prediksi label 1 sebanyak 243 data.
3. Kelas prediksi dengan label 2 menghasilkan klasifikasi sesuai dengan kelas aktual sebanyak 773 data, serta klasifikasi yang tidak sesuai sebanyak 141 data (terdiri dari 11 data dari kelas 0 dan 130 data dari kelas 1), dengan total prediksi label 2 sebanyak 914 data.

Berdasarkan hasil tersebut, jumlah keseluruhan data adalah 1.195 data, dengan jumlah prediksi yang benar (diagonal confusion matrix) sebesar 808 data.

Nilai evaluasi model berdasarkan classification report dapat dilihat pada Tabel 4.25.

Tabel 4. 25 Classification Report KNN Manual SMOTE

Kelas	Precision	Recall	F1-Score	Support
0	7,895%	18,750%	11,111%	16
1	13,169%	19,162%	15,610%	167
2	84,573%	76,383%	80,270%	1012
Accuracy	67,615%			
Macro Avg	35,212%	38,098%	35,664%	1195
Weighted Avg	73,568%	67,615%	70,308%	1195

Berdasarkan hasil classification report pada model KNN dengan penerapan SMOTE, diperoleh nilai accuracy sebesar 67,615%. Nilai accuracy mengalami penurunan dibandingkan model Non-SMOTE, namun performa model pada kelas minoritas mengalami peningkatan. Hal ini terlihat dari nilai macro average yang meningkat, dengan precision sebesar 35,212%, recall sebesar 38,098%, dan F1-Score sebesar 35,664%.

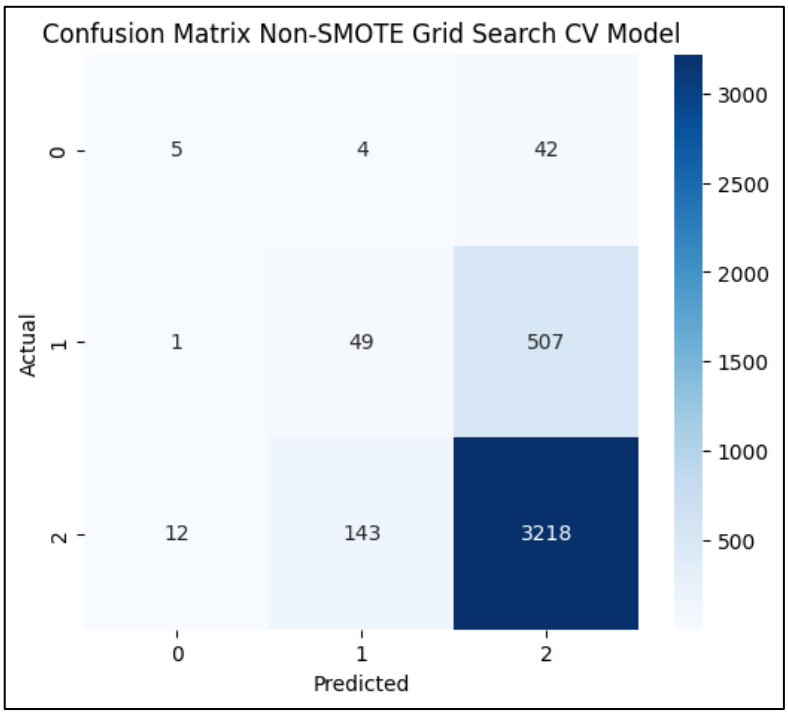
Pada kelas 0, model menghasilkan precision sebesar 7,895%, recall sebesar 18,750%, dan F1-Score sebesar 11,111%. Nilai recall pada kelas 0 meningkat dibandingkan model Non-SMOTE, yang menunjukkan bahwa model mulai mampu mengenali lebih banyak data pada kelas minoritas tersebut. Hal serupa juga terjadi pada kelas 1 dengan precision sebesar 13,169%, recall sebesar 19,162%, dan F1-Score sebesar 15,610%. Peningkatan recall pada kelas 1 menunjukkan bahwa penerapan SMOTE membantu model dalam mengurangi bias terhadap kelas mayoritas.

Sementara itu, pada kelas 2 diperoleh precision sebesar 84,573%, recall sebesar 76,383%, dan F1-Score sebesar 80,270%. Nilai recall pada kelas mayoritas mengalami

penurunan dibandingkan model Non-SMOTE karena model tidak lagi terlalu fokus pada kelas mayoritas dan mulai memberikan perhatian terhadap kelas minoritas.

3. Model KNN Grid Search CV (Non SMOTE)

Pengujian dilakukan pada model KNN menggunakan skema data tanpa SMOTE dengan parameter terbaik hasil Grid Search CV, yaitu metric = Manhattan, K tetangga = 3, dan weight = distance. Selanjutnya model dievaluasi menggunakan confusion matrix untuk mengetahui kemampuan model dalam melakukan klasifikasi pada setiap kelas data. Hasil pengujian model KNN dengan skema data kecelakaan tanpa SMOTE dapat dilihat pada Gambar 4.6.



Gambar 4. 8 Confusion Matrix KNN GridSearch CV (Non SMOTE)

Berdasarkan Gambar 4.8 tersebut diperoleh hasil sebagai berikut:

1) Kelas prediksi dengan label 0 menghasilkan klasifikasi sesuai sebanyak 5 data, serta klasifikasi yang tidak sesuai sebanyak 13 data (terdiri dari 1 data kelas 1 dan 12 data kelas 2), dengan total prediksi label 0 sebanyak 18 data.
2) Kelas prediksi dengan label 1 menghasilkan klasifikasi sesuai sebanyak 49 data, serta klasifikasi yang tidak sesuai sebanyak 147 data (terdiri dari 4 data kelas 0 dan 143 data kelas 2), dengan total prediksi label 1 sebanyak 196 data.

3) Kelas prediksi dengan label 2 menghasilkan klasifikasi sesuai sebanyak 3218 data, serta klasifikasi yang tidak sesuai sebanyak 549 data (terdiri dari 42 data kelas 0 dan 507 data kelas 1), dengan total prediksi label 2 sebanyak 3767 data.

Berdasarkan hasil tersebut, jumlah keseluruhan data pengujian adalah 3981 data dengan jumlah prediksi benar sebesar 3272 data. Nilai evaluasi model berdasarkan classification report dapat dilihat pada Tabel 4.26.

Tabel 4. 26 Classification Report KNN Grid Search CV (Non SMOTE)

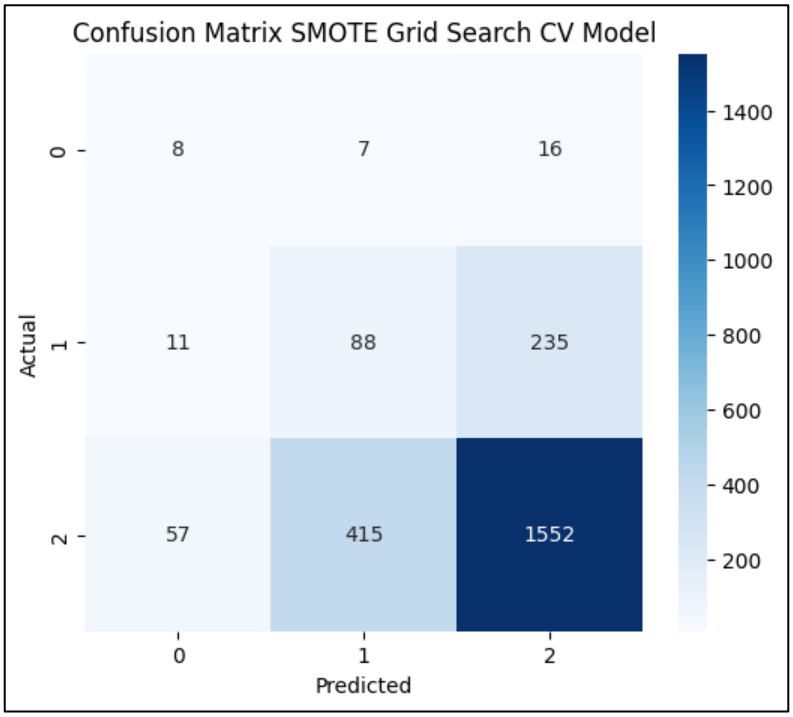
Kelas	Precision	Recall	F1-Score	Support
0	27,778%	9,804%	14,493%	51
1	25,000%	8,797%	13,015%	557
2	85,426%	95,405%	90,140%	3373
Accuracy	82,190%			
Macro Avg	46,068%	38,002%	39,216%	3981
Weighted Avg	76,233%	82,190%	78,380%	3981

Berdasarkan hasil classification report pada model Grid Search CV Non-SMOTE, diperoleh nilai accuracy sebesar 82,190%. Nilai tersebut menunjukkan bahwa model memiliki performa yang cukup baik secara keseluruhan. Namun, apabila ditinjau berdasarkan nilai macro average, performa model masih tergolong rendah dengan precision sebesar 46,068%, recall sebesar 38,002%, dan F1-Score sebesar 39,216%.

Pada kelas 0, model menghasilkan precision sebesar 27,778%, recall sebesar 9,804%, dan F1-Score sebesar 14,493%. Nilai recall yang rendah menunjukkan bahwa sebagian besar data pada kelas minoritas masih belum dapat dikenali dengan baik oleh model. Pada kelas 1, model memperoleh precision sebesar 25,000%, recall sebesar 8,797%, dan F1-Score sebesar 13,015%, yang menunjukkan bahwa kemampuan model dalam mendeteksi kelas 1 juga masih rendah. Sebaliknya, performa model pada kelas 2 sangat tinggi dengan precision sebesar 85,426%, recall sebesar 95,405%, dan F1-Score sebesar 90,140%. Nilai tersebut menunjukkan bahwa model sangat baik dalam mengenali kelas mayoritas dibandingkan kelas minoritas.

4. Model KNN Grid Search CV (SMOTE)

Pengujian dilakukan pada model KNN menggunakan skema data dengan implementasi SMOTE menggunakan parameter terbaik hasil Grid Search CV, yaitu metric = Manhattan dan K tetangga = 3. Selanjutnya model dievaluasi menggunakan confusion matrix untuk mengetahui kemampuan model dalam melakukan klasifikasi pada setiap kelas data. Hasil pengujian model KNN dengan skema data kecelakaan menggunakan SMOTE dapat dilihat pada Gambar 4.7.



Gambar 4. 7 Confusion Matrix KNN GridSearch CV (SMOTE)

Berdasarkan Gambar 4.12 tersebut diperoleh hasil sebagai berikut:

1) Kelas prediksi dengan label 0 menghasilkan klasifikasi sesuai sebanyak 8 data, serta klasifikasi yang tidak sesuai sebanyak 68 data (terdiri dari 11 data kelas 1 dan 57 data kelas 2), dengan total prediksi label 0 sebanyak 76 data.
2) Kelas prediksi dengan label 1 menghasilkan klasifikasi sesuai sebanyak 88 data, serta klasifikasi yang tidak sesuai sebanyak 422 data (terdiri dari 7 data kelas 0 dan 415 data kelas 2), dengan total prediksi label 1 sebanyak 510 data.
3) Kelas prediksi dengan label 2 menghasilkan klasifikasi sesuai sebanyak 1552 data, serta klasifikasi yang tidak sesuai sebanyak 251 data (terdiri dari 16 data kelas 0 dan 235 data kelas 1), dengan total prediksi label 2 sebanyak 1803 data.

Berdasarkan hasil tersebut, jumlah keseluruhan data pengujian adalah 2389 data dengan jumlah prediksi benar sebesar 1648 data. Nilai evaluasi model berdasarkan classification report dapat dilihat pada Tabel 4.27.

Tabel 4. 27 Classification Report KNN Grid Search CV (SMOTE)

Kelas	Precision	Recall	F1-Score	Support
0	10,526%	25,806%	14,953%	31
1	17,255%	26,347%	20,853%	334
2	86,079%	76,680%	81,108%	2024
Accuracy	68,983%			
Macro Avg	37,953%	42,945%	38,971%	2389
Weighted Avg	75,476%	68,983%	71,825%	2389

Berdasarkan hasil classification report pada model Grid Search CV dengan SMOTE, diperoleh nilai accuracy sebesar 68,983%. Nilai tersebut lebih rendah dibandingkan model Non-SMOTE, namun performa model terhadap kelas minoritas mengalami peningkatan. Hal ini terlihat dari nilai macro average yang memperoleh precision sebesar 37,953%, recall sebesar 42,945%, dan F1-Score sebesar 38,971%.

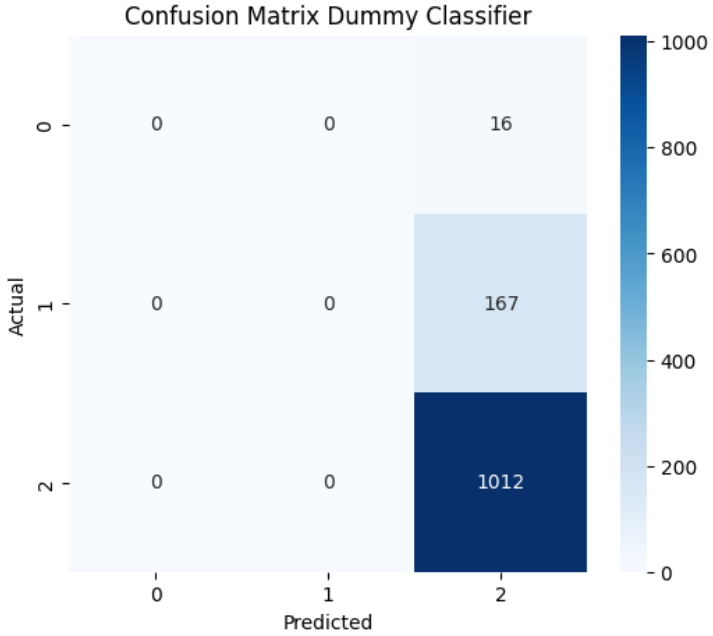
Pada kelas 0, model menghasilkan precision sebesar 10,526%, recall sebesar 25,806%, dan F1-Score sebesar 14,953%. Nilai recall yang meningkat dibandingkan model Non-SMOTE menunjukkan bahwa penerapan SMOTE membantu model dalam mengenali data pada kelas minoritas, meskipun precision masih rendah akibat banyaknya kesalahan prediksi pada kelas tersebut.

Pada kelas 1, model memperoleh precision sebesar 17,255%, recall sebesar 26,347%, dan F1-Score sebesar 20,853%. Nilai recall yang lebih tinggi menunjukkan bahwa model menjadi lebih sensitif dalam mendeteksi kelas 1 dibandingkan skenario tanpa SMOTE.

Sementara itu, pada kelas 2 diperoleh precision sebesar 86,079%, recall sebesar 76,680%, dan F1-Score sebesar 81,108%. Nilai recall pada kelas mayoritas mengalami penurunan dibandingkan model Non-SMOTE karena model menjadi lebih fokus dalam mengenali kelas minoritas setelah dilakukan penyeimbangan data menggunakan SMOTE.

5. Model KNN Dummy Baseline

Pengujian dilakukan pada model *Dummy Baseline Classifier* menggunakan data tanpa penerapan SMOTE. Model ini digunakan sebagai pembanding awal (*baseline*) untuk mengetahui performa dasar klasifikasi sebelum dilakukan penerapan algoritma K-Nearest Neighbors (KNN). Evaluasi dilakukan menggunakan confusion matrix serta classification report untuk mengukur nilai accuracy, precision, recall, dan F1-Score. Hasil pengujian confusion matrix Dummy Baseline dapat dilihat pada Gambar 4.8.



Gambar 4. 8 Confusion Matrix Dummy Classifier

Berdasarkan hasil confusion matrix tersebut diperoleh hasil sebagai berikut:

1) Kelas prediksi dengan label 0 tidak menghasilkan klasifikasi yang sesuai dengan kelas aktual. Seluruh data pada kelas aktual 0 sebanyak 16 data diprediksi ke kelas 2, sehingga total prediksi label 0 sebanyak 0 data.
2) Kelas prediksi dengan label 1 juga tidak menghasilkan klasifikasi yang sesuai dengan kelas aktual. Seluruh data pada kelas aktual 1 sebanyak 167 data diprediksi ke kelas 2, sehingga total prediksi label 1 sebanyak 0 data.
3) Kelas prediksi dengan label 2 menghasilkan klasifikasi sesuai dengan kelas aktual sebanyak 1012 data. Selain itu, terdapat klasifikasi yang tidak sesuai sebanyak 183 data, terdiri dari 16 data dari kelas 0 dan 167 data dari kelas 1, sehingga total prediksi label 2 sebanyak 1195 data.

Berdasarkan hasil tersebut, jumlah keseluruhan data adalah 1195 data, dengan jumlah prediksi yang benar (diagonal confusion matrix) sebesar 1012 data. Nilai accuracy, precision, recall, dan F1-Score dapat dilihat pada Tabel 4.28.

Tabel 4. 28 Confusion Matrix *Dummy Baseline*

Kelas	Precision	Recall	F1-Score	Support
0	0,00000	0,00000	0,00000	16
1	0,00000	0,00000	0,00000	167
2	0,84686	1,00000	0,91708	1012
accuracy	0,84686	0,84686	0,84686	0
macro avg	0,28229	0,33333	0,30569	1195
weighted avg	0,71718	0,84686	0,77664	1195

Berdasarkan hasil classification report pada model Dummy Baseline, diperoleh nilai accuracy sebesar 84,686%. Meskipun nilai accuracy terlihat tinggi, hasil tersebut tidak menunjukkan performa model yang baik dalam mengklasifikasikan seluruh kelas. Hal ini disebabkan karena model Dummy Baseline hanya memprediksi seluruh data ke dalam kelas mayoritas, yaitu kelas 2. Akibatnya, nilai precision, recall, dan F1-score pada kelas 0 dan kelas 1 bernilai 0,00000 karena tidak ada satupun data dari kedua kelas tersebut yang berhasil diprediksi dengan benar.

Pada kelas 2, model memperoleh precision sebesar 84,686%, recall sebesar 100,000%, dan F1-score sebesar 91,708%. Nilai recall sebesar 100% menunjukkan bahwa seluruh data pada kelas mayoritas berhasil diprediksi sebagai kelas 2. Namun, performa tersebut tidak diikuti oleh kemampuan model dalam mengenali kelas minoritas. Nilai macro average menunjukkan precision sebesar 28,229%, recall sebesar 33,333%, dan F1-score sebesar 30,569%. Nilai Macro F1-Score yang rendah menunjukkan bahwa model memiliki performa yang buruk secara keseluruhan ketika seluruh kelas diperlakukan secara seimbang. Sementara itu, weighted average menghasilkan nilai yang lebih tinggi karena perhitungan dipengaruhi oleh dominasi jumlah data pada kelas mayoritas.

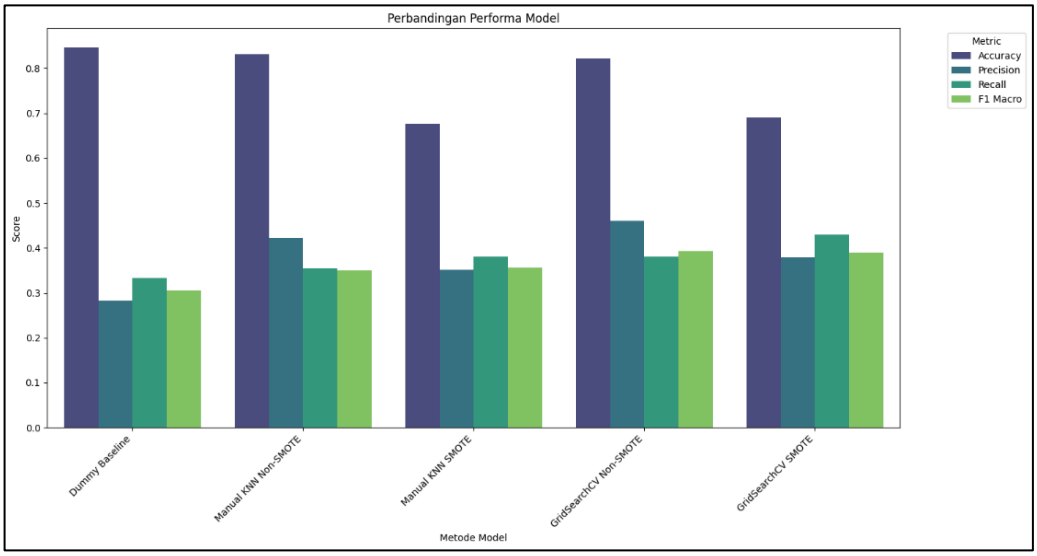
7. Perbandingan Performa Model

Setelah seluruh proses pengujian dilakukan, tahap selanjutnya adalah membandingkan performa seluruh model yang telah diuji, yaitu Dummy Baseline, Manual KNN Non-SMOTE, Manual KNN SMOTE, GridSearchCV Non-SMOTE, dan GridSearchCV SMOTE. Perbandingan dilakukan menggunakan beberapa metrik evaluasi, yaitu accuracy, precision, recall, dan F1 Macro. Hasil perbandingan performa model dapat dilihat pada Tabel 4.29 dan Gambar 4.29.

Tabel 4. 29 Perbandingan Performa seluruh model

Metode	Accuracy	Precision Macro Avg	Recall Macro Avg	F1-Score Macro Avg
Dummy Baseline	84,686%	28,229%	33,333%	30,569%
Manual KNN Non-SMOTE	83,180%	42,138%	35,457%	34,988%
Manual KNN SMOTE	67,615%	35,212%	38,098%	35,664%
GridSearchCV Non-SMOTE (Model Terbaik)	**82,190%**	**46,068%**	**38,002%**	**39,216%**
GridSearchCV SMOTE	68,983%	37,953%	42,945%	38,971%

Keterangan: *Baris yang dicetak tebal menunjukkan model yang dipilih sebagai model terbaik pada penelitian ini.*



Gambar 4. 9 Grafik Perbandingan seluruh Model

Berdasarkan Tabel 4.29, model GridSearchCV Non-SMOTE dipilih sebagai model terbaik pada penelitian ini. Meskipun nilai *accuracy* bukan yang tertinggi dibandingkan *Dummy Baseline*, model tersebut memiliki nilai *Precision Macro Average* dan *F1-Score Macro Average* yang lebih baik. Pemilihan model tidak hanya didasarkan

pada nilai *accuracy*, tetapi juga mempertimbangkan kemampuan model dalam Berdasarkan Tabel 4.29, model GridSearchCV Non-SMOTE dipilih sebagai model terbaik pada penelitian ini. Meskipun nilai *accuracy* bukan yang tertinggi dibandingkan *Dummy Baseline*, model tersebut memiliki nilai *Precision Macro Average* dan *F1-Score Macro Average* yang lebih baik. Pemilihan model tidak hanya didasarkan pada nilai *accuracy*, tetapi juga mempertimbangkan kemampuan model dalam mengklasifikasikan seluruh kelas secara seimbang. Oleh karena itu, model GridSearchCV Non-SMOTE digunakan sebagai model akhir dalam penelitian ini.mengklasifikasikan seluruh kelas secara seimbang. Oleh karena itu, model GridSearchCV Non-SMOTE digunakan sebagai model akhir dalam penelitian ini.

Berdasarkan Tabel 4.29 dan Gambar 4.7, model Dummy Baseline memperoleh accuracy tertinggi sebesar 84,686%, namun nilai F1-Score Macro hanya sebesar 30,569%. Hal ini menunjukkan bahwa model baseline hanya cenderung memprediksi kelas mayoritas dan gagal mengenali kelas minoritas. Pada confusion matrix Dummy Baseline terlihat bahwa seluruh data diprediksi sebagai kelas 2, sehingga precision, recall, dan F1-score untuk kelas 0 dan kelas 1 bernilai 0%. Oleh karena itu, meskipun accuracy terlihat tinggi, model baseline tidak dapat digunakan sebagai model klasifikasi yang baik pada dataset tidak seimbang.

Pada model Manual KNN Non-SMOTE diperoleh accuracy sebesar 83,180% dengan F1-Score Macro sebesar 34,988%. Hasil ini menunjukkan bahwa model tanpa penanganan imbalance masih sangat dominan dalam memprediksi kelas mayoritas. Hal tersebut terlihat dari nilai recall kelas 1 yang hanya sebesar 2,395% serta recall kelas 0 sebesar 6,250%. Dengan demikian, model Non-SMOTE masih mengalami kesulitan dalam mengenali kelas minoritas meskipun accuracy cukup tinggi.

Setelah diterapkan SMOTE pada model Manual KNN, accuracy mengalami penurunan menjadi 67,615%, namun recall Macro meningkat menjadi 38,098% dan F1-Score Macro meningkat menjadi 35,664%. Peningkatan ini menunjukkan bahwa SMOTE membantu model dalam mengenali kelas minoritas dengan lebih baik. Hal tersebut terlihat dari peningkatan recall kelas 0 menjadi 18,750% dan recall kelas 1 menjadi 19,162%. Meskipun accuracy menurun, distribusi prediksi model menjadi lebih seimbang dibandingkan model Non-SMOTE.

Pada model GridSearchCV Non-SMOTE diperoleh accuracy sebesar 82,190% dengan F1-Score Macro sebesar 39,216%, yang merupakan salah satu nilai F1 Macro tertinggi pada skenario tanpa SMOTE. Hal ini menunjukkan bahwa optimasi parameter menggunakan Grid Search CV mampu meningkatkan keseimbangan performa model dibandingkan tuning manual Non-SMOTE. Namun demikian, model masih cenderung mendominasi prediksi pada kelas mayoritas sehingga performa kelas minoritas masih belum optimal.

Sementara itu, model GridSearchCV SMOTE menghasilkan accuracy sebesar 68,983% dengan recall Macro tertinggi sebesar 42,945% dan F1-Score Macro sebesar 38,971%. Hasil ini menunjukkan bahwa kombinasi Grid Search CV dan SMOTE mampu meningkatkan kemampuan model dalam mengenali kelas minoritas dibandingkan seluruh model lainnya. Hal tersebut terlihat dari peningkatan recall kelas 0 sebesar 25,806% dan recall kelas 1 sebesar 26,347%. Walaupun accuracy lebih rendah dibandingkan model Non-SMOTE, model ini memiliki performa yang lebih seimbang pada seluruh kelas.

4.5 Pembahasan

Penelitian ini dilakukan untuk mengoptimalkan performa algoritma K-Nearest Neighbors (KNN) dalam klasifikasi tingkat keparahan kecelakaan lalu lintas menggunakan beberapa skenario pengujian, yaitu model manual tanpa SMOTE, model manual dengan SMOTE, optimasi menggunakan Grid Search CV tanpa SMOTE, serta Grid Search CV dengan SMOTE. Dataset yang digunakan merupakan dataset sekunder dari Kaggle berjudul *Road Traffic Severity Classification* yang memiliki karakteristik data tidak seimbang (*imbalanced data*), di mana jumlah data pada kelas mayoritas jauh lebih banyak dibandingkan kelas minoritas.

Sebelum proses pemodelan dilakukan, data terlebih dahulu melalui tahap preprocessing yang meliputi penanganan missing value menggunakan *SimpleImputer* dengan strategi *most frequent*, transformasi data kategorikal, serta normalisasi menggunakan *MinMaxScaler*. Tahapan preprocessing ini bertujuan agar seluruh fitur berada pada rentang skala yang sama sehingga proses perhitungan jarak pada algoritma KNN dapat berjalan lebih optimal. Selain itu, pada beberapa skenario penelitian

diterapkan teknik SMOTE (*Synthetic Minority Oversampling Technique*) untuk menangani ketidakseimbangan data dengan cara menghasilkan data sintetis pada kelas minoritas.

Tahapan pengujian diawali dengan penentuan jumlah K-Fold terbaik menggunakan variasi K-Fold 3, 5, dan 10. Penentuan parameter terbaik dilakukan berdasarkan rata-rata nilai accuracy dan F1-Score macro. Pada skenario Non-SMOTE diperoleh K-Fold terbaik yaitu K = 3 dengan accuracy sebesar 81,116% dan F1-Score sebesar 36,437%. Sementara pada skenario SMOTE diperoleh K-Fold terbaik yaitu K = 5 dengan accuracy sebesar 56,419% dan F1-Score sebesar 34,864%. Setelah memperoleh nilai K-Fold terbaik, dilakukan pengujian jumlah K tetangga menggunakan nilai K = 3, 5, 7, dan 9. Pada skenario Non-SMOTE diperoleh K tetangga terbaik yaitu K = 5 dengan accuracy sebesar 83,251% dan F1-Score sebesar 34,575%. Sedangkan pada skenario SMOTE diperoleh K terbaik yaitu K = 3 dengan accuracy sebesar 56,419% dan F1-Score sebesar 34,864%.

Tahapan berikutnya adalah pengujian metric jarak menggunakan metric Euclidean dan Manhattan. Pada skenario Non-SMOTE diperoleh metric terbaik yaitu Euclidean dengan accuracy sebesar 83,251% dan F1-Score sebesar 34,575%. Sedangkan pada skenario SMOTE diperoleh metric terbaik yaitu Manhattan dengan accuracy sebesar 68,663% dan F1-Score sebesar 39,036%. Hasil ini menunjukkan bahwa metric Manhattan lebih efektif pada data hasil oversampling karena lebih stabil terhadap distribusi data sintetis yang dihasilkan oleh SMOTE.

Pengujian selanjutnya dilakukan pada parameter bobot (*weights*) menggunakan bobot *uniform* dan *distance*. Pada skenario Non-SMOTE diperoleh bobot terbaik yaitu *uniform* dengan accuracy sebesar 83,251% dan F1-Score sebesar 34,575%. Sedangkan pada skenario SMOTE diperoleh bobot terbaik yaitu *distance* dengan accuracy sebesar 68,805% dan F1-Score sebesar 38,953%. Penggunaan bobot *distance* pada data SMOTE membuat tetangga yang memiliki jarak lebih dekat memberikan pengaruh lebih besar terhadap proses klasifikasi sehingga performa model menjadi lebih baik dalam mengenali kelas minoritas.

Selain pengujian manual, penelitian ini juga menggunakan optimasi parameter otomatis melalui Grid Search CV. Hasil Grid Search CV pada skenario Non-SMOTE

menghasilkan parameter terbaik berupa metric Manhattan, jumlah tetangga K = 3, dan bobot *distance*. Sedangkan pada skenario SMOTE diperoleh parameter terbaik berupa metric Manhattan dan jumlah tetangga K = 3 dengan pipeline yang mengintegrasikan proses SMOTE di dalam cross validation.

Evaluasi akhir model dilakukan menggunakan confusion matrix dan classification report. Berdasarkan hasil pengujian Dummy Baseline, model hanya memprediksi seluruh data ke kelas mayoritas sehingga menghasilkan accuracy tinggi sebesar 84,686%, namun nilai precision, recall, dan F1-Score pada kelas minoritas bernilai 0%. Nilai Macro F1 sebesar 30,569% menunjukkan bahwa model baseline gagal mengenali kelas minoritas meskipun accuracy terlihat tinggi. Hal ini menunjukkan bahwa accuracy saja tidak cukup digunakan sebagai indikator performa pada data yang tidak seimbang.

Pada model Manual KNN Non-SMOTE diperoleh accuracy sebesar 83,180% dengan Macro F1 sebesar 34,988%. Hasil confusion matrix menunjukkan bahwa model masih sangat dominan memprediksi kelas mayoritas. Kelas 0 hanya memiliki recall sebesar 6,250%, sedangkan kelas 1 hanya sebesar 2,395%. Hal ini menunjukkan bahwa model Non-SMOTE masih gagal mengenali data minoritas karena distribusi data yang tidak seimbang membuat model lebih fokus pada kelas mayoritas.

Setelah diterapkan SMOTE, performa model mengalami perubahan. Model Manual KNN SMOTE menghasilkan accuracy sebesar 67,615% dengan Macro F1 sebesar 35,664%. Meskipun accuracy menurun, kemampuan model dalam mengenali kelas minoritas meningkat. Recall kelas 0 meningkat menjadi 18,750% dan recall kelas 1 meningkat menjadi 19,162%. Hasil ini menunjukkan bahwa SMOTE berhasil meningkatkan sensitivitas model terhadap kelas minoritas meskipun harus mengorbankan sebagian accuracy pada kelas mayoritas.

Pada model Grid Search CV Non-SMOTE diperoleh accuracy sebesar 82,190% dengan Macro F1 sebesar 39,216%. Model ini menghasilkan performa yang lebih seimbang dibandingkan model manual Non-SMOTE. Recall kelas minoritas mengalami peningkatan meskipun masih relatif rendah, yaitu 9,804% untuk kelas 0 dan 8,797% untuk kelas 1. Hal ini menunjukkan bahwa optimasi parameter menggunakan Grid Search CV mampu meningkatkan keseimbangan performa model dibandingkan tuning manual tanpa SMOTE.

Sementara itu, model Grid Search CV SMOTE menghasilkan accuracy sebesar 68,983% dengan Macro F1 sebesar 38,971%. Recall kelas minoritas meningkat menjadi 25,806% pada kelas 0 dan 26,347% pada kelas 1. Hasil ini menunjukkan bahwa kombinasi Grid Search CV dan SMOTE mampu meningkatkan kemampuan model dalam mendeteksi kelas minoritas secara lebih baik dibandingkan model Non-SMOTE. Namun demikian, peningkatan recall tersebut menyebabkan penurunan accuracy karena model menjadi lebih sensitif terhadap seluruh kelas.

Berdasarkan seluruh hasil pengujian, dapat disimpulkan bahwa seluruh model yang dibangun masih memiliki keterbatasan dalam mengklasifikasikan kelas minoritas secara optimal. Hal ini terlihat dari nilai Macro F1-Score yang masih berada pada rentang 30% hingga 39%. Kondisi tersebut menunjukkan bahwa meskipun model mampu memperoleh accuracy yang cukup tinggi, performa klasifikasi pada seluruh kelas belum sepenuhnya seimbang akibat pengaruh ketidakseimbangan data.

Model dengan accuracy tertinggi bukan berarti menjadi model terbaik karena pada data tidak seimbang diperlukan evaluasi menggunakan Macro F1 agar performa seluruh kelas dapat diukur secara lebih adil. Pada penelitian ini, model Dummy Baseline bahkan menghasilkan accuracy tertinggi sebesar 84,686%, namun gagal mendeteksi kelas minoritas sehingga tidak dapat dianggap sebagai model yang baik.

Secara keseluruhan, penerapan SMOTE terbukti memberikan dampak positif terhadap peningkatan kemampuan model dalam mengenali kelas minoritas. Hal ini terlihat dari peningkatan nilai recall dan Macro F1-Score pada model SMOTE dibandingkan model Non-SMOTE. Selain itu, penggunaan Grid Search CV juga membantu menghasilkan kombinasi parameter yang lebih optimal sehingga performa model menjadi lebih seimbang.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan seluruh rangkaian pengujian dan analisis yang telah dilakukan pada penelitian ini, maka dapat diperoleh beberapa kesimpulan sebagai berikut:

1. Penelitian ini menunjukkan bahwa permasalahan ketidakseimbangan data (imbalanced data) sangat mempengaruhi performa model klasifikasi K-Nearest Neighbors (KNN). Model tanpa penanganan ketidakseimbangan data cenderung lebih fokus memprediksi kelas mayoritas sehingga menghasilkan nilai accuracy yang tinggi, namun memiliki kemampuan yang rendah dalam mendeteksi kelas minoritas. Hal ini terlihat pada model Non-SMOTE yang memperoleh accuracy tinggi, tetapi nilai recall dan F1-Score pada kelas minoritas masih rendah.
2. Berdasarkan hasil pengujian parameter, diperoleh parameter terbaik pada model Manual KNN Non-SMOTE yaitu K-Fold = 3, jumlah tetangga K = 5, metric = Euclidean, dan weight = uniform. Sedangkan pada model Manual KNN SMOTE diperoleh parameter terbaik yaitu K-Fold = 5, jumlah tetangga K = 3, metric = Manhattan, dan weight = distance. Perbedaan parameter terbaik tersebut menunjukkan bahwa perubahan distribusi data akibat penerapan SMOTE mempengaruhi proses pencarian parameter optimal pada algoritma KNN.
3. Penggunaan *Grid Search Cross Validation* (*Grid Search CV*) mampu membantu proses optimasi parameter secara otomatis dengan mengevaluasi berbagai kombinasi nilai *k-nearest neighbors*, *distance metric*, dan *weight*. Hasil optimasi menunjukkan bahwa parameter terbaik pada setiap skenario tidak selalu sama, baik pada dataset Non-SMOTE maupun SMOTE. Hal ini menunjukkan bahwa karakteristik distribusi data memengaruhi pemilihan kombinasi parameter yang paling optimal dalam algoritma *K-Nearest Neighbor* (KNN).
4. Berdasarkan hasil evaluasi menggunakan *confusion matrix* dan *classification report*, penerapan metode SMOTE memberikan kemampuan yang lebih baik dalam

mengenali kelas minoritas dibandingkan model Non-SMOTE. Hal tersebut ditunjukkan oleh peningkatan nilai *recall macro* dan *Macro F1-Score* pada model SMOTE. Meskipun nilai *accuracy* mengalami penurunan dibandingkan model Non-SMOTE, model SMOTE mampu menghasilkan performa yang lebih seimbang antar kelas sehingga lebih sesuai digunakan pada dataset yang memiliki distribusi data tidak seimbang.

5. Berdasarkan seluruh hasil pengujian dan evaluasi, model yang dipilih sebagai model terbaik pada penelitian ini adalah Grid Search Cross Validation (Grid Search CV) Non SMOTE. Model tersebut memperoleh nilai *accuracy* sebesar 82,190%, *precision macro* sebesar 46,068%, *recall macro* sebesar 38,002%, dan *Macro F1-Score* sebesar 39,216%. Pemilihan model terbaik tidak hanya didasarkan pada nilai *accuracy*, tetapi juga mempertimbangkan keseimbangan performa klasifikasi pada seluruh kelas. Nilai *recall macro* dan *Macro F1-Score* yang lebih baik menunjukkan bahwa model ini memiliki kemampuan yang lebih optimal dalam mengenali kelas mayoritas maupun kelas minoritas, sehingga lebih sesuai digunakan untuk klasifikasi tingkat keparahan kecelakaan lalu lintas dengan karakteristik data yang tidak seimbang.

5.2 Saran

Dalam penelitian ini masih terdapat beberapa keterbatasan yang dapat menjadi bahan pengembangan pada penelitian selanjutnya. Oleh karena itu, peneliti memberikan beberapa saran sebagai berikut:

1. Penelitian selanjutnya diharapkan dapat mencoba teknik penanganan ketidakseimbangan data selain SMOTE, seperti ADASYN, Borderline-SMOTE, atau metode undersampling, guna mengetahui metode yang paling optimal dalam meningkatkan performa model pada data kecelakaan lalu lintas.
2. Penelitian berikutnya disarankan untuk menggunakan atau membandingkan dengan metode klasifikasi lain, seperti Decision Tree, Random Forest, Support Vector Machine (SVM), maupun pendekatan berbasis Deep Learning, sehingga dapat

diketahui model mana yang paling efektif dalam memprediksi tingkat keparahan kecelakaan.

3. Pengembangan penelitian juga dapat dilakukan dengan menambahkan teknik optimasi parameter lain selain Grid Search, seperti Random Search atau Bayesian Optimization, agar diperoleh kombinasi parameter yang lebih optimal dan efisien.
4. Penelitian selanjutnya diharapkan dapat menggunakan dataset yang lebih besar dan lebih beragam, serta menambahkan fitur-fitur lain yang relevan, seperti kondisi cuaca, waktu kejadian, dan faktor lingkungan, sehingga dapat meningkatkan akurasi dan generalisasi model

DAFTAR PUSTAKA

1. Abrori, B. A. (2025). Analisis Perbandingan Kinerja Algoritma Klasifikasi Di Data Deals Dengan Menggunakan Metode K-Fold Cross Validation. *JUTIF: Jurnal Teknologi Informasi*, *2*(1). Retrieved from https://ojs.unpari.ac.id/jutif/article/view/468
2. Adenia, R., Minarno, A. E., Azhar, Y. (2024). Implementasi Convolutional Neural Network Untuk Ekstraksi Fitur Citra Daun Dalam Kasus Deteksi Penyakit Pada Tanaman Mangga Menggunakan Random Forest. *Jurnal Repositor*, *4*(4). https://doi.org/10.22219/repositor.v4i4.32287
3. Aldean, M. Y., Paradise, P., Nugraha, N. A. S. (2022). Analisis Sentimen Masyarakat Terhadap Vaksinasi Covid-19 di Twitter Menggunakan Metode Random Forest Classifier (Studi Kasus: Vaksin Sinovac). *Journal of Informatics Information System Software Engineering and Applications (INISTA)*, *4*(2), 64–72. https://doi.org/10.20895/inista.v4i2.575
4. Anitasari, D., Yulianto, F., Siswa, T. A. Y. (2025). KlasifikasiKecelakaan Lalu Lintas di kota SamarindaMenggunakan Algoritma K-Nearest Neighbor. *Jurnal Mnemonic*, *8*(1), 33–39. https://doi.org/10.36040/mnemonic.v8i1.12747
5. Bekelcho, T., Olani, A. B., Woldemeskel, A., Alemayehu, M., Guta, G. (2023). Identification of determinant factors for crash severity levels occurred in Addis Ababa City, Ethiopia, from 2017 to 2020: using ordinal logistic regression model approach. *BMC Public Health*, *23*(1), 1884. https://doi.org/10.1186/s12889-023-16785-3
6. Bijl, E. van de, Klein, J., Pries, J., Bhulai, S., Hoogendoorn, M., Mei, R. van der. (2022). *The Dutch Draw: Constructing a Universal Baseline for Binary Prediction Models* (arXiv:2203.13084). arXiv. https://doi.org/10.48550/arXiv.2203.13084
7. Candra, A. P. (2025). Analisis Data Menggunakan Python: Memperkenalkan Pandas dan NumPy. *Journal of Information System and Education Development*, *3*(1), 11–16. https://doi.org/10.62386/jised.v3i1.118
8. Fauzan, B. L., Agustin, T., Mahmudah, A. M. H. (2024). Prediksi Klasifikasi Kecelakaan Lalu Lintas di Kota Surakarta dengan Menggunakan Metode Regresi Logistik Multinomial. *Sustainable Civil Building Management and Engineering Journal*, *1*(4), 9–9. https://doi.org/10.47134/scbmej.v1i4.3159
9. Febrianto, M. F., Priyatno, A., Adisty, H., Saputri, A. F., Amanullah, R., Krissella, T. P., Irzavika, N., Matondang, N. H. (2024). Prediksi Situasi Lalu Lintas Menggunakan Machine Learning Dengan Algoritma K-Nearest Neighbors Classifier. *Informatik : Jurnal Ilmu Komputer*, *20*(1), 28–34. https://doi.org/10.52958/iftk.v20i1.7042
10. Firdaus, A. (2022). Aplikasi Algoritma K-Nearest Neighbor pada Analisis Sentimen Omicron Covid-19. *Jurnal Riset Statistika*, 85–92. https://doi.org/10.29313/jrs.v2i2.1148
11. Hafid, H. (2023). *Penerapan K-Fold Cross Validation untuk Menganalisis Kinerja Algoritma K-Nearest Neighbor Pada Data Kasus Covid-19 di Indonesia | Hafid | Journal of Mathematics Computations and Statistics*. https://doi.org/https://doi.org/10.35580/jmathcos.v6i2.53043

12. Hariadhy, R. P., Sutoyo, E., Pratiwi, O. (2021). *Application of K-Nearest Neighbor Algorithm for Prediction of Television Advertisement Rating* (pp. 82–91). https://doi.org/10.1007/978-3-030-80216-5_7
13. Himawan, S. N., Sohiburoyyan, R., Iryanto, I. (2023). Hyperparameter Tuning on Graph Neural Network for the Classification of SARS-CoV-2 Inhibitors. *Journal of Applied Informatics and Computing*, *7*(2), 186–191. https://doi.org/10.30871/jaic.v7i2.6735
14. Ichwani, A., Kesuma, R. I., Setiawan, A., Wicaksono, I. E., Hanifah, R. (2026). Preventing Data Leakage in Classification via Integrated Machine Learning Pipelines: Preprocessing, Feature Transformation, and Hyperparameter Tuning. *Jurnal Teknik Informatika (Jutif)*, *7*(1), 391–410. https://doi.org/10.52436/1.jutif.2026.7.1.5490
15. Irmanda, H., Astriratma, R. (2020). Klasifikasi Jenis Pantun Dengan Metode Support Vector Machines (SVM). *Jurnal RESTI (Rekayasa Sistem Dan Teknologi Informasi)*, *4*, 915–922. https://doi.org/10.29207/resti.v4i5.2313
16. Koziarski, M., Bellinger, C., Woźniak, M. (2021). RB-CCR: Radial-Based Combined Cleaning and Resampling algorithm for imbalanced data classification. *Machine Learning*, *110*(11), 3059–3093. https://doi.org/10.1007/s10994-021-06012-8
17. Kumar, M., Bhardwaj, V. (2025). Evaluating Label Encoding and Preprocessing Techniques for Breast Cancer Prediction Using Machine Learning Algorithms. *International Journal of Computational Intelligence Systems*, *18*(1), 218. https://doi.org/10.1007/s44196-025-00957-7
18. Kurniawati, Y. E. (2019). Class Imbalanced Learning Menggunakan Algoritma Synthetic Minority Over-sampling Technique – Nominal (SMOTE-N) pada Dataset Tuberculosis Anak. *Jurnal Buana Informatika*, *10*(2), 134–143. https://doi.org/10.24002/jbi.v10i2.2441
19. Lindawati, L., Fadhli, M., Wardana, A. S. (2023). Optimasi Gaussian Naïve Bayes dengan Hyperparameter Tuning dan Univariate Feature Selection dalam Prediksi Cuaca. *Edumatic: Jurnal Pendidikan Informatika*, *7*(2), 237–246. https://doi.org/10.29408/edumatic.v7i2.21179
20. Mega Sari, D., Lestari, F. (2024). *FAKTOR PENYEBAB KECELAKAAN LALU LINTAS DAN KLASIFIKASI CEDERA KORBAN DI UNIVERSITAS X | Jurnal Kesehatan Tambusai*. Retrieved from https://journal.universitaspahlawan.ac.id/index.php/jkt/article/view/38040
21. Nababan, A. H., Hutagalung, M. Y. (2023). Hyperparameter Tuning pada Model Stance Detection Menggunakan GridSearchCV. *Jurnal Sains Dan Teknologi*, *5*(1), 205–209.
22. Purnomo, H., Gonsalves, T., Mailoa, E., Santoso, F. J., Pribadi, M. R. (2024). Metaheuristics Approach for Hyperparameter Tuning of Convolutional Neural Network. *Jurnal RESTI (Rekayasa Sistem Dan Teknologi Informasi)*, *8*(3), 340–345. https://doi.org/10.29207/resti.v8i3.5730
23. Putri, S. K. N. A., Jumiatin, I., Sulistia, I., Saputra, N. A. B., Wiranda, N. (2025). Penerapan Hyperparameter Tuning pada Model Klasifikasi untuk Prediksi Risiko Penyakit Jantung : Implementation of Hyperparameter Tuning for Classification Models in Heart Disease Risk Prediction. *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, *5*(4), 1181–1189. https://doi.org/10.57152/malcom.v5i4.2138

24. Rahmawati, L., Rifki Arrosid, M., Gugus Azhari, M. (2023). Pemanfaatan Teknologi Computer Vision Untuk Implementasi Deteksi Masker Menggunakan Metode Supervised Learning. *Jurnal JEETech*, *4*(2), 75–80. https://doi.org/10.32492/jeetech.v4i2.4201
25. Rahmi, F., Yanuar, F., Asdi, Y. (2024). Penggunaan Metode Support Vector Machine (SVM) dalam Mengidentifikasi Tingkat Keparahan Pada Kecelakaan Lalu Lintas. *Statistika*, *24*(1), 1–10. https://doi.org/10.29313/statistika.v24i1.1690
26. Ramadhani, M. F., Siswa, T. A. Y., Pranoto, W. J. (2025). Penerapan Metode Forward Selection dan ADASYN Pada Algoritma SVM Untuk Klasifikasi Data Kecelakaan Lalu Lintas. *Buffer Informatika*, *11*(2), 25–34. https://doi.org/10.25134/buffer.v11i2.472
27. Salsabila, C. A., Yulianto, F., Siswa, T. A. Y. (2025). IMPLEMENTASI METODE NAIVE BAYES UNTUK KLASIFIKASI KECELAKAAN LALU LINTAS DI KOTA SAMARINDA. *Jurnal Informatika Dan Teknik Elektro Terapan*, *13*(1). https://doi.org/10.23960/jitet.v13i1.5890
28. Salsabila, N. A. (2023). Klasifikasi Tingkat Keparahan Korban Kecelakaan Lalu Lintas Di Kota Samarinda Menggunakan Algoritma K-Nearest Neighbor dan Naive Bayes. *EKSPONENSIAL*, *14*(2), 99. https://doi.org/10.30872/eksponensial.v14i2.1085
29. Sarker, I. H. (2021). Machine Learning: Algorithms, Real-World Applications and Research Directions. *SN Computer Science*, *2*(3), 160. https://doi.org/10.1007/s42979-021-00592-x
30. Setiawan, A. (2022). Perbandingan Penggunaan Jarak Manhattan, Jarak Euclid, dan Jarak Minkowski dalam Klasifikasi Menggunakan Metode KNN pada Data Iris. *Jurnal Sains Dan Edukasi Sains*, *5*(1), 28–37. https://doi.org/10.24246/juses.v5i1p28-37
31. Sieg, J., Feldmann, C. W., Hemmerich, J., Stork, C., Sandfort, F., Eiden, P., Mathea, M. (2024). MolPipeline: A Python Package for Processing Molecules with RDKit in Scikit-learn. *Journal of Chemical Information and Modeling*, *64*(24), 9027–9033. https://doi.org/10.1021/acs.jcim.4c00863
32. Sinaga, R. M., Fauzan, M. R., Pratama, E., Alfahri, M. R., Saputra, K. (2025). Pemanfaatan Teknik Machine Learning dalam Memprediksi Kepadatan Lalu Lintas Guna Efisiensi Transportasi. *PROSISKO: Jurnal Pengembangan Riset Dan Observasi Sistem Komputer*, *12*(2). https://doi.org/10.30656/prosisko.v12i2.10453
33. Sulistiyono, M., Pristyanto, Y., Adi, S., Gumelar, G. (2021). Implementasi Algoritma Synthetic Minority Over-Sampling Technique untuk Menangani Ketidakseimbangan Kelas pada Dataset Klasifikasi. *Sistemasi: Jurnal Sistem Informasi*, *10*(2), 445–459. https://doi.org/10.32520/stmsi.v10i2.1303
34. Surahman, G., Sukma, I. F. D., Kustanrika, I. W. (2025). KAJIAN KARAKTERISTIK KECELAKAAN LALU LINTAS DI KABUPATEN JEMBER. *Jurnal Riset Teknik Sipil Dan Sains*, *4*(1), 43–48. https://doi.org/10.57203/jriteks.v4i1.2025.43-48
35. Suryati, M., Saputra, T. A. D., Ikasari, I. H. (2025). Penggunaan Python Dalam Analisis Data Dengan Machine Learning. *JRIIN :Jurnal Riset Informatika Dan Inovasi*, *2*(10), 1959–1968.
36. Susanti, B. S., Hirzi, R. H., Wulandhya, S. A., Hastuti, S. H., Chintyana, A. (2024). Analisis Klasifikasi Kecelakaan Lalu Lintas Lombok Timur Berdasarkan Tingkat

Keparahan Korban Kecelakaan Menggunakan Metode Support Vector Machine (SVM) dan Bootstrap Aggregating (Bagging). *Jurnal Eksbar*, *1*(1), 1–8. https://doi.org/10.29408/eksbar.v1i1.27123
37. Syafarina, G. A., Zaenuddin, Z. (2023). Implementasi Framework Streamlit Sebagai Prediksi Harga Jual Rumah Dengan Linear Regresi. *METIK Jurnal*, *7*(2), 121–125. https://doi.org/10.47002/metik.v7i2.608
38. Syahriza, M. (2019). Kecelakaan Lalulintas: Perlukah Mendapatkan Perhatian Khusus?. *Averrous: Jurnal Kedokteran dan Kesehatan Malikussaleh*, 89-101. https://doi.org/10.29103/averrous.v5i2.2083
39. Toha, A., Purwono, P., Gata, W. (2022). Model Prediksi Kualitas Udara dengan Support Vector Machines dengan Optimasi Hyperparameter GridSearch CV. *Buletin Ilmiah Sarjana Teknik Elektro*, *4*(1), Article 1. https://doi.org/10.12928/biste.v4i1.6079
40. Uddin, S., Haque, I., Lu, H., Moni, M. A., Gide, E. (2022). Comparative performance analysis of K-nearest neighbour (KNN) algorithm and its different variants for disease prediction. *Scientific Reports*, *12*(1), 6256. https://doi.org/10.1038/s41598-022-10358-x
41. Wang, H., Xu, P., Zhao, J. (2021). Improved KNN algorithms of spherical regions based on clustering and region division. *Alexandria Engineering Journal*, *61*. https://doi.org/10.1016/j.aej.2021.09.004
42. Yaro, A., Maly, F., Pražák, P. (2023). Outlier Detection in Time-Series Receive Signal Strength Observation Using Z-Score Method with Sn Scale Estimator for Indoor Localization. *Applied Sciences*, *13*, 3900. https://doi.org/10.3390/app13063900
43. Yumak, A., Hengirmen Tercan, S., Colak, U. C., Ozcanan, S. (2025). A Machine Learning Approach to Identify High-Risk Road Segments and Accident Severity Patterns Based on Categorical Data. *Applied Sciences*, *15*(23), 12824. https://doi.org/10.3390/app152312824

LAMPIRAN

Lampiran 1 Arsip Keseluruhan

Link Arsip : https://github.com/alpiashari/2209106017_Skripsi